

NESNEYE DAYALI TASARIM VE MODELLEME

Ekim 2024

Dr.Öğr.Üyesi Yunus Emre SELÇUK

GENEL BİLGİLER

DERS İÇERİĞİ

- Kısım 1: Gang of Four Tasarım Kalıpları
- Kısım 2: Code Smells and Refactoring (Birim sınamaları ile)

KAYNAKLAR

- Kısım 1:
 - Design Patterns – Elements of Reusable OO Software, Erich Gamma et.al (Gang of Four), Addison-Wesley, 1994
 - Ek kaynaklardan da çalışılması yararlı olacaktır (sonraki yansı)
- Kısım 2:
 - Refactoring: Improving the Design of Existing Code, Martin Fowler. Addison-Wesley, 1999

1

NESNEYE DAYALI TASARIM VE MODELLEME

KAYNAK KİTAPLAR

- Ek kaynaklar (Elektronik ortamda bulunuyorlar):
 - Design Patterns Explained: A New Perspective on Object-Oriented Design, Alan Shalloway and James R. Trott. Addison-Wesley, 2004 (2nd Ed.)
 - Head First Design Patterns, Elisabeth Freeman et.al, O'Reilly, 2004
 - Design Patterns (Wordware Applications Library), Christopher G. Lasater. Jones & Bartlett Publishers, 2006 (1st ed.)
 - Design Patterns Java Workbook, S. John Metsker, Addison-Wesley, 2002
 - Applied Java Patterns, S. Stelting and O. Maassen, Prentice Hall, 2002
 - ... ve sizin bulup bana önereceğiniz kaynaklar

GEREKSİNİMLER

- Herhangi bir güncel NYP diline hakimiyet
 - Derste kod örnekleri java dili üzerinden verilecektir.
- UML sınıf şemalarına hakimiyet
 - Bu konuda çok eksik öğrenciler olabiliyor. Lütfen birinden inceleyiniz:
 - UML Distilled, 3rd ed. (2003), Martin Fowler, Addison-Wesley.
 - UML 2.0 in a Nutshell, Dan Pilone and Neil Pitman, O'Reilly.
 - Vb.

2

NESNEYE DAYALI TASARIM VE MODELLEME

DEĞERLENDİRME

- Küçük sınav: %0, UML sınıf şemalarına ve nesneye yönelime hakimiyetinizi görmek amaçlı, soru ve cevapları ders notları ile paylaşılacak.
- 1. Ara sınav: %30, Tasarım kalıpları konulu, 8. hafta (21 Kasım 2024)
- 2. Ara sınav: %30, Tasarım kalıpları konulu, 12. hafta (19 Aralık 2024) (seçimlik, 1. ara sınavdan yüksek not alanlar 2. ara sınava girmezlerse 1. ara sınav notları %60 ağırlıkla değerlendirilecektir).
- Vize haftalarında değişiklikler olabilir, bölüm web sayfasından ve hocanın sayfasından duyuruları izleyiniz.
- Ara sınav telafi: 15. hafta (9 Ocak 2024) (resmi mevzuata sıkı sıkıya uyan bir mazerete sahip olanlar katılabilir).
- Final sınavı: %40, Refactoring konulu, Final haftasında
- Bütünleme sınavı: Final sınavı telafisidir. Bütünleme haftasında.
- Not verilen tüm sınavlar yüz yüze yapılacaktır.

3

NESNEYE DAYALI TASARIM VE MODELLEMeye GİRİŞ

YAZILIM KALİTESİ

- 'İyi' tasarım ile 'kaliteli' yazılıma ulaşmak amaçlanır.
 - Problem alanı hangi düzeyde parçalara ayrılabiliriyorsa, o düzeyde parçalara ayrılabilen bir tasarım da 'iyi' olarak nitelendirilebilir.
- Ölçütler:
 - Dış kalite özellikleri (kullanıcıya yönelik)
 - İç kalite özellikleri (programcıya yönelik)
- Dış kalite ölçütleri:
 - Doğruluk, etkinlik, kolaylık, ...
- İç kalite özellikleri:
 - Yeniden kullanılabilirlik
 - İlgi alanlarının ayrılması
 - Esneklik
 - Taşınabilirlik
 - Okunabilirlik ve anlaşılabilirlik
 - Sınanabilirlik
 - Bakım kolaylığı
 - ...

4

NESNEYE DAYALI TASARIM VE MODELLEMeye GİRİŞ

YAZILIM KALİTESİ

- Neden 'iyi' bir tasarım?
 - Yazılım projeleri önemli oranda başarısızlığa uğramaktadır.
 - Etken sayısı şüphesiz fazladır, ancak 'tasarımın iyiliği' denetlenebilen bir etkindir.
 - Oluşan bir hatayı düzeltmenin bedeli, yazılım hayat döngüsünde ilerlendikçe üstel olarak artmaktadır.
 - Yazılım geliştirme çabalarının çoğu, bakım aşamasında harcanmaktadır.
- İyi bir tasarıma götüren temel ilkeler:
 - Düşük bağlaşım (Low coupling)
 - Yüksek uyum (High cohesion)
 - İlgili alanlarının ayrılması (Separation of concerns)
 - İlk iki ilke ile açıklanabilir.

5

NESNEYE DAYALI TASARIM İLKELERİ

DÜŞÜK BAĞLAŞIM – LOW COUPLING

- Bağlaşım: Bir parçanın diğer parçalara bağımlılık oranı.
 - Parça: Sınıf, alt sistem, paket
- Bağımlılık: Bir sınıfın diğerinin:
 - Hizmetlerinden yararlanması,
 - İç yapısından haberdar olması,
 - Çalışma prensiplerinden haberdar olması,
 - Özelleşmiş veya genelleşmiş hali olması (kalıtım ilişkisi).
- Diğer sınıfların sayısı arttıkça bağlaşım oranı artar.
- Düşük bağlaşımın yararları:
 - Bir sınıfta yapılan değişikliğin geri kalan sınıfların daha azını etkilemesi,
 - Yeniden kullanılabilirliğin artması

6



NESNEYE DAYALI TASARIM İLKELERİ

YÜKSEK UYUM – HIGH COHESION

- Uyum: Bir parçanın sorumluluklarının birbirleri ile uyumlu olma oranı.
- Yüksek uyumun yararları:
 - Sınıfın anlaşılma kolaylığı artar.
 - Yeniden kullanılabilirlik artar.
 - Bakım kolaylığı artar
 - Sınıfın değişikliklerden etkilenme olasılığı düşer.
- Genellikle:
 - Düşük bağlaşım getiren bir tasarım yüksek uyumu,
 - Yüksek bağlaşım getiren bir tasarım ise düşük uyumu beraberinde getirir.

7



NESNEYE DAYALI TASARIM İLKELERİ

İLGİ ALANLARININ AYRILMASI

- İlgili alanlarının ayrılması ile daha iyi bir tasarıma ulaşılması:
 - Yazılımın sadece belirli bir amaç, kavram veya hedef ile ilgili kısımlarının tanımlanabilmesi, birleştirilebilmesi ve değiştirilebilmesi yeteneğine işaret eder.
 - Sorunu parçalayarak fethet ve parçaları yeniden kullan!
 - Her programlama yaklaşımı, sorunu parçalara bölmek için yollar sunar.
 - Parçaların düşük bağlaşım ve yüksek uyuma sahip olması istenir.

8



NESNEYE DAYALI TASARIM VE MODELLEME

KISIM 1: TASARIM KALIPLARI

1.0. TASARIM KALIPLARINA GİRİŞ

9



TASARIM KALIPLARINA GİRİŞ

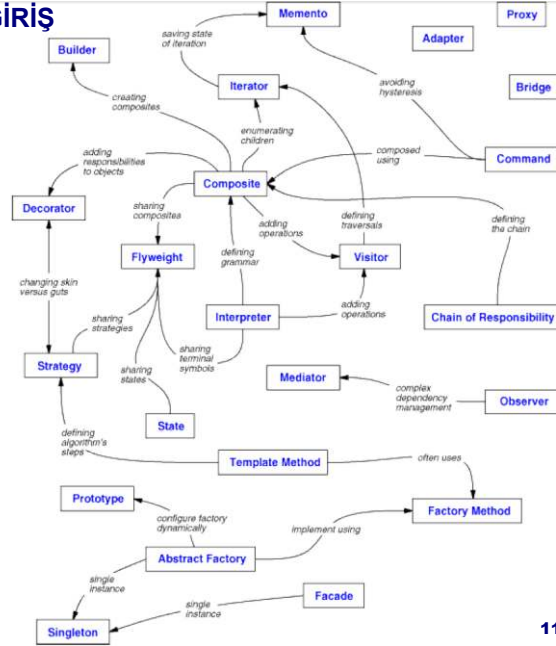
TASARIM KALIBI (DESIGN PATTERN) NEDİR?

- Yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir.
 - Çözüm somut bir gerçekleştirme olmak zorunda değildir, soyut bir tarif de olabilir.
- Amaç: Tekrarla yeniden keşfetmeden 'iyi' tasarıma ulaşmak.
 - Sorunların esnek biçimde ve yeniden kullanılabilirliği arttıracak yönde çözülmesi.
- Bir tasarım kalıbının ana bileşenleri:
 - İsim: Kalıbı en güzel şekilde anlatacak bir veya birkaç kelimelik isim.
 - Sorun: Kalıbın çözdüğü sorunun tanımı.
 - Çözüm: Sorunu çözmek için önerilen tasarımın bileşenleri; bu bileşenlerin ilişkileri, sorumlulukları, birlikte çalışmaları.
 - Tartışma/Sonuçlar: Çözümün sistemin geneline esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden yaptığı etkiler; çözümün güçlü ve zayıf yönleri, yer-zaman çelişkileri, başka kalıplar ile işbirliği olanakları, vb.

10

TASARIM KALIPLARINA GİRİŞ

- Kalıplar arasında olası ilişkiler:



11

TASARIM KALIPLARINA GİRİŞ

TASARIM KALIBI (DESIGN PATTERN) NEDİR?

- Yararları:
 - Tekerleği yeniden icat etmemek.
 - Üzerinde birçok deneyimli kişinin emeği geçtiğinden, 'kaliteli' bir çözüm.
 - Deneyimin yazılı biçimde aktarılabilmesi.
 - Yazılım ekibi arasında ortak bir sözlük oluşturmaları.
- Tartışma:
 - Bir kişinin 'kalıp' dediğine, diğeri 'basit bir yapı taşı' veya 'temel bir ilke' diyebilir.

12

TASARIM KALIPLARINA GİRİŞ

TASARIM KALIBI (DESIGN PATTERN) NEDİR?

- Ders notlarındaki kalıplar ve anlatım sırası hakkında:
 - Ders notlarında anlatılan GoF tasarım kalıpları, kaynak kitaptaki sıralamaya sadık kalınarak anlatılmıştır.
- GoF kalıpları amaçlarına göre 3 kümeye ayrılmıştır:
 - Creational: Yaratımsal. Nesnelerin oluşturulması, temsili ve ilişkilendirilmelerindeki bağımlılığı azaltmaya yönelik kalıplardır.
 - Structural: Yapısal. Sınıflar ve nesnelerin bir araya getirilerek daha büyük yapılar elde edilmesine yönelik kalıplardır.
 - Behavioral: Davranışsal. Sorumlulukların nesnelere dağıtılması ve algoritma seçimine yönelik kalıplardır.

13

TASARIM KALIPLARINA GİRİŞ

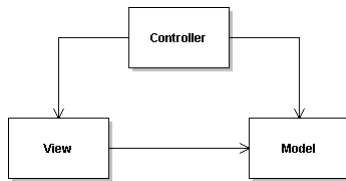
TASARIM KALIBI (DESIGN PATTERN) NEDİR?

- Ders notlarındaki kalıplar ve anlatım sırası hakkında (devam):
 - GoF kalıplarının bazılarının odağı sınıflar, bazılarının odağı ise nesnelerdir.
 - Sınıfsal kalıplar: Sınıflar ve bunların alt sınıfları arasındaki ilişkilere yönelik kalıplardır. Sınıf yapısı ve kalıtım ilişkisinin doğası gereği derleme zamanında belirlenen ve daha durağan ilişkilerdir.
 - Nesnesel kalıplar: Çeşitli türden nesneler arasındaki ilişkilere yönelik kalıplardır. Kodun çalışması süresince rahatlıkla değiştirilebilen ve daha dinamik ilişkilerdir.
 - Adapter kalıbı GoF tarafından hem sınıfsal hem de nesnesel bir kalıp olarak değerlendirilmiştir. Diğer kalıplar bu iki gruptan sadece birine dahil edilmiştir.

14

ÖRNEK TASARIM KALIBI: MODEL-VIEW-CONTROLLER (MVC)

- Sorun:
 - Aynı veriyi farklı biçimlerde gösterebilmek istiyoruz.
 - Kullanıcıya, verinin nasıl ve ne kadarının gösterileceğini seçebilme gibi olanaklar sağlamak istiyoruz.
- Çözüm:
 - Veriyi, verinin gösterimini ve gösterimin yönetimini ayrı sınıflar şeklinde ele almak.



- Model sınıfı:
 - Ham veriyi simgeler.
- View sınıfı:
 - Verinin çeşitli gösterim biçimleri.
- Controller sınıfı:
 - Kullanıcı komutlarını alıp işler.

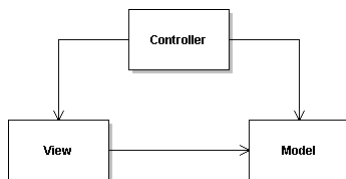
- Benzetim:
 - Bir ressam ve çizdiği insan.
 - Başka?

15

TASARIM KALIPLARINA GİRİŞ

MVC TASARIM KALIBI

- Yararlar:
 - Gösterim, kontrol ve veriyi birbirlerinden ayırarak;
 - Bunları çalışma anında değiştirebilmek.
 - İlgili alanlarının ayrılmasını sağlamak.
 - Yeniden kullanımı arttırmak.



- Çeşitlemeler:
 - Model'in geçirdiği değişiklikleri View'a bildirmesi
 - Birden fazla view:
 - İç içe (nested).
 - Ayrı/tek görevcikte (thread)
 - Örtüşen/ayrık
 - Birden fazla kontrolcü:
 - Farklı denetim biçimleri (klavye, fare, ses, ...)

16



NESNEYE DAYALI TASARIM VE MODELLEME
KISIM 1: TASARIM KALIPLARI
1.1. YARATIMSAL (CREATIONAL) KALIPLAR

17

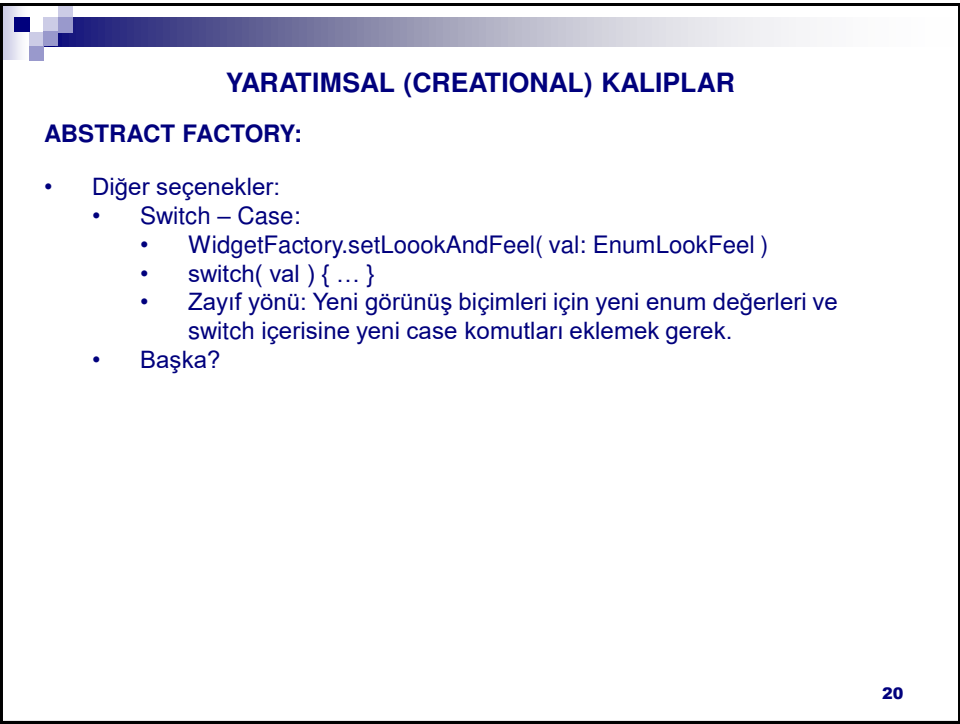
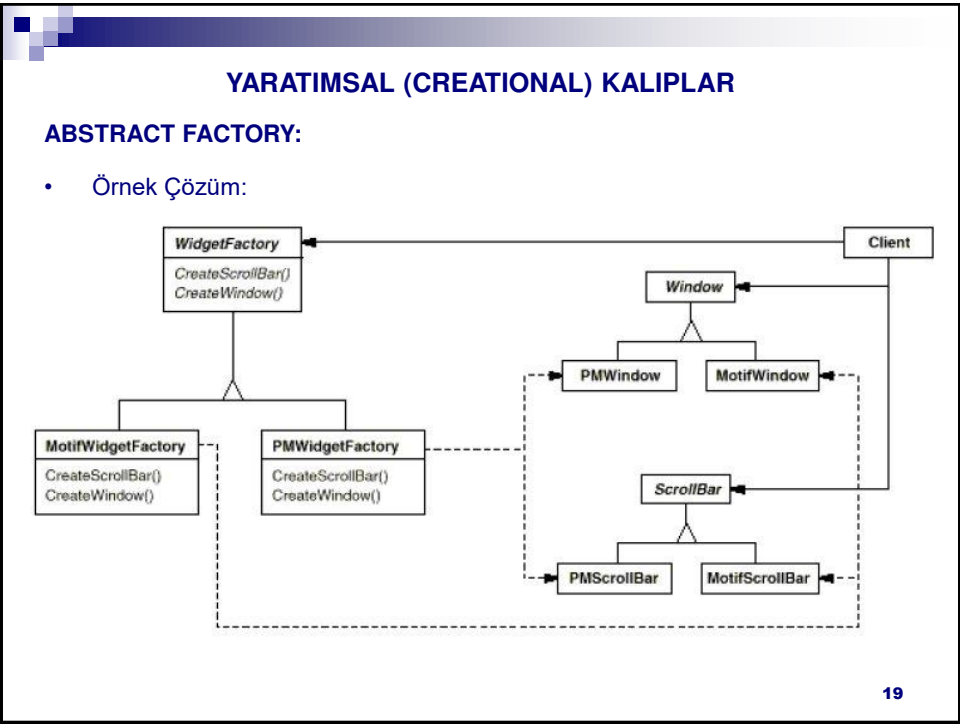


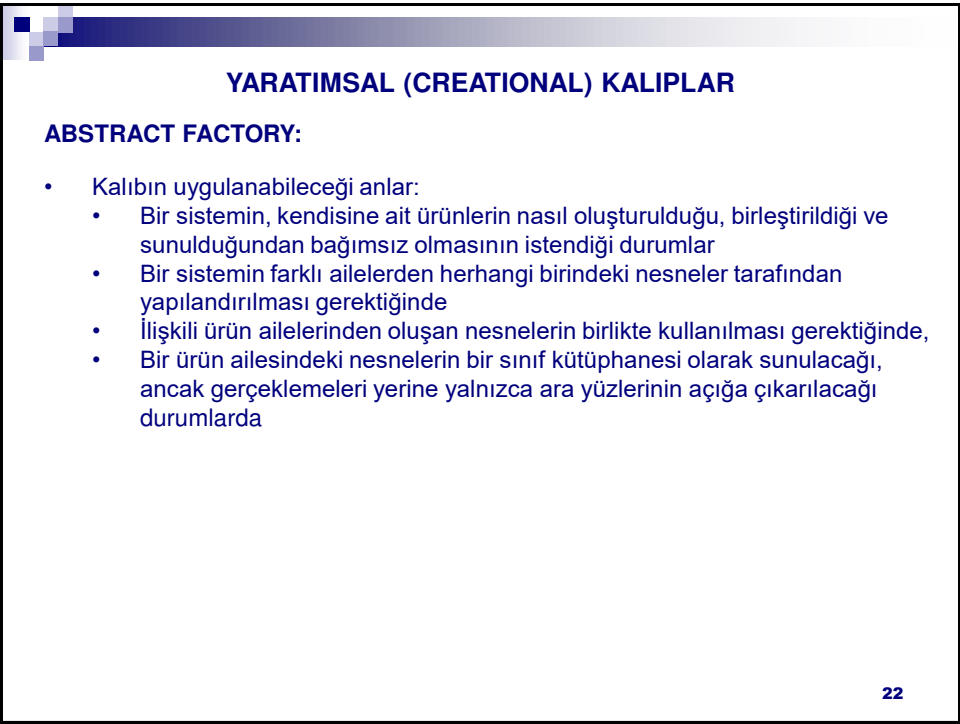
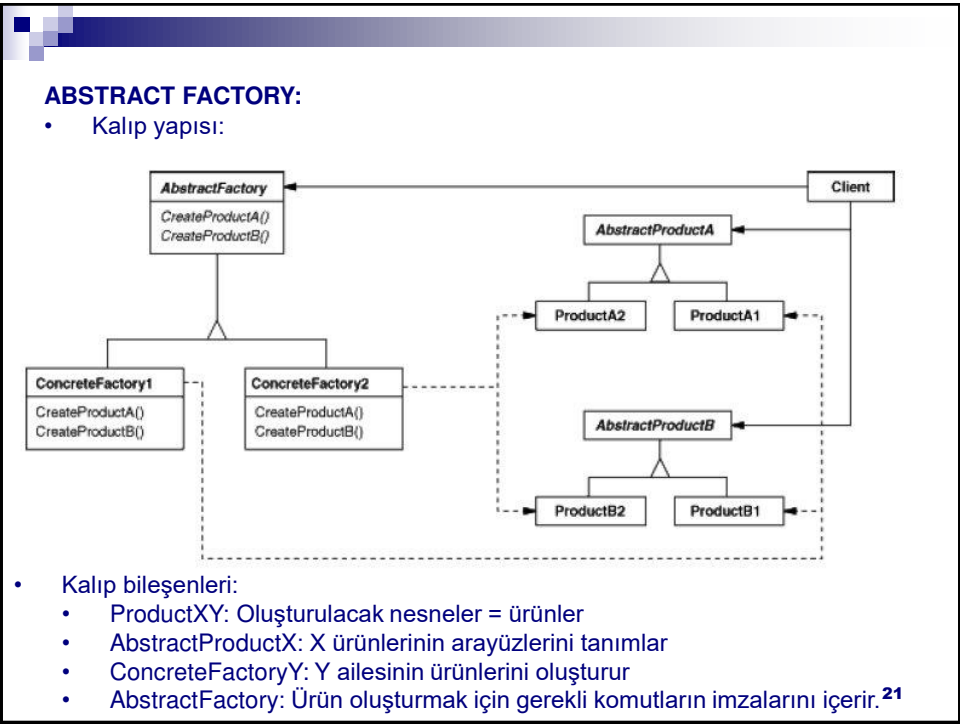
YARATIMSAL (CREATIONAL) KALIPLAR

ABSTRACT FACTORY

- Amaç:
 - Birbirleri ile ilişkili ya da aynı aileden olan nesneleri, sınıflarını belirtmeden oluşturabilmek.
- Örnek:
 - Java’da çeşitli GUI bileşenlerinin görünüşünün (look-and-feel) çalışma anında değiştirilebilmesi
 - Farklı Linux dağıtımlarında Gnome, KDE gibi pencere yöneticilerinden herhangi birini kullanarak çalışılabilmenin gerekmesi
- Sorun:
 - Taşınabilirliği sağlamak için, istekçinin hangi tür görünüşün hangi sınıftan nesnelerle gerçekleştirildiğini bilmemesi gerekir.

18





ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜLECEK PROBLEM:

- Bir 3D oyun motoru hazırlıyoruz.
- Kullanılacak işletim sistemi ve ekran kartına göre OpenGL veya DirectX kütüphanesinin renderer, shader, vb bileşenleri kullanılacak.

• Kaynak: YES

23

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 1: Switch – Case Kullanımı

```
package dp.abstractFactory.solution1;
public enum LibraryType { OpenGL, DirectX; }

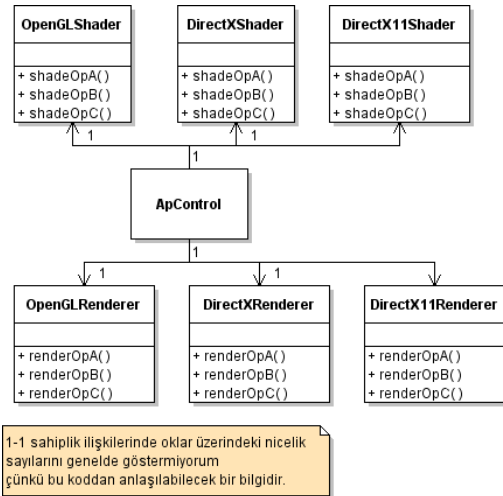
public class ApControl {
    private LibraryType library;
    private OpenGLRenderer or;    private OpenGLShader os;
    private DirectXRenderer dr;   private DirectXShader ds;
    public ApControl(LibraryType library) {this.library=library;}
    void doRendering ( ) {
        switch (library){
            case OpenGL:
                or.renderOpA(); or.renderOpB(); or.renderOpC(); break;
            case DirectX:
                dr.renderOpA(); dr.renderOpB(); dr.renderOpC(); break;
        }
    }
    void doShading ( ) {
        switch (library){
            case OpenGL:
                os.shadeOpA(); os.shadeOpB(); os.shadeOpC(); break;
            case DirectX:
                ds.shadeOpA(); ds.shadeOpB(); ds.shadeOpC(); break;
        }
    }
}
```

24

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 1: Switch – Case Kullanımı

```
public class OpenGLRenderere {
    public void renderOpA() { }
    public void renderOpB() { }
    public void renderOpC() { }
}
public class OpenGLShader {
    public void shadeOpA() { }
    public void shadeOpB() { }
    public void shadeOpC() { }
}
public class DirectXRenderere {
    public void renderOpA() { }
    public void renderOpB() { }
    public void renderOpC() { }
}
public class DirectXShader {
    public void shadeOpA() { }
    public void shadeOpB() { }
    public void shadeOpC() { }
}
```



25

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜMÜN ZAYIF YÖNLERİ:

- Yeni bir kütüphane eklemek için (Ör. DirectX 10, 11 ve 12 için ayrı kütüphaneler) çok fazla yerde kod değişikliği gerek.

TASARIM İPUCU:

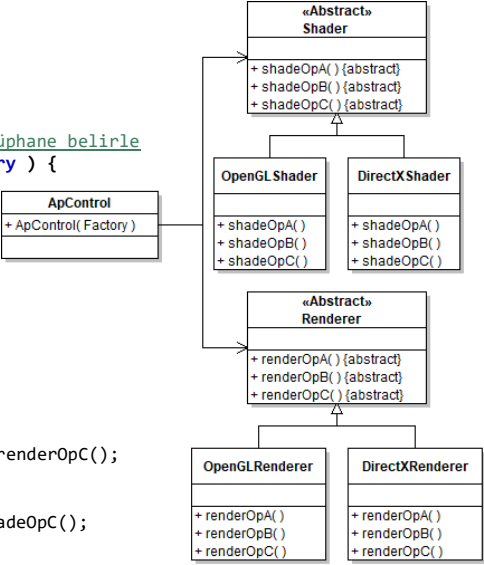
- Switch-case kullanımı çok biçimliliğe gereksinim duyulduğunun ve/veya sorumlulukların atanmasında yanlışlık olduğunun uyarı işareti olabilir.

26

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 2: Kalıtım Kullanımı

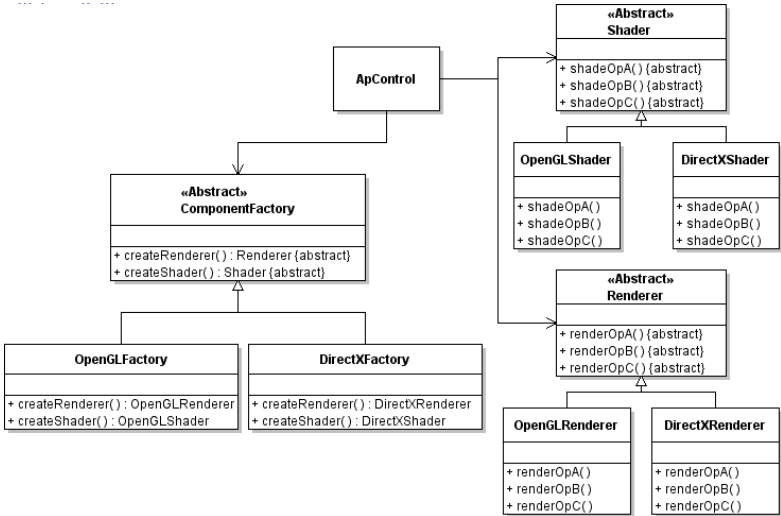
```
package dp.abstractFactory.solution2;
public class ApControl {
    private Renderer r;
    private Shader s;
    //library nesnesini ilklendirip kütüphane belirle
    public ApControl( LibraryType library ) {
        switch (library){
            case OpenGL:
                r = new OpenGLRenderer();
                s = new OpenGLShader();
                break;
            case DirectX:
                r = new DirectXRenderer();
                s = new DirectXShader();
                break;
        }
    }
    public void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    public void doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```



ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 3: Abstract Factory Kalıbının Kullanımı

- Ana program kurucusunda uygun bir ComponentFactory gerçektelemesi



ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 3: Abstract Factory Kalıbının Kullanımı

```
package dp.abstractFactory.solution3;
public class ApControl {
    private ComponentFactory library;
    private Renderer r;
    private Shader s;
    public ApControl( ComponentFactory library ) {
        this.library = library;
        /* Seçenek A: library nesnesini ilklendirerek kütüphane türünü
        * switch-case ile önceki örnekteki gibi belirle
        * Seçenek B: buradaki gibi kurucu metoda parametre olarak ver.
        * Seçenek C: Farklı tür hedef bilgisayarlar için
        * farklı dağıtımlar yap ve alttaki uygun satırın yorumunu kaldır.
        // library = new OpenGLFactory();
        // library = new DirectXFactory();
        */
        r = this.library.createRenderer();
        s = this.library.createShader();
    }
    public void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    public doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```

29

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 3: Abstract Factory Kalıbının Kullanımı

- Çözümlerdeki soyut sınıflar yerine arayüz kullanımı da mümkündür.
- Tüm çözümlerde library nesnesi ApControl kurucusuna parametre olarak verilirse daha doğru bir iş yapılmış olunur.
 - Neden? Bize ne kazandırır?

```
package dp.abstractFactory.solution3;
public class ApControl {
    private Renderer r;
    private Shader s;
    public ApControl(ComponentFactory aFactory) {
        r = aFactory.createRenderer();
        s = aFactory.createShader();
    }
    public void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    public doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```

30

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇÖZÜM 3: Abstract Factory Kalıbının Kullanımı

- Refactoring konusunu gördükten sonra, Preserve Whole Object eyleminin yürütülmüş hali tercih edilir:
 - Neden? Bize ne kazandırır?

```
package dp.abstractFactory.solution3;
public class ApControl {
    private ComponentFactory factory;
    public ApControl(ComponentFactory aFactory) {
        factory = aFactory;
    }
    void doRendering ( ) {
        factory.r.renderOpA(); factory.r.renderOpB();
        factory.r.renderOpC();
    }
    void doShading ( ) {
        factory.s.shadeOpA(); factory.s.shadeOpB();
        factory.s.shadeOpC();
    }
}
```

31

ÖRNEK KALIP GERÇEKLEMELERİ – ABSTRACT FACTORY

ÇIKARTILAN DERSLER = TASARIMA AİT GENEL KURALLAR

- Neyin değiştiğini bul ve onu sarmala
 - Böylece bu şeyi soyutlamış olursun
 - Böylece bu şeyi kara kutu yaklaşımı ile kullanabilirsin
 - Böylece bu konu ile ilgili değişiklikler sadece bu kısmı etkiler
 - Örnekte kullanılan yer: Hangi grafik kütüphanesinin kullanılacağı bir bilgisayardan başka bilgisayara geçilince değişeceğinden, bu değişim ComponentFactory sınıfında soyutlanmıştır.
- Kalıtım ilişkisi yerine parça/bütün ilişkisini tercih et
 - Kalıtım farklı bir sakıncalı biçimde kurulabilirdi: ApControl üst sınıfından OpenGLApControl ve DirectXApControl sınıfları türetilbilirdi.
 - Sonuçta ortaya benzer kodun birden fazla yerde tekrarlanması nedeniyle kusurlu bir tasarım ortaya çıkacaktı.
- Gerçeklemelere göre değil, arayüzlere göre tasarlama yap
 - Böylece kara kutu yaklaşımını kullanabilirsin
 - Örnekte ApControl sınıfı ComponentFactory sınıfından bir renderer ve bir shader oluşturmasını istiyor, bunu nasıl oluşturduğu umurunda değil.

32

YARATIMSAL (CREATIONAL) KALIPLAR

FACTORY METHOD:

- Amaç:
 - Bir nesne oluşturmak için öyle bir ara yüz sunmak ki, oluşturulacak nesnenin sınıfına bu ara yüzü sağlayan sınıfın alt sınıfları karar verebilsin.
- Örnek:
 - Farklı belge türleri ile çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program (framework) hazırlanıyor.
 - Temel sınıflar: Belge ve Uygulama.
 - Kullanıcı bu iki sınıftan kalıtım ile yeni sınıflar türeterek, kendi yazılımını hazırlıyor.
- Sorun:
 - Bir belge oluşturmak gerekince (aç, yeni, vb. komutlar) çerçeve program bu işlemi nasıl yapacak?
 - Uygulama, kullanıcının türeteceği yeni belge sınıfları hakkında önceden bilgi sahibi olamaz.

33

YARATIMSAL (CREATIONAL) KALIPLAR

FACTORY METHOD:

- Örnek Çözüm:

- Diğer seçenekler:
 - Java'da Generic sınıflar:
 - class Document<DocType>
 - Zayıf yönü: Dile özel
 - Başka?

34

YARATIMSAL (CREATIONAL) KALIPLAR

FACTORY METHOD:

- Kalıp yapısı:

```
classDiagram
    class Product {
        <<interface>>
    }
    class ConcreteProduct
    class Creator {
        <<interface>>
        FactoryMethod()
        AnOperation()
    }
    class ConcreteCreator {
        FactoryMethod()
    }
    Product <|-- ConcreteProduct
    Creator <|-- ConcreteCreator
    ConcreteCreator ..> Product : product = FactoryMethod()
    ConcreteCreator ..> ConcreteProduct : return new ConcreteProduct
```

35

YARATIMSAL (CREATIONAL) KALIPLAR

FACTORY METHOD:

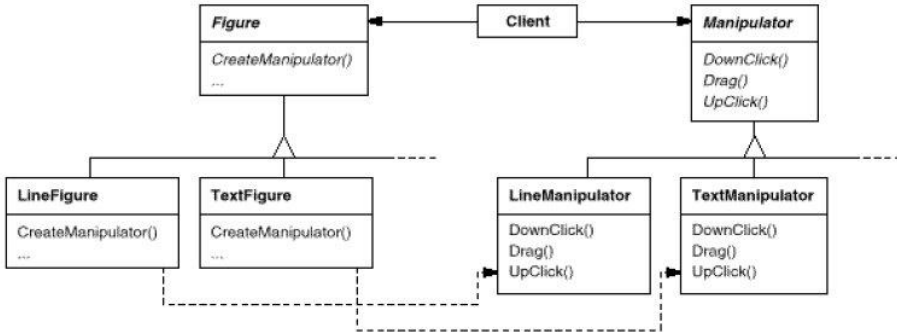
- Kalıbın uygulanabileceği anlar:
 - Yeni nesneler oluşturması gereken bir sınıfın, oluşturacağı nesnenin türünü bilemediği yerlerde,
 - Bir sınıfın kendi üzerindeki bir sorumluluğu çeşitli yardımcı alt sınıflarına ileteceği ve hangi tür yardımcı alt sınıfın oluşturulması gerektiği bilgisinin ise yerleştirilmesi gerektiği yerlerde.
 - Örnek: Bir çizim uygulamasında şekiller fare ile değiştirilecek.

36

YARATIMSAL (CREATIONAL) KALIPLAR

FACTORY METHOD:

- Örnek: Bir çizim uygulamasında şekiller fare ile değiştirilecek.



- Bazı şekil türleri özel bir değiştiriciye gerek duymuyorsa, Figure sınıfı varsayılan bir değiştirici örneği döndürebilir. Bu durumda her bir şekil türü için ayrı bir değiştirici sınıfı yazmaya gerek kalmaz.

37

ÖRNEK KALIP GERÇEKLEMELERİ – FACTORY METHOD

ÇÖZÜLECEK PROBLEM:

- E-ticaret sitesi farklı tür ürünler satışa sunabiliyor.
- Site çalışanı, bir müşterinin siparişindeki farklı tür ürünleri işleme koyacak.
 - Müşteri siparişini işleme adımları belli bir sırada yürütülüyor.
 - Ancak adımlardaki iş mantığı ürün türüne göre çok farklılık gösteriyor.
- Site çalışanına sipariş vermede kullanabileceği bir araç sağlayalım.
- Kullanım şu şekilde olabilir:

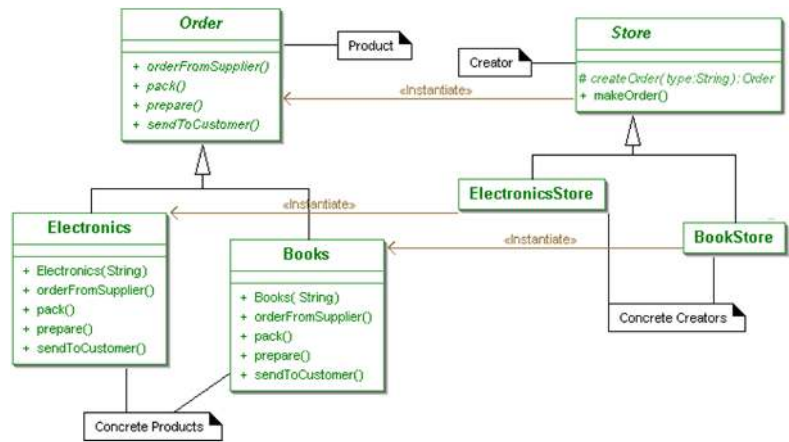
```
package factoryMethod.example;
public class MainApp {
    public static void main(String[] args) {
        Store store;
        //Elektronik ürün siparişi geldi
        store = new ElectronicsStore( );
        store.createOrder("5524345678");
        //Kitap siparişi geldi
        store = new BookStore( );
        store.createOrder("8693243565");
    }
}
```

- Esinlenme: Head First DP

38

ÖRNEK KALIP GERÇEKLEMELERİ – FACTORY METHOD

ÇÖZÜM:



39

ÖRNEK KALIP GERÇEKLEMELERİ – FACTORY METHOD

ÇÖZÜM:

```
package dp.factoryMethod.example;
public abstract class Order {
    public abstract void prepare( );
    public abstract void orderFromSupplier( );
    public abstract void pack( );
    public abstract void sendToCustomer( );
}

public class Electronics extends Order {
    private String barcode;

    public Electronics( String barcode ) {
        super( ); this.barcode = barcode;
    }
    public void orderFromSupplier() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
    public void pack() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
    public void prepare() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
    public void sendToCustomer() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
}

public class Books extends Order { /*Coded similarly*/ }
```

40

ÖRNEK KALIP GERÇEKLEMELERİ – FACTORY METHOD

ÇÖZÜM:

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}

public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}

public class BookStore extends Store {
    protected Order createOrder(String uniqueID ) {
        Books newBookOrder = new Books(uniqueID);
        //Kitap siparişi için yapılacak işlemlerin tanımlanması
        return newBookOrder;
    }
}
```

41

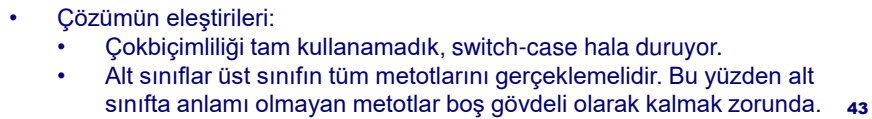
YARATIMSAL (CREATIONAL) KALIPLAR

BUILDER:

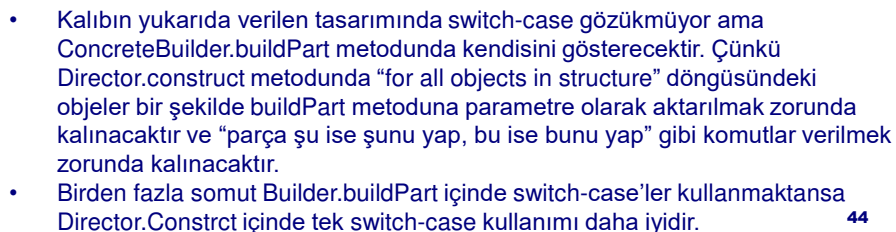
- Amaç:
 - Karmaşık bir nesnenin oluşturulması ile gösterimini birbirinden ayırmak. Böylece aynı oluşturma işlemi farklı gösterimler ortaya çıkarabilir.
- Örnek:
 - Bir RTF metin okuyucu programı, bu belgeyi çeşitli diğer metin türlerine, hatta metni düzenleyebilecek bir GUI bileşenine çevirebilsin.
- Sorun:
 - Bir çok farklı metin türleri var, hepsi hemen gerçekleştirilmeyecek.
 - Bu yüzden ileride yeni bir metin türü eklemek zor olmamalı.
- Çözüm:
 - Metni okumakla sorumlu sınıf, bir biçim çevirici nesnesine çeviri isteğini göndersin.
 - Çevirici nesne hem çevrim işleminden, hem de çevrilen metni belli bir biçimde simgelemekle yükümlü olsun.
 - Bu durumda yeni metin türleri için çevirici sınıfının alt sınıfları oluşturulabilir.

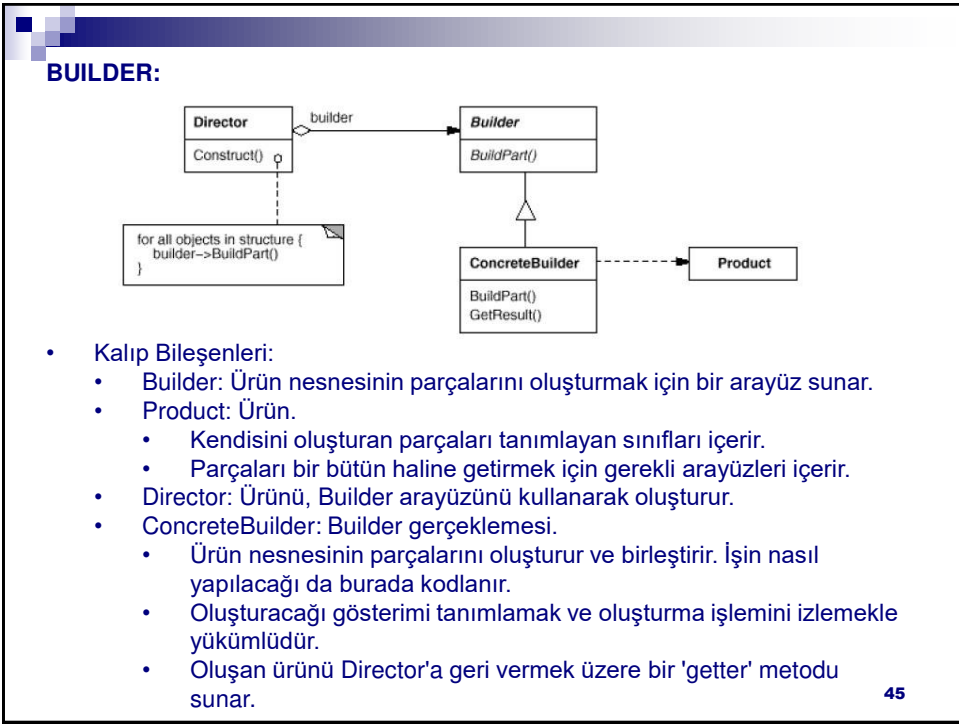
42

- Çözüm:



- Kalıp Yapısı:





YARATIMSAL (CREATIONAL) KALIPLAR

BUILDER:

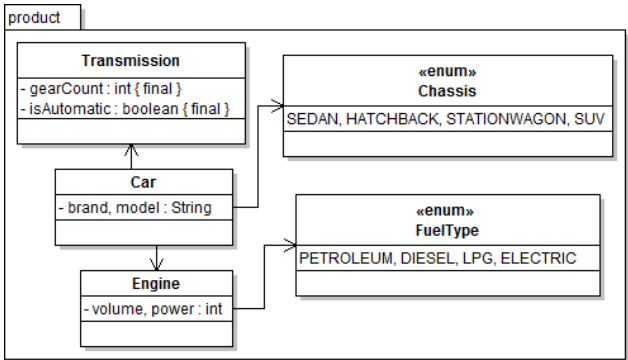
- Kalıbın uygulanabileceği anlar:
 - Karmaşık bir nesneyi oluşturma algoritmasının, nesneyi oluşturan parçalardan bağımsız olması gerektiği ve bu parçaların nasıl birleştirileceğinden bağımsız olması gerektiği anlarda.
 - Bir nesneyi oluşturma sürecinin bu nesnenin farklı gösterimlerinin olmasına izin verebilmesi gerektiği anlarda.
- Sonuçlar:
 - Ürünün iç yapısının en az yan etki ile değiştirilebilmesi mümkün olur (yeni bir ConcreteBuilder sınıfı yazarak).
 - Önceden gördüğümüz 'factory' kalıpları ile karşılaştırıldığında, ürün oluşturma süreci üzerinde daha fazla denetim sahibi olunmakta (Director örneği Builder örneğine hangi işlemi hangi sırada yapması gerektiği emrini verir).

47

ÖRNEK KALIP GERÇEKLEMELERİ – BUILDER

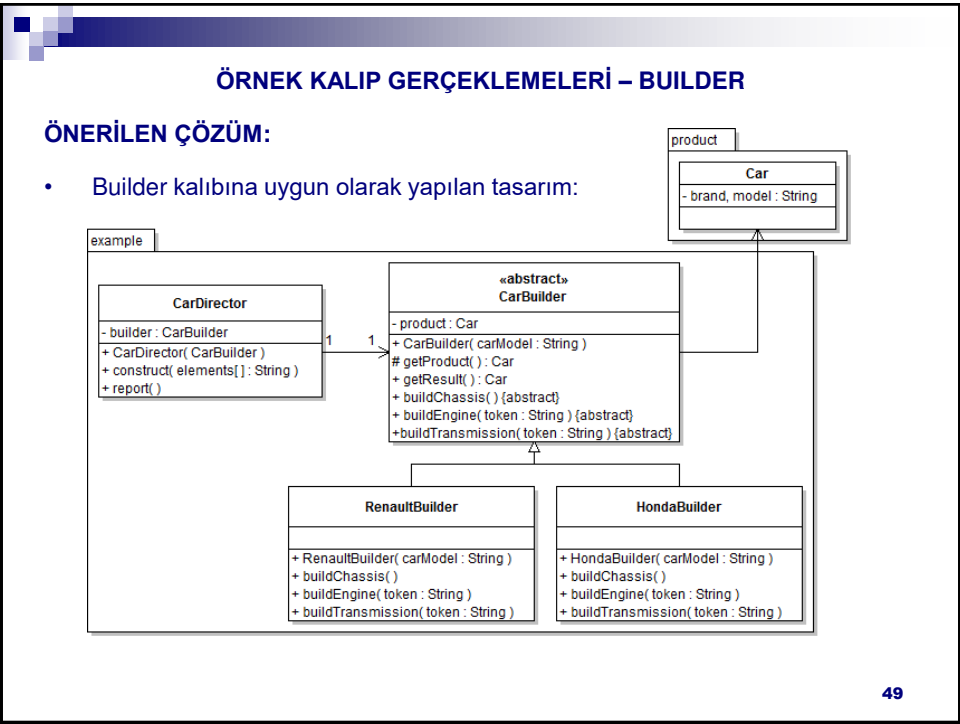
ÇÖZÜLECEK PROBLEM:

- Farklı marka ve model araçların satıldığı bir bayide markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacaktır.
- Araçlar kasa, yakıt türü, motor hacmi ve gücü, vb. bileşenlerin bir araya getirilmesi ile karmaşık bir süreçle oluşmaktadır.



- Kaynak: YES

48



ÖRNEK KALIP GERÇEKLEMELERİ – BUILDER

ÖNERİLEN ÇÖZÜM:

- Kaynak kodlar:

```
package dp.builder.example;
import dp.builder.product.*;
public abstract class CarBuilder {
    private Car product;
    public CarBuilder( String carModel ) {
        product = new Car(carModel);
    }
    public Car getResult() {
        if( product == null )
            return null;
        Car clone = new Car(product.getModel());
        clone.setBrand(product.getBrand());
        clone.setChassis(product.getChassis());
        clone.setEngine(product.getEngine());
        clone.setGear(product.getGear());
        return clone;
    }
    protected Car getProduct() { return product; }
    public abstract void buildChassis( );//Şasi'yi modele bağladım.
    public abstract void buildEngine(String token);
    public abstract void buildTransmission(String token);
}
```

50

ÖRNEK KALIP GERÇEKLEMELERİ – BUILDER

ÖNERİLEN ÇÖZÜM:

```
public class CarDirector {
    private CarBuilder builder;
    public CarDirector(CarBuilder builder) { this.builder = builder; }
    public void construct(String elements[]) {
        builder.buildChassis();
        for( String element : elements ) {
            int tab = element.indexOf("\t");
            String item = element.substring(0,tab);
            String info = element.substring(tab+1,element.length());
            if( item.compareToIgnoreCase("Engine")== 0 )
                builder.buildEngine(info);
            else if( item.compareToIgnoreCase("Gear")== 0 )
                builder.buildTransmission(info);
        }
    }
    public void report( ) { System.out.println(builder.getResult()); }
    public static void main(String[] args) {
        CarDirector director = new CarDirector(
            new HondaBuilder("Civic Sedan") );
        String info[] = {"Engine\t125PS","Engine\t1.6L","Gear\t6AT"};
        director.construct(info);
        director.report();
    }
}
```

ÖRNEK KALIP GERÇEKLEMELERİ – BUILDER

ÖNERİLEN ÇÖZÜM:

- Somut bir CarBuilder alt sınıfı örneği:

```
package dp.builder.example;
import dp.builder.product.*;
public class HondaBuilder extends CarBuilder {
    public HondaBuilder( String carModel ) {
        super(carModel);
        getProduct().setBrand("Honda");
    }
    public void buildChassis( ) {
        if( getProduct().getModel().toUpperCase().contains("BACK") )
            getProduct().setChassis(Chassis.HATCHBACK);
        else if( getProduct().getModel().toUpperCase().contains("CR-V") )
            getProduct().setChassis(Chassis.SUV);
        else
            getProduct().setChassis(Chassis.SEDAN);
    }
}
```

ÖRNEK KALIP GERÇEKLEMELERİ – BUILDER

ÖNERİLEN ÇÖZÜM:

- Somut bir CarBuilder alt sınıfı örneği (devam):

```
public void buildEngine(String token) {
    //Min. values for a Honda engine
    if( getProduct().getEngine() == null )
        getProduct().setEngine(new Engine(FuelType.PETROLEUM,1150,60));
    token = token.toUpperCase();
    if( token.contains("ECO") )
        getProduct().getEngine().setFuel(FuelType.LPG);
    else if( token.contains("PS") ) {
        int end = token.indexOf("PS");
        token = token.substring(0, end);
        getProduct().getEngine().setPower( Integer.parseInt(token) );
    }
    else if( token.contains("L") ) {
        int end = token.indexOf("L");
        token = token.substring(0, end);
        getProduct().getEngine().setVolume(
            (int)(Double.parseDouble(token) * 1000) );
    }
}
```

53

ÖRNEK KALIP GERÇEKLEMELERİ – BUILDER

ÖNERİLEN ÇÖZÜM:

- Somut bir CarBuilder alt sınıfı örneği (devam):

```
public void buildTransmission(String token) {
    //By default, AutoTransmission has 4 gears and ManualT has 5.
    int gearCount = Integer.parseInt(token.substring(0,1));
    boolean isAutomatic = false;
    if( token.toUpperCase().contains("AT") )
        isAutomatic = true;
    if( isAutomatic && gearCount == 0 )
        gearCount = 4;
    if( !isAutomatic && gearCount == 0 )
        gearCount = 5;
    getProduct().setGear( new Transmission(gearCount, isAutomatic) );
}
```

54

YARATIMSAL (CREATIONAL) KALIPLAR

PROTOTYPE:

- Amaç:
 - Oluşturulacak nesnelerin türlerini belirlemek için bir prototip nesne kullanmak ve yeni nesneleri bu prototipi kopyalayarak oluşturmak.
- Örnek:
 - Grafik uygulamalarına yönelik mevcut bir çerçeve programını kullanarak, bir müzik besteleme programı yazılacak.
 - Çerçeve programının soyut bir Graphic sınıfı, çeşitli kontrol düğmelerinin yer alabileceği paletleri simgeleyen soyut bir Tool sınıfı, bundan kalıtımla türetilmiş ve yeni grafik şekillerini çizim alanına eklemeye yarayan GraphicTool sınıfı var.
 - Notaları eklemek için GraphicTool sınıfı kullanılabilir.
- Sorun:
 - Çerçeve programının çeşitli nota sınıfları hakkında önceden bilgisi olamaz.
 - GraphicTool sınıfı bilmediği sınıflardan çizim nesnelerini nasıl oluşturur?

55

YARATIMSAL (CREATIONAL) KALIPLAR

PROTOTYPE:

- Örnek Çözüm:

```
classDiagram
    class Tool {
        Manipulate()
    }
    class RotateTool {
        Manipulate()
    }
    class GraphicTool {
        Manipulate()
    }
    class Graphic {
        Draw(Position)
        Clone()
    }
    class Staff {
        Draw(Position)
        Clone()
    }
    class MusicalNote {
    }
    class WholeNote {
        Draw(Position)
        Clone()
    }
    class HalfNote {
        Draw(Position)
        Clone()
    }

    Tool <|-- RotateTool
    Tool <|-- GraphicTool
    Graphic <|-- Staff
    Graphic <|-- MusicalNote
    MusicalNote <|-- WholeNote
    MusicalNote <|-- HalfNote

    GraphicTool o--> Graphic : prototype
    GraphicTool ..> GraphicTool : p = prototype->Clone()
    GraphicTool ..> GraphicTool : while (user drags mouse) {
    GraphicTool ..> GraphicTool : p->Draw(new position)
    GraphicTool ..> GraphicTool : }
    GraphicTool ..> GraphicTool : insert p into drawing

    WholeNote ..> WholeNote : return copy of self
    HalfNote ..> HalfNote : return copy of self
```

p = prototype->Clone()
while (user drags mouse) {
p->Draw(new position)
}
insert p into drawing

return copy of self

return copy of self

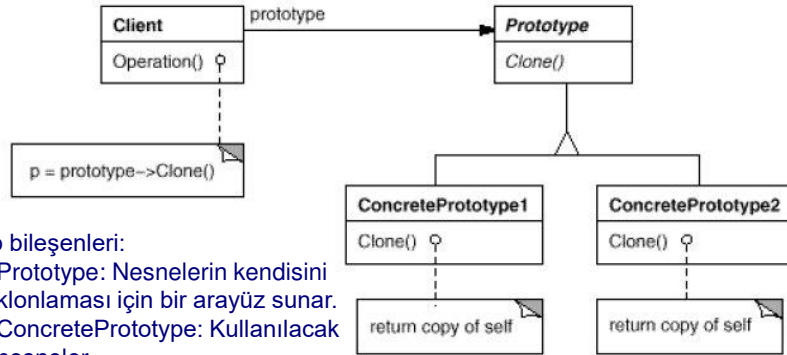
- Diğer seçenekler:
 - Factory method, abstract factory:
 - Her nota için ayrı fabrika sınıfı gerekecek.
 - Başka?

56

YARATIMSAL (CREATIONAL) KALİPLAR

PROTOTYPE:

- Kalıp yapısı:



- Kalıp bileşenleri:
 - Prototype: Nesnelerin kendisini klonlaması için bir arayüz sunar.
 - ConcretePrototype: Kullanılacak nesneler
 - Client: Bir prototipin kendisini kopyalamasını isteyerek yeni bir nesne oluşturur.

57

YARATIMSAL (CREATIONAL) KALİPLAR

PROTOTYPE:

- Kalıbın uygulanabileceği anlar:
 - Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar
 - ve şu durumlardan biri söz konusu ise (Factory Method kalıbından farklılaşmayı sağlıyor):
 - Oluşturulacak sınıflar çalışma anında belirlenecekse
 - Ürünlerin sınıf hiyerarşisi ile paralel bir factory hiyerarşisi kurmak istenmiyorsa
 - Ürün sınıfının örneklerinin durum uzayı sınırlı ise
- Bu kalıp C++ gibi sınıfların doğrudan işlenebilecek varlıklar olmadığı dillerde daha çok yarar sağlar.
 - SmallTalk gibi geç ilişkilendirme (late binding) ve Java gibi yansıtma (reflection) yetenekleri olan dillerde bu kalıp yerine dilin yetenekleri kullanılabilir.

58

YARATIMSAL (CREATIONAL) KALIPLAR

PROTOTYPE:

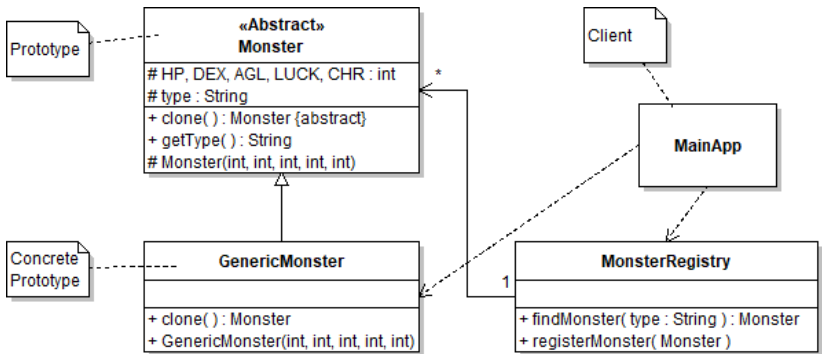
- Tartışmalar:
 - Sığ kopyalama veya derin kopyalama (shallow or deep copy)
 - Sığ kopyalama yetersiz kalabilir
 - Çevrimsellik derin kopyalamada sorun çıkartır.
 - Klonların üyelerine değer atanması için Prototype sınıfında ek metotlar tanımlamak gerekebilir.

59

ÖRNEK KALIP GERÇEKLEMELERİ – PROTOTYPE

ÇÖZÜLECEK PROBLEM:

- Bir bilgisayar oyunu için level editörü hazırlanıyor.
- Haritaya çok çeşitli türlerden canavarları kolayca ekleyebilmek istiyoruz.
- Ama bir canavar oluşturmak kolay iş değil, bir sürü durum bilgisi var:
 - HP, DEX, AGL, LUCK, CHR, vb.
- Çözüm:



- Kaynak: Head First DP

60

ÖRNEK KALIP GERÇEKLEMELERİ – PROTOTYPE

ÖNERİLEN ÇÖZÜM:

```
package dp.prototype.example;
public abstract class Monster {
    protected int HP, DEX, AGL, LUCK, CHR;
    protected String type;

    public abstract Monster clone( );

    protected Monster(int hp, int dex, int agl, int luck, int chr) {
        HP = hp; DEX = dex; AGL = agl; LUCK = luck; CHR = chr;
    }
    public String getType() { return type; }
    public int getHP() { return HP; }
    protected void setHP(int hp) { HP = hp; }
    public int getDEX() { return DEX; }
    protected void setDEX(int dex) { DEX = dex; }
    public int getAGL() { return AGL; }
    protected void setAGL(int agl) { AGL = agl; }
    public int getLUCK() { return LUCK; }
    protected void setLUCK(int luck) { LUCK = luck; }
    public int getCHR() { return CHR; }
    protected void setCHR(int chr) { CHR = chr; }
    protected void setType(String type) { this.type = type; }
}
```

protected setter'lar ile erişimin tek yolu Register olur, tercih ederim.

ÖRNEK KALIP GERÇEKLEMELERİ – PROTOTYPE

ÖNERİLEN ÇÖZÜM:

```
package dp.prototype.example;
public class GenericMonster extends Monster {
    public GenericMonster(int hp, int dex, int agl, int luck, int chr) {
        super(hp, dex, agl, luck, chr);
        type = "Generic Monster";
    }
    public Monster clone( ) {
        return new GenericMonster(HP, DEX, AGL, LUCK, CHR);
    }
}

public class MonsterRegistry {
    private HashMap<String, Monster> monsters;
    public MonsterRegistry() { monsters = new HashMap<String,Monster>(); }

    public Monster findMonster( String type ) {
        return monsters.get( type );
    }
    public void registerMonster( Monster m ) {
        monsters.put( m.getType(), m );
    }
}
```

ÖRNEK KALIP GERÇEKLEMELERİ – PROTOTYPE

ÖNERİLEN ÇÖZÜM:

```
public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
            reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bölüm sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

63

YARATIMSAL (CREATIONAL) KALIPLAR

SINGLETON:

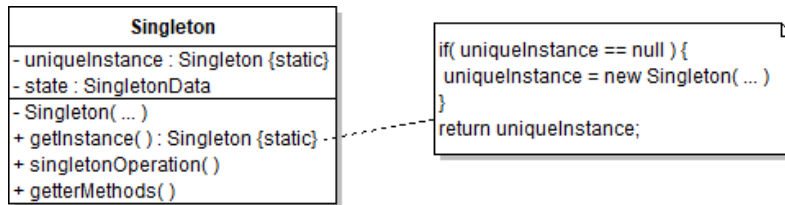
- Amaç:
 - Bir sınıfın sadece bir örneğinin bulunmasını sağlamak ve bu nesneye ortak bir erişim noktası vermek.
- Örnek:
 - Bazı nesnelerin türünün tek örneği olmasının gerekli olduğu alanlar bulunabilir.
 - Bir bilgisayarda birden fazla yazıcı tanımlı olabilmesine rağmen bir tek yazdırma biriktiricisi bulunur.
 - Bir işletim sisteminde tek bir dosya sistemi ve pencere yöneticisi bulunur.
- Çözüm 1:
 - Tek olacak nesne, static olarak tanımlanır.
 - Bu nesne farklı sınıflar tarafından kullanılacaksa, bunlardan hangisinin üyesi olacak?

64

YARATIMSAL (CREATIONAL) KALİPLAR

SINGLETON:

- Önerilen Çözüm:
 - Bir sınıfa, kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak.
 - Sınıfın kendi türünden static bir üyesi olur,
 - Sınıfın kurucusu private tanımlanarak kurucuya erişim engellenir,
 - Sınıfın bu üye için bir factory metodu bulunur.
 - Kalıba adını veren bu sınıfın adı Singleton olarak belirlenmiştir.



65

YARATIMSAL (CREATIONAL) KALIPLAR

SINGLETON:

- Gerçekleme ayrıntıları:
 - Strategy ve State gerçeklemeleri, aynı zamanda iyi birer Singleton adayıdır (davranışsal kalıplar ileride incelenecek).
 - Bu kalıpta Singleton sınıflarını kalıtım yolu ile özelleştirmek, ilk önerilen çözüme göre daha kolaydır.
 - Ancak Singleton nesnesi farklı türden nesneler tarafından kullanılmayacaksa, ilk öneriyi kullanmak yeterli olacaktır.
- Kalıbın kullanılabilmesi için:
 - Bazı nesnelerin türünün tek örneği olmasının gerekli olduğu ve bu nesnelerin bir çok farklı sınıf örneklerince kullanılması istendiği durumlarda.

ÖRNEK KALIP GERÇEKLEMELERİ – SINGLETON

- Basit bir kalıp olduğundan bu aşamada örnek yapılmayacaktır.

66

NESNEYE DAYALI TASARIM VE MODELLEME
KISIM 1: TASARIM KALIPLARI
1.2. YAPISAL (STRUCTURAL) KALIPLAR

67

YAPISAL (STRUCTURAL) KALIPLAR

ADAPTER

- Amaç:
 - İşlev açısından uyumlu ancak metot adları açısından uyumsuz bir istekçi sınıf ile bir sunucu sınıfı, birbirleri ile tek yönlü konuşurmak.
- Örnek:
 - İngiliz standardındaki elektrik prizine Avrupa standardındaki cihazı takabilmek için bir adaptör kullanmak gerekir.
 - Adapter kalıbı da aynı şekilde çalışır.



68

YAPISAL (STRUCTURAL) KALIPLAR

ADAPTER:

- Kalıp yapısı:

```
classDiagram
    class Client
    class Target {
        Request()
    }
    class Adapter {
        Request()
    }
    class Adaptee {
        SpecificRequest()
    }
    Client --> Target
    Adapter --|> Target
    Adapter --> Adaptee : adaptee
    Adapter ..> Adaptee : adaptee-> SpecificRequest()
```

- Kalıp katılımcıları:
- Adaptee: Uyarlanması istenen sınıf (Sunucu)
- Client: Uyarlanacak sınıfa iş yaptırmak isteyen istekçi sınıf
- Target: İstekçinin konuşmak istediği arayüz
- Adapter: Uyarlama işlemini yapan, programcının yazacağı sınıf.
 - Diğer sınıflar halihazırda mevcuttur.

69

YAPISAL (STRUCTURAL) KALIPLAR

ADAPTER:

- Kalıbın uygulanabileceği anlar:
- Elimizdeki bir sınıfın bir istekçi tarafından kullanılabilmesini önleyen tek engelin, istekçinin yolladığı mesajlar ile eldeki sınıfın anladığı mesajların adlarının farklı olduğu anlarda.

70

ÖRNEK KALIP GERÇEKLEMELERİ – ADAPTER

ÇÖZÜLECEK PROBLEM:

- Bir çizim programı üzerinde çalışılıyor.
- Programın bir kısmı halihazırda gerçekleştirilmiştir (aşağıda solda)

Shape

+setLocation()

+getLocation()

+display()

+fill()

+setColor()

+undisplay()

Point

+display()

+fill()

+undisplay()

Line

+display()

+fill()

+undisplay()

Square

+display()

+fill()

+undisplay()

XXCircle

+setLocation()

+getLocation()

+displayIt()

+fillIt()

+setItsColor()

+undisplayIt()

← ??? →

- Daire kodunu yazmadan önce farkettilik ki, elimizde hazır bir daire sınıfı var.
- Ancak bu daire sınıfı mevcut programımıza uymuyor (yukarıda sağda).

ÖRNEK KALIP GERÇEKLEMELERİ – ADAPTER

ÇÖZÜM:

```
class Circle extends Shape {  
    ...  
    private XXCircle myXXCircle;  
    ...  
    public Circle () {  
        myXXCircle= new XXCircle();  
    }  
  
    void public display() {  
        myXXCircle.displayIt();  
    }  
    ...  
}
```

Shape

+setLocation()

+getLocation()

+display()

+fill()

+setColor()

+undisplay()

Point

+display()

+fill()

+undisplay()

Line

+display()

+fill()

+undisplay()

Square

+display()

+fill()

+undisplay()

Circle

+setLocation()

+getLocation()

+display()

+fill()

+setColor()

+undisplay()

XXCircle

+displayIt()

+displayIt()

+fillIt()

+undisplayIt()

+setLocation()

+getLocation()

+setItsColor()

◊

36

YAPISAL (STRUCTURAL) KALIPLAR

BRIDGE:

- Amaç:
 - Nesnenin arayüzü ile gerçeklemesini birbirinden ayırmak.
 - 'Handle' olarak da bilinir (Win32 programcıları!)
- Örnek:
 - Bir GUI çerçeve programının pencere soyutlaması.
 - Gnome, KDE gibi farklı masaüstü ortamlarının desteklenmesi isteniyor.
 - Pencere soyut sınıfından kalıtımla yeni pencere sınıfları türetilir.
- Sorun:
 - Pencere soyutlamasının görev çubuğunda simge haline getirilmiş pencereler için de kullanılması isteniyor.
 - Pencere sınıfından kalıtımla yeni bir IconWindow sınıfı üretmek yetmez, desteklenecek tüm masaüstü ortamları için IconWindow sınıfının alt sınıflarını da oluşturmak gerekir:

73

YAPISAL (STRUCTURAL) KALIPLAR

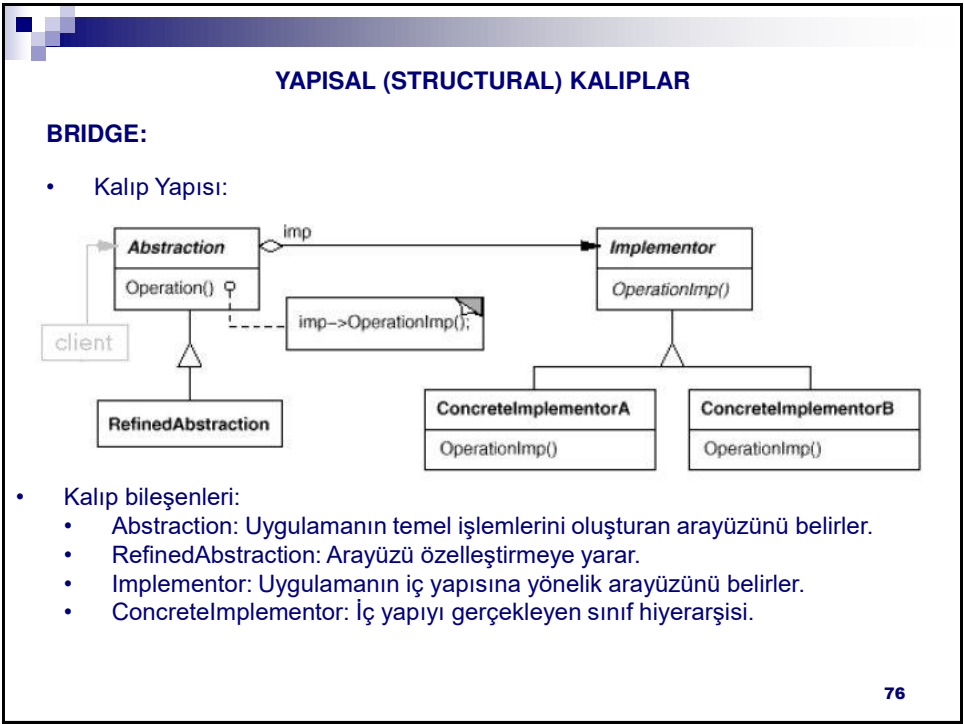
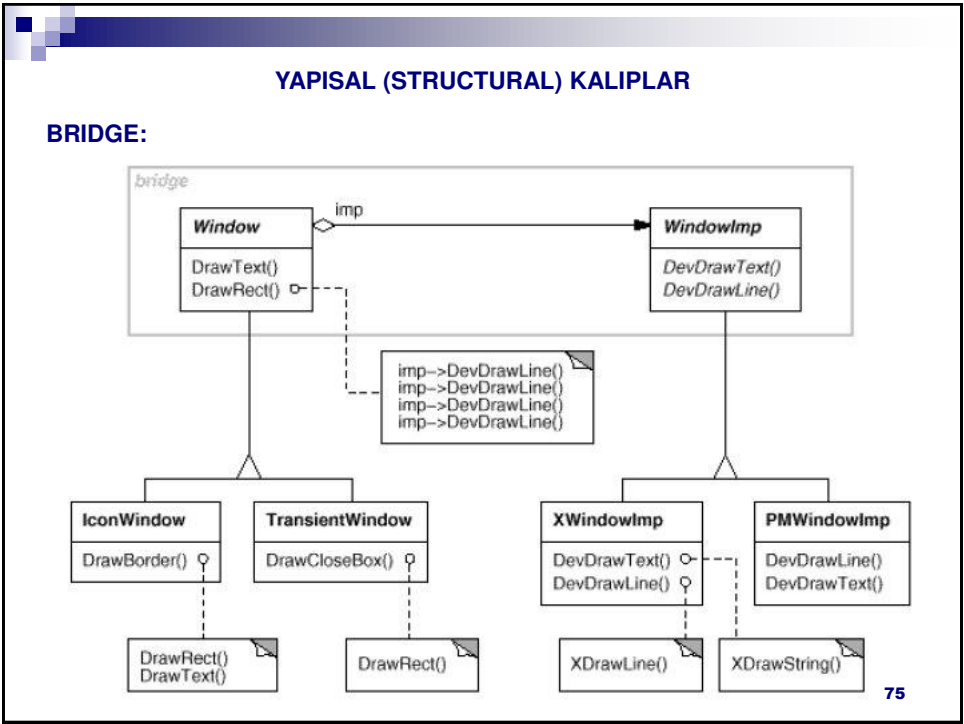
BRIDGE:

- Sorun:

```
graph TD
    subgraph "Before"
        Window1[Window]
        XWindow1[XWindow]
        PMWindow1[PMWindow]
        Window1 --- XWindow1
        Window1 --- PMWindow1
    end
    subgraph "After"
        Window2[Window]
        XWindow2[XWindow]
        PMWindow2[PMWindow]
        IconWindow[IconWindow]
        XIconWindow[XIconWindow]
        PMIconWindow[PMIconWindow]
        Window2 --- XWindow2
        Window2 --- PMWindow2
        Window2 --- IconWindow
        IconWindow --- XIconWindow
        IconWindow --- PMIconWindow
    end
```

- Çözüm:
 - Pencere soyutlaması ve gerçeklemeleri ayrı sınıf hiyerarşilerine koyulur
 - Farklı gerçeklemeler soyutlama nesnesinde saklanır.
 - İşlemler soyutlamalar üzerinden çağırılır.
 - Soyutlama ile gerçekleştirme arasında **köprü** kurulmuş olur.

74



YAPISAL (STRUCTURAL) KALIPLAR

BRIDGE:

- Kalıbın uygulanabileceği anlar:
 - Bir soyutlama ile gerçeklemeleri arasındaki bağı kalıcı olmasının istenmediği anlarda.
 - Hem soyutlamaların hem de gerçeklemelerin ayrı ayrı özelleştirilmesine gerek duyulduğu anlarda.

77

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÇÖZÜLECEK PROBLEM:

- Bir çizim programı üzerinde çalışılıyor.
- İlk aşamada şekiller sadece dikdörtgenlerden oluşuyor, ancak iki farklı dikdörtgen ailesi var.
- Farklı iki aileden olan dikdörtgenler farklı iki yoldan çizilecek.
- Bir dikdörtgen nasıl çizileceğini biliyor.

(x1, y2)

(x2, y2)

(x1, y1)

(x2, y1)

DP1	DP2
Used to draw a line	
<code>draw_a_line(x1, y1, x2, y2)</code>	<code>drawline(x1, x2, y1, y2)</code>

78

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÖNERİLEN ÇÖZÜM (1):

- İki tip farklı dikdörtgen varsa bunlar bir üst sınıftan kalıtımla türetilir.
- Bu ikisinin çizim metotları istenen iki farklı şekilde gerçekleştirir.

```
classDiagram
    class Client
    class Rectangle {
        +draw()
        #drawLine()
    }
    class V1Rectangle {
        #drawLine()
    }
    class V2Rectangle {
        #drawLine()
    }
    class DP1 {
        +draw_a_line()
    }
    class DP2 {
        +drawline()
    }
    Client --> Rectangle
    Rectangle <|-- V1Rectangle
    Rectangle <|-- V2Rectangle
    V1Rectangle ..> DP1 : #drawLine()
    V2Rectangle ..> DP2 : #drawLine()
```

The diagram illustrates the Bridge Pattern. A **Client** depends on an abstract **Rectangle** interface. **Rectangle** defines a public `draw()` method and a protected `drawLine()` method. Two concrete classes, **V1Rectangle** and **V2Rectangle**, inherit from **Rectangle** and implement the `drawLine()` method. **V1Rectangle** delegates the call to **DP1** (which has a `draw_a_line()` method), while **V2Rectangle** delegates to **DP2** (which has a `drawline()` method).

- protected metodun amacı:
 - `drawLine` metodu nesnenin iç çalışması ile ilgilidir, o yüzden public yapılmamalıdır.
 - private metotlar kalıtım ile aktarılır ama alt sınıfta doğrudan kullanılamaz.
 - protected metotlar hem farklı türden nesnelere görünmez, hem de alt sınıflara kalıtımla aktarılırlar.

79

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÖNERİLEN ÇÖZÜM (1):

```
package bridge.solution1;
public abstract class Rectangle {
    private double _x1, _x2, _y1, _y2;
    public void draw() {
        drawLine(_x1, _y1, _x2, _y1);
        drawLine(_x2, _y1, _x2, _y2);
        drawLine(_x2, _y2, _x1, _y2);
        drawLine(_x1, _y2, _x1, _y1);
    }
    abstract protected void drawLine ( double x1, double y1,
        double x2, double y2);
}
public class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1,
        double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
}
```

80

40

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÖNERİLEN ÇÖZÜM (1):

```
package bridge.solution1;
public abstract class Rectangle {
    private double _x1, _x2, _y1, _y2;
    public void draw( ) {
        drawLine(_x1,_y1,_x2,_y1);
        drawLine(_x2,_y1,_x2,_y2);
        drawLine(_x2,_y2,_x1,_y2);
        drawLine(_x1,_y2,_x1,_y1);
    }
    abstract protected void drawLine ( double x1, double y1,
        double x2, double y2);
}
public class V2Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1,
        double x2, double y2) {
        DP2.drawline(x1,x2,y1,y2);
    }
}
```

81

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÇÖZÜMÜN ZAYIF YÖNLERİ:

- Yeni bir gereksinim ortaya çıkıyor:
 - Çizim programındaki şekillere iki farklı çember ailesi de ekleniyor.
 - Farklı çember ailelerinin farklı çizim metotları var.
 - 1. dikdörtgen ailesi ile 1. çember ailesi ortaktır.
 - 2. dikdörtgen ailesi ile 2. çember ailesi ortaktır.
 - Bu durumda çember çizim metotları az önceki çözümün statik çizim metotlarında biriktirilebilir (veya bunlar DP1 ve DP2 kütüphane sınıflarında zaten vardı).
 - Çizim programın bir şeklin dikdörtgen veya çember olmasını dert etmemesi isteniyor.
 - Bunu karşılamak için dikdörtgen ve çember şekilleri, ortak bir Şekil üst sınıfı altında birleştirilebilir.
 - Tüm bunlar önceki tasarımı bozmadan yapılırsa ortaya çıkan sınıf şeması şu şekilde olacaktır:

82

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÇÖZÜMÜN ZAYIF YÖNLERİ:

- Yeni bir gereksinimi karşılayan sınıf şeması:

```
classDiagram
    class Client
    class Shape {
        +draw()
    }
    class Rectangle {
        +draw()
        #drawLine()
    }
    class Circle {
        +draw()
        #drawCircle()
    }
    class V1Rectangle {
        #drawLine()
    }
    class V2Rectangle {
        #drawLine()
    }
    class V1Circle {
        #drawCircle()
    }
    class V2Circle {
        #drawCircle()
    }
    class DP1 {
        +draw_a_line()
        +draw_a_circle()
    }
    class DP2 {
        +drawline()
        +drawcircle()
    }
    Client --> Shape
    Shape <|-- Rectangle
    Shape <|-- Circle
    Rectangle <|-- V1Rectangle
    Rectangle <|-- V2Rectangle
    Circle <|-- V1Circle
    Circle <|-- V2Circle
    V1Rectangle ..> DP1
    V1Rectangle ..> DP2
    V2Rectangle ..> DP1
    V2Rectangle ..> DP2
    V1Circle ..> DP1
    V1Circle ..> DP2
    V2Circle ..> DP1
    V2Circle ..> DP2
```

83

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÇÖZÜMÜN ZAYIF YÖNLERİ:

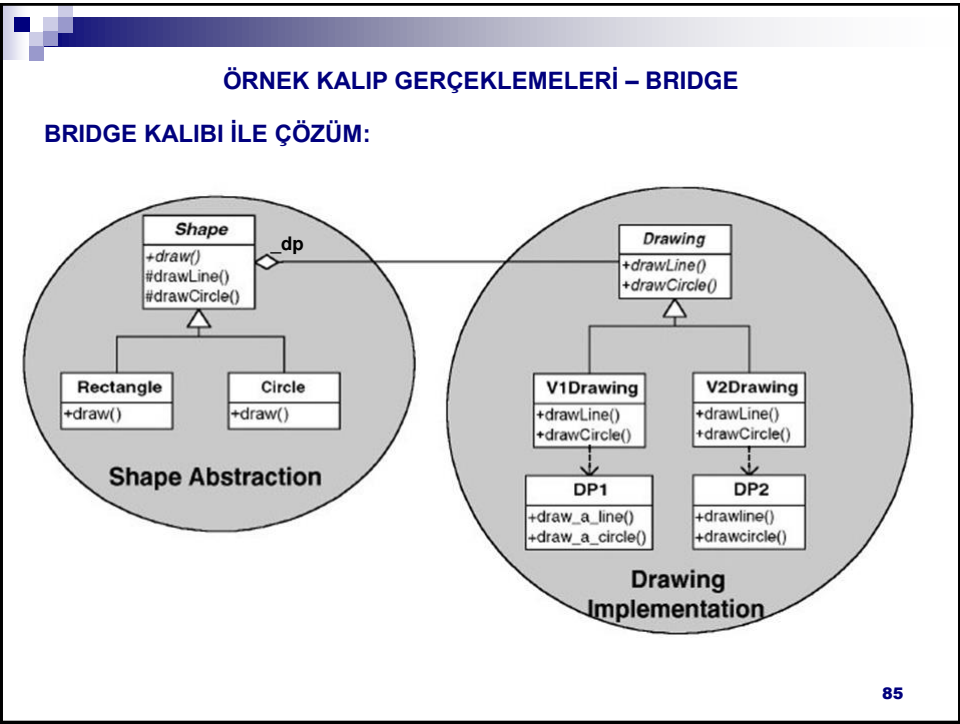
- Değerlendirme:
 - Mevcut durumda 4 belirli şekil var (2 aile ve 2 şekil, $2 \times 2 = 4$).
 - Yeni bir statik çizim metodu eklemek gerekirse sınıf sayısı 6 olacak.
 - Bunun üstüne yeni bir şekil (ör. üçgen) eklense sınıf sayısı 9 olacak ($3 \times 3 = 9$).
 - Her yeni eklenecek çizim metodu veya şekil türü ile sınıf sayısı fırlayacak!

YENİ ÇÖZÜM ÖNERİSİ:

- Sorunun kaynağı, farklı şekil türleri ile farklı çizim yollarının sıkı birlikteliğidir.
- Şekil türlerini simgeleyen soyutlamalar ile çizim yollarını simgeleyen gerçeklemeleri birbirinden ayırmak, sorunumuzu çözebilir.
- Bridge kalıbının amacını hatırlayın:
 - Nesnenin arayüzü ile gerçeklemesini birbirinden ayırmak.
 - Bir başka deyişle, soyutlamalar ile bunların gerçeklemelerini birbirinden ayırmak.

84

42



ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

BRIDGE KALIBI İLE ÇÖZÜM:

```
package bridge.solution2;
public abstract class Shape {
    public abstract void draw( );
    private Drawing _dp;
    public Shape (Drawing dp) { _dp= dp; }
    protected void drawLine(double x1, double y1,
        double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x,double y,
        double r ) { _dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle (Drawing dp,
        double x,double y,double r) {
        super( dp) ;
        _x= x; _y= y; _r= r ;
    }
    public void draw () { drawCircle(_x,_y,_r); }
}
```

86

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

BRIDGE KALIBI İLE ÇÖZÜM:

```
package bridge.solution2;
public class Rectangle extends Shape {
    private double _x1, _y1, _x2, _y2;
    public Rectangle(Drawing dp, double x1, double y1,
        double x2, double y2) {
        super(dp);
        _x1= x1; _x2= x2; _y1= y1; _y2= y2;
    }
    public void draw() {
        drawLine(_x1, _y1, _x2, _y1);
        drawLine(_x2, _y1, _x2, _y2);
        drawLine(_x2, _y2, _x1, _y2);
        drawLine(_x1, _y2, _x1, _y1);
    }
}
```

87

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

BRIDGE KALIBI İLE ÇÖZÜM:

```
package bridge.solution2;
public abstract class Drawing {
    public abstract void drawLine( double x1, double y1,
        double x2, double y2 );
    public abstract void drawCircle (double x, double y,
        double r);
}
public class V1Drawing extends Drawing {
    public void drawCircle(double x, double y, double r) {
        DP1.draw_a_circle(x,y,r);
    }
    public void drawLine(double x1, double y1,
        double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
}
```

88

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

BRIDGE KALIBI İLE ÇÖZÜM:

```
package bridge.solution2;
public class V2Drawing extends Drawing {
    public void drawCircle(double x, double y, double r) {
        DP2.drawcircle(x,y,r);
    }
    public void drawLine(double x1, double y1,
        double x2, double y2) {
        DP2.drawline(x1,x2,y1,y2);
    }
}
```

89

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

BRIDGE KALIBI İLE ÇÖZÜM:

```
package bridge.solution2;
public class DP1 {
    public static void draw_a_line( double x1, double y1,
        double x2, double y2 ) {
        //çizimi yap
    }
    public static void draw_a_circle( double x,
        double y, double r ) {
        //çizimi yap
    }
}
```

90

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

BRIDGE KALIBI İLE ÇÖZÜM:

```
package bridge.solution2;
public class DP2 {
    public static void drawline( double x1, double x2,
        double y1, double y2 ) {
        //çizimi yap
    }
    public static void drawcircle ( double x, double y,
        double r ) {
        //çizimi yap
    }
}
```

91

ÖRNEK KALIP GERÇEKLEMELERİ – BRIDGE

ÇIKARTILAN DERSLER = TASARIMA AİT GENEL KURALLAR

- Neyin değiştiğini bul ve onu sarmala.
 - Değişik türden şekiller ve değişik türden çizimler ortaya çıktı.
- Kalıtım ilişkisi yerine parça/bütün ilişkisini tercih et
 - Şekiller ve çizim türlerini aynı sınıflarda toplamak bağlaşımlı arttırdı ve yeni bir şekil/çizim türü eklemek gerekli sınıf sayısını üstel olarak artırdı.

92

YAPISAL (STRUCTURAL) KALIPLAR

COMPOSITE:

- Amaç:
 - Hiyerarşi şeklindeki parça-bütün ilişkilerini desteklemeye yöneliktir.
 - Bu kalıp sayesinde bireysel nesneler ve bir nesneler bütünü, istekçilere aynı şekilde gözükür.
- Örnek: Bir grafik çizim programı hazırlanıyor.
 - Çizgi, dörtgen, metin gibi grafik bileşenleri kendilerini çizebilir.
 - Bu bileşenler bir resim adı altında gruplandırılabilir.
 - Bir resim ise alt resim parçalarından oluşabilir.

```
graph TD; aPicture1(aPicture) --> aPicture2(aPicture); aPicture1 --> aLine1(aLine); aPicture1 --> aRectangle1(aRectangle); aPicture2 --> aText(aText); aPicture2 --> aLine2(aLine); aPicture2 --> aRectangle2(aRectangle);
```

93

YAPISAL (STRUCTURAL) KALIPLAR

COMPOSITE:

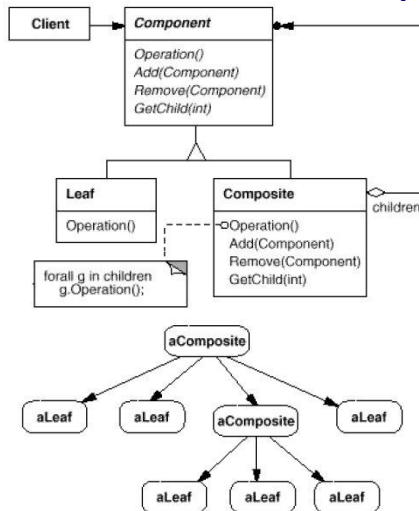
- Örnek çözüm:

```
classDiagram
    class Graphic {
        Draw()
        Add(Graphic)
        Remove(Graphic)
        GetChild(int)
    }
    class Line {
        Draw()
    }
    class Rectangle {
        Draw()
    }
    class Text {
        Draw()
    }
    class Picture {
        Draw()
        Add(Graphic g)
        Remove(Graphic)
        GetChild(int)
        graphics : List<Graphic>
    }
    Graphic <|-- Line
    Graphic <|-- Rectangle
    Graphic <|-- Text
    Graphic <|-- Picture
    Picture o--> Graphic : graphics
```

94

COMPOSITE:

- Kalıp yapısı:



- Kalıp katılımcıları:
 - Component:
 - Nesneler topluluğunun katılımcıları için ortak kullanım arayüzü sunar.
 - Bir bütünün parçaları için erişim ve düzenleme metotları tanımlar.
 - Leaf:
 - Topluluktaki parçaları simgeler.
 - Hiyerarşik yapının en altındaki düğümlerdir, çocukları olamaz.
 - Composite:
 - Hiyerarşik yapının ara düğümleridir.
 - Farklı tür ara düğümler varsa bu sınıfın alt sınıfları şeklinde modellenir.
 - Client: Parça ve toplulukları benzer şekilde kullanılabilir.
 - Client bir Visitor olabilir.
- 95

95

YAPISAL (STRUCTURAL) KALIPLAR

COMPOSITE:

- Kalıbın kullanılabileceği anlar:
 - Nesneler arasındaki hiyerarşik parça-bütün ilişkilerinin modellenmesi istenildiğinde.
 - Bir nesneler topluluğu ve topluluktaki bireysel nesnelere eş biçimli erişim gerekli olduğunda.
- Gerçekleme ayrıntıları:
 - Alt düğüm, üst düğümünden haberdar kılınabilir.
 - Bir düğüm nasıl silinir? Kim siler? Nasıl bir veri yapısı kullanılır?
 - children üyesi belki üst sınıfa taşınabilir (zaten aşağıdaki ilk zayıflık mevcut olduğundan yapılabilir bir eylemdir).
 - ...
- Kalıbın zayıf yönleri:
 - Kalıtım ilişkisinin özüne aykırı bir davranışta bulunur: Leaf bir alt sınıf olmasına rağmen, üst sınıfı olan Component'teki çocuk yönetim metotlarının bu alt sınıf için bir anlamı yoktur.
 - Halbuki alt sınıf üst sınıftaki tüm metotları kalımla almak zorundadır.
 - Programcının karar vermesi gereken gerçekleştirme ayrıntısı sayısı, diğer kalıplara göre çok fazladır.

96

ÖRNEK KALIP GERÇEKLEMELERİ – COMPOSITE

ÇÖZÜLECEK PROBLEM:

- Makineler ve parçalarından oluşan üretim kataloğumuz var.
- Bu katalogda makinelerin ve/veya parçaların hangi fabrikada üretildiği, maliyeti, vb. gibi bilgilerden çeşitli raporlar üretilmesi isteniyor.
- Bazı makineler kendi içinde makine parçaları içerebilmektedir.
- Bu durumda Composite kalıbını uygulamak yerinde olacaktır.

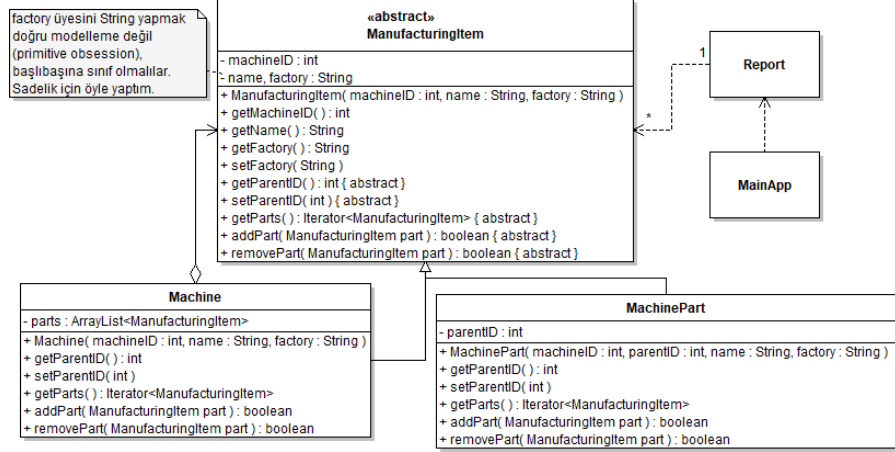
• Esinlenme: DP Java Workbook

97

ÖRNEK KALIP GERÇEKLEMELERİ – COMPOSITE

COMPOSITE KALIBI İLE ÇÖZÜM

- Tasarım:



- Makinenin alt makineleri olabilir deseydik parentID de üst sınıfa taşıyırđı. 98

ÖRNEK KALIP GERÇEKLEMELERİ – COMPOSITE

COMPOSITE KALIBI İLE ÇÖZÜM

- Kaynak kodlar:

```
public abstract class ManufacturingItem {
    private int machineID;
    private String name, factory;

    public ManufacturingItem(int machineID, String name, String factory) {
        this.machineID = machineID; this.name = name;
        this.factory = factory;
    }
    public int getMachineID() { return machineID; }
    public String getName() { return name; }
    public String getFactory() { return factory; }
    public void setFactory(String factory) { this.factory = factory; }
    public abstract int getParentID();
    public abstract void setParentID(int parentID);
    public abstract Iterator<ManufacturingItem> getParts();
    public abstract boolean addPart( ManufacturingItem part );
    public abstract boolean removePart( ManufacturingItem part );
}
```

99

ÖRNEK KALIP GERÇEKLEMELERİ – COMPOSITE

COMPOSITE KALIBI İLE ÇÖZÜM

```
public class Machine extends ManufacturingItem {
    private ArrayList<ManufacturingItem> parts;
    public Machine( int machineID, String name, String factory ) {
        super(machineID, name, factory);
        parts = new ArrayList<ManufacturingItem>();
    }
    public int getParentID() { return 0; }
    public void setParentID(int parentID) { }
    public boolean addPart( ManufacturingItem part ) {
        part.setParentID(getMachineID());
        for( ManufacturingItem aPart : parts )
            if( aPart == part )
                return false;
        return parts.add(part);
    }
    public boolean removePart( ManufacturingItem part ) {
        return parts.remove(part);
    }
    public Iterator<ManufacturingItem> getParts() {
        return parts.iterator();
    }
}
```

100

ÖRNEK KALIP GERÇEKLEMELERİ – COMPOSITE

COMPOSITE KALIBI İLE ÇÖZÜM

```
public class MachinePart extends ManufacturingItem {
    private int parentID;
    public MachinePart( int machineID, int parentID,
        String name, String factory) {
        super(machineID, name, factory);
        this.parentID = parentID;
    }
    public int getParentID() { return parentID; }
    public void setParentID(int parentID) { this.parentID = parentID; }
    public Iterator<ManufacturingItem> getParts() { return null; }
    public boolean addPart(ManufacturingItem part) { return false; }
    public boolean removePart(ManufacturingItem part) { return false; }
}
```

```
public class Report {
    public static int findItems(ManufacturingItem items[],String factory) {
        int nr = 0;
        System.out.println("Items manufactured in " + factory + ":");
        System.out.println("Nr. \tID\tName");
        System.out.println("-----");
        for( ManufacturingItem item : items ) {
            if( item==null )continue;
            if( item.getFactory().compareToIgnoreCase(factory) == 0 ) {
                nr++; System.out.println( nr + ".\t" + item.getMachineID()
                    + "\t" + item.getName() );
            }
            Iterator<ManufacturingItem> subItems = item.getParts();
            if( subItems==null ) continue;
            while( subItems.hasNext() ) {
                ManufacturingItem part = subItems.next();
                if( part.getFactory().compareToIgnoreCase(factory) == 0 ) {
                    nr++; System.out.println( nr + ".\t" +
                        part.getMachineID() + "\t" + part.getName() );
                }
            }
        }
        if( nr == 0 ) System.out.println("No such item found.");
        return nr;
    }
}
```

//Duplicate Code sorunu olmayan versiyonu için bkz. dp.composite.solution1ReportV2.java

ÖRNEK KALIP GERÇEKLEMELERİ – COMPOSITE

COMPOSITE KALIBI İLE ÇÖZÜM

```
public class MainApp {
    public static void main(String[] args) {
        ManufacturingItem[] items = new ManufacturingItem[4];
        items[0] = new MachinePart(99, 127, "Krank şaftı KŞ-7", "Bursa");
        Machine jenerator = new Machine(135, "Jeneratör JMX-99", "İstanbul");
        jenerator.addPart(new MachinePart(259,135,"Flanş FTR-5", "Bursa"));
        jenerator.addPart(new MachinePart(378,135,"Kasnak KS-9", "İstanbul"));
        jenerator.addPart(new MachinePart(196,135,"Rulman RL-3", "Ankara"));
        items[1] = jenerator;
        items[2] = new MachinePart(63, 76, "Distribütör DST-12", "Bursa");
        items[3] = new MachinePart(82, 19, "Pnömatik Vana VPN-7", "Ankara");
        int sayi = Report.findItems(items, "Bursa");
        System.out.println("\t" + sayi + " parça bulundu.");
    }
}
```

103

YAPISAL (STRUCTURAL) KALIPLAR

DECORATOR:

- Amaç:
 - Nesnelere çalışma anında (dinamik olarak) ek sorumluluklar atamak.
 - Bu iş kalıtımla ancak kodlama anında yapılabilir.
 - Böylece bireysel nesneler özelleştirilebilir.
 - Kalıtımda özelleşme sınıf düzeyinde olduğundan, aynı tipteki bütün nesneler birden özelleşecektir.
- Örnek:
 - Bir GUI çerçeve uygulaması yazılıyor.
 - Metin alanlarını bazen çerçeve ile çevrelemek, bazen kaydırma çubuğu ile donatmak isteniyor.

104

YAPISAL (STRUCTURAL) KALIPLAR

DECORATOR:

- Çözüm 1:

```
classDiagram
    class TextArea
    class ScrollTextArea
    class BorderTextArea
    class ScrollBorderTextArea
    TextArea <|-- ScrollTextArea
    TextArea <|-- BorderTextArea
    TextArea <|-- ScrollBorderTextArea
    ScrollBorderTextArea <|-- ScrollTextArea
    ScrollBorderTextArea <|-- BorderTextArea
```

- Çözümün zayıf yönleri:
 - Scroll ve Border yetenekleri iki ayrı yerde tekrar gerçekleşmek zorunda.
 - TextArea'ya ek olarak bir de Panel sınıfına Scroll ve Border yeteneği kazandırmak için ek üç sınıf daha gerekecek.

105

YAPISAL (STRUCTURAL) KALIPLAR

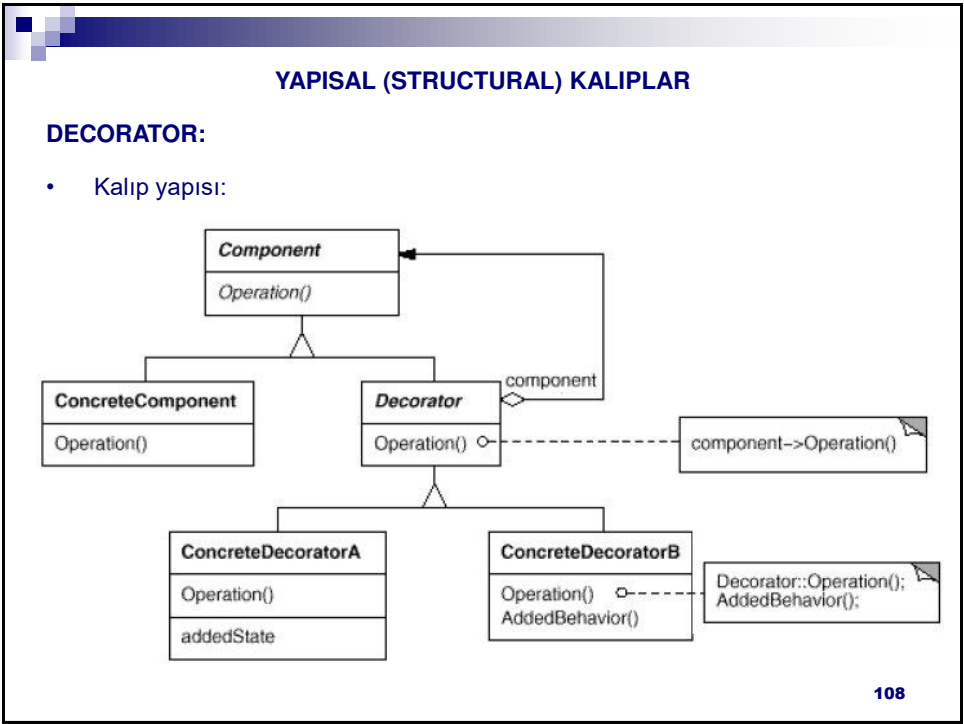
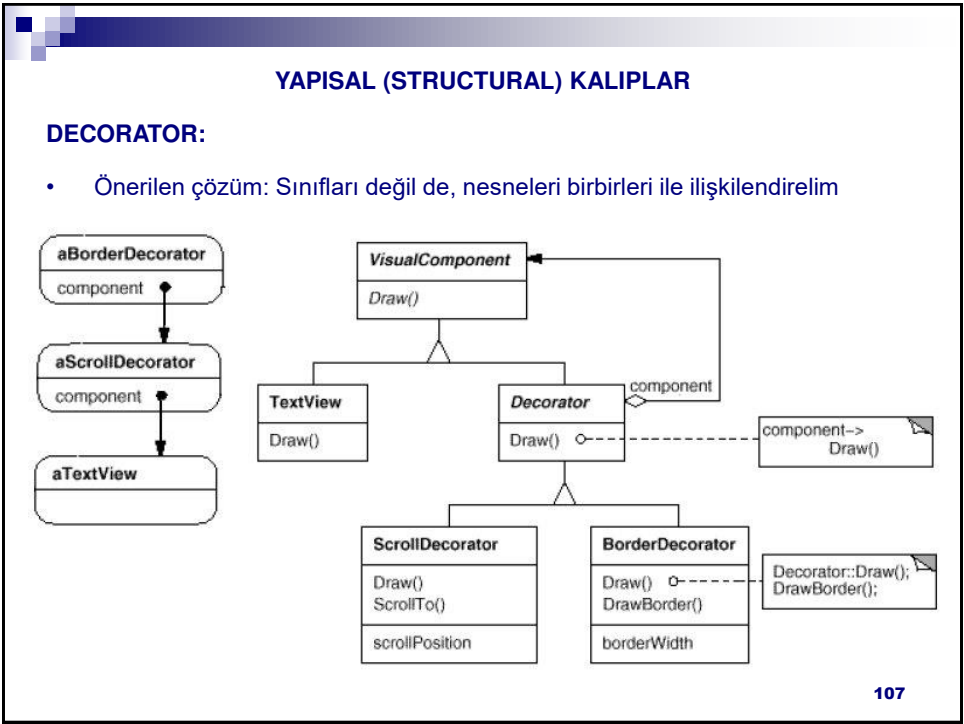
DECORATOR:

- Çözüm 2:

```
classDiagram
    class TextArea
    class ScrollTextArea
    class BorderTextArea
    class ScrollBorderTextArea
    TextArea <|-- ScrollTextArea
    TextArea <|-- BorderTextArea
    ScrollBorderTextArea <|-- ScrollTextArea
    ScrollBorderTextArea <|-- BorderTextArea
```

- Çözümün zayıf yönleri:
 - Çoklu kalıtım her dilde desteklenmez.
 - Çoklu kalıtımın bu çözümdeki şekli 'diamond problem' sorununun bir örneğidir.
 - TextArea'ya ek olarak bir de Panel sınıfına Scroll ve Border yeteneği kazandırmak için ek üç sınıf daha gerekecek.
 - Üstelik bu durumda bu örneğin Scroll ve Border yeteneklerinin tek yerde gerçekleşmesi avantajı ortadan kalkacak.

106



YAPISAL (STRUCTURAL) KALIPLAR

DECORATOR:

- Kalıp katılımcıları:
 - Component: Bileşen nesnesi
 - Kendisine ek sorumluluklar kazandırılacak nesnelerin arayüzünü tanımlar.
 - Bu sorumluluk, sadece bu arayüzde tanımlanan eylemlere kazandırılabilir.
 - Ek sorumlulukların tanımlanacağı donatıcılar da bu arayüzü gerçekleştirmelidir.
 - ConcreteComponent: Bileşen gerçekleştirilmesi.
 - Decorator: Donatıcı
 - Bir bileşen nesnesine işaretçi tutar.
 - ConcreteDecorator: Donatıcı gerçekleştirilmesi.

109

YAPISAL (STRUCTURAL) KALIPLAR

DECORATOR:

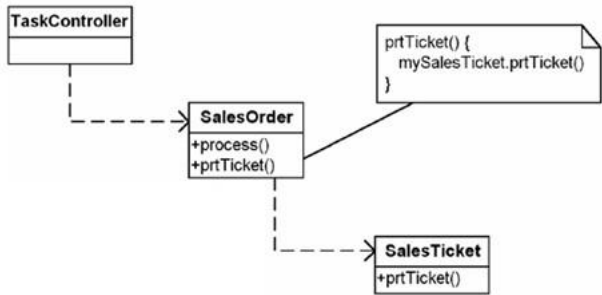
- Kalıbın kullanılabileceği anlar:
 - Tek tek nesnelere dinamik olarak (çalışma anında) yeni sorumluluklar eklenmek istediğinde
 - Tek tek nesnelere şeffaf olarak (diğerlerini etkilemeden) yeni sorumluluklar eklenmek istediğinde
 - Bazı sorumlulukların iptal edilebileceği veya askıya alınabileceği durumlarda
 - Kalıtımın uygun olmadığı anlarda.
 - Ör: Sınıf kodu mevcut değil, yeni eklemeler üstel artan sınıf sayısına neden olacak, vb.
 - Java'nın Stream tabanlı I/O kütüphanesi bu kalıbı kullanır.
- Kalıbın zayıf yönü:
 - Yeni sorumluluk, ancak şu şekillerde eklenebilir:
 - Eski sorumluluğun tamamen iptal edilmesi.
 - Dikkat: Nesne zinciri kopabilir!
 - Eski sorumluluktan önce ve/veya sonra yeni komutlar eklenmesi.
 - Yeni yeteneklere doğrudan dekore edilen nesne üzerinden değil, en üst dekore eden nesne üzerinden erişilmesi gerekmektedir.

110

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÇÖZÜLECEK PROBLEM:

- Satış emirlerinin makbuzlarını basacak şekilde TaskController adlı programımıza yeni bir işlevsellik eklemek istiyoruz.
- İlk çözüm:



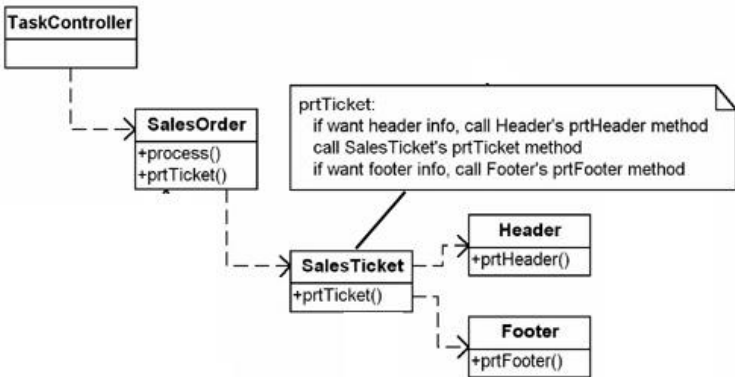
- Yeni gereksinim: Makbuzlara üstbilgi ve altbilgi (header/footer) eklemek.

111

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÇÖZÜM 1:

- Kullanım ilişkisi ile:



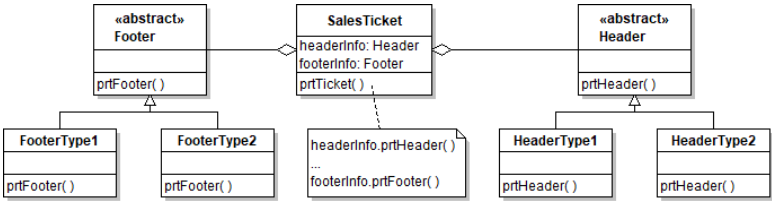
- Makbuz basılacağı zaman SalesTicket sınıfında ayrı if sınımaları ile üstbilgi ve/veya altbilgi basılıp basılmayacağına karar verilir.
- Farklı üstbilgi ve altbilgi türleri söz konusu ise ne olacak?

112

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÇÖZÜM 2:

- Farklı üstbilgi ve altbilgi türleri için ayrı kalıtım hiyerarşileri kullanılabilir:
 - (Henüz incelenmedi ama bir Strategy kalıbı gerçekteşidir)



- Peki ya aynı makbuzda birden fazla farklı üstbilgi ve/veya altbilgi basılması isteniyorsa?
- Örneğin:

HEADER1
HEADER2
Receipt
FOOTER1
FOOTER2

HEADER1
Receipt
FOOTER1
FOOTER2

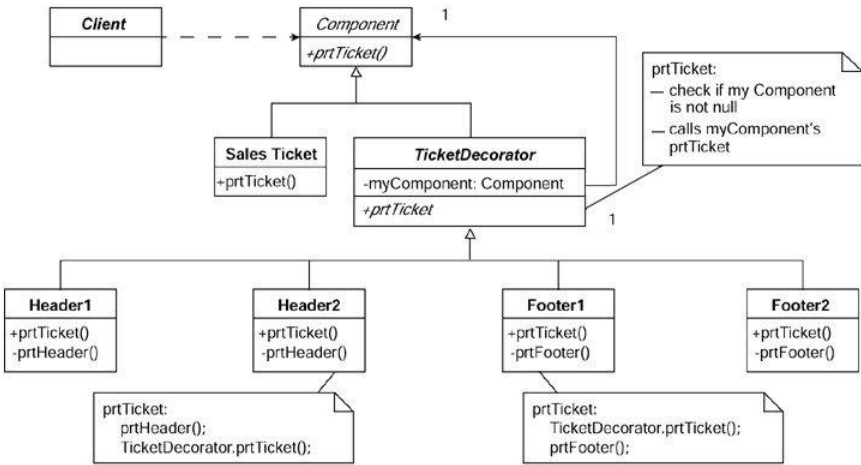
...

- Bu durumda Decorator kalıbı bu gereksinimi daha esnek biçimde karşılar.

113

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÖNERİLEN ÇÖZÜM:



114

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÖNERİLEN ÇÖZÜM:

```
package decorator.example;
public abstract class Component {
    abstract public void prtTicket();
}
public class SalesTicket extends Component {
    public void prtTicket() {
        // place sales ticket printing code here
        System.out.println(getClass().getName());
    }
}
public abstract class TicketDecorator extends Component {
    private Component myComponent;
    public TicketDecorator (Component myComponent) {
        this.myComponent= myComponent;
    }
    public void prtTicket() {
        if (myComponent != null) myComponent.prtTicket();
    }
}
```

115

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÖNERİLEN ÇÖZÜM:

```
package decorator.example;
public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket() {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.prtTicket();
    }
}
public class Header2 extends TicketDecorator {
    public Header2 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 2 code here
        System.out.println(getClass().getName());
        super.prtTicket();
    }
}
```

116

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÖNERİLEN ÇÖZÜM:

```
package decorator.example;
public class Footer1 extends TicketDecorator {
    public Footer1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket() {
        super.prtTicket();
        // place printing footer 1 code here
        System.out.println(getClass().getName());
    }
}
public class Footer2 extends TicketDecorator {
    public Footer2 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket() {
        super.prtTicket();
        // place printing footer 2 code here
        System.out.println(getClass().getName());
    }
}
}
```

117

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÖNERİLEN ÇÖZÜM:

- Peki Client sınıfı nasıl kodlanacak?
 - Ne tür üst/alt bilgi bileşimi seçileceğine (ilgili dekorasyona) Client karar verebilir.
 - Tasarımı bu aşamada FactoryMethod kalıbı ile geliştirebiliriz.

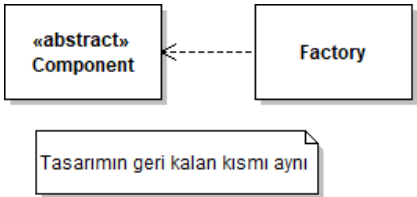
```
package decorator.example;
public class Client {
    public static void main(String[] args) {
        Factory myFactory = new Factory();
        Component myComponent = myFactory.getComponent();
        myComponent.prtTicket();
    }
}
public class Factory {
    public Component getComponent () {
        return(new Header1( new Header2( new Footer2(
            new Footer1( new SalesTicket()))));
    }
}
}
```

118

ÖRNEK KALIP GERÇEKLEMELERİ – DECORATOR

ÖNERİLEN ÇÖZÜM:

- Factory sınıfında yapılan zincirleme (chaining) işleminin açılımı ve etkileri için kaynak kodları inceleyiniz.
- Son tasarım:



- Çalışma sonucu:

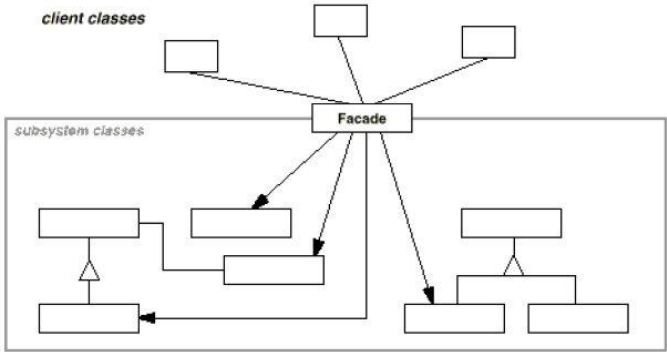
HEADER1
HEADER2
Receipt
FOOTER1
FOOTER2

119

YAPISAL (STRUCTURAL) KALIPLAR

FACADE:

- Amaç:
 - Çeşitli alt sistemlerde bulunan farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamak.
 - Böylece istekçiler alt sistemler ile ilgili alt düzey ayrıntılardan soyutlanabilir.
- Kalıp yapısı:



120

YAPISAL (STRUCTURAL) KALIPLAR

FACADE:

- Kalıbın uygulanabileceği anlar:
 - Karmaşık bir alt sisteme basit bir arayüz sağlanmak istenildiğinde,
 - Alt sistemlerin çeşitli katmanlara ayrılması istenildiğinde,
 - Bazı istekçilerin sistemin belli bir kısmıyla bağlaşımının azaltılması istenildiğinde.

ÖRNEK KALIP GERÇEKLEMELERİ – FACADE

- Basit bir kalıp olduğundan örnek yapılmayacaktır.

121

YAPISAL (STRUCTURAL) KALIPLAR

FLYWEIGHT:

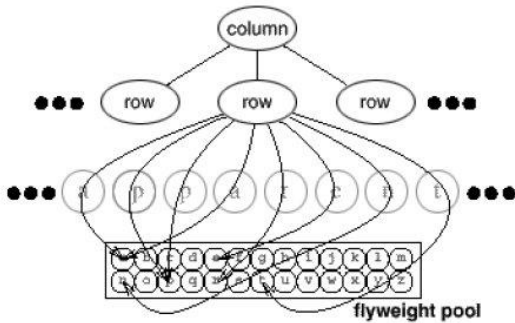
- Amaç:
 - Paylaşım yolu ile, çok sayıda basit nesnenin kullanımının desteklenmesi.
- Örnek:
 - Bir metin düzenleme programı yazılıyor.
 - Programın font ve diğer biçimlendirme bilgilerine göre, metin bilgisini çok net göstermesi isteniyor (excellent rendering).
 - Harfleri grafik nesneleri olarak gösterip, en iyi sonucu almak istiyoruz.
 - Metin sütunlar ve satırlar içerisinde bulunan harfler olarak ele alınabilir.
- Sorun:
 - Ancak her harfi ayrı bir nesne örneği olarak ele alırsak, küçük bir belgede bile binlerce nesne olacaktır.
 - Bu nesnelerin bellekte ve ikincil saklama ortamlarında kaplayacağı yer çok aşırı olacaktır.
 - Bu dersin üç sayfalık öneri formunda bile, boşluklarla birlikte 5000 kadar harf bulunmakta.

122

YAPISAL (STRUCTURAL) KALİPLAR

FLYWEIGHT:

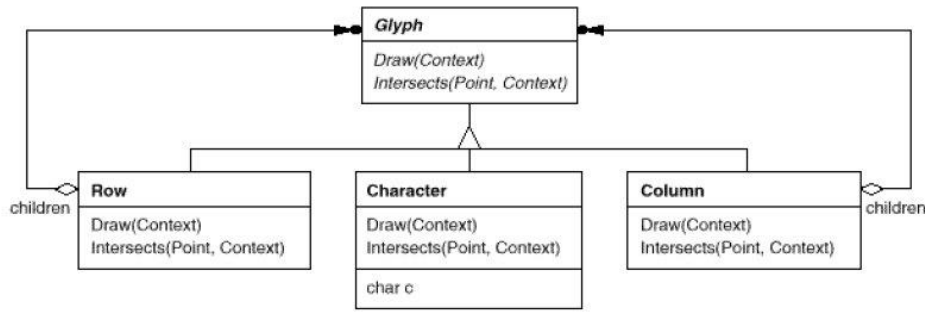
- **Çözüm:**
 - Unicode alfabesindeki her harfi bir harf nesnesi olarak ele alalım,
 - Metnin sütun ve satırlarında yeni bir harf nesnesi oluşturmak yerine,
 - Her bir harfi simgeleyen tek harf nesnesine, o harfin yer aldığı konulardan referans verelim.
 - Diğer bir deyişle, harf nesnelerinin ortak bir havuzdan paylaşımlı kullanımını sağlayalım.
 - Bu tip paylaşılan nesnelere flyweight adı verilir.
-

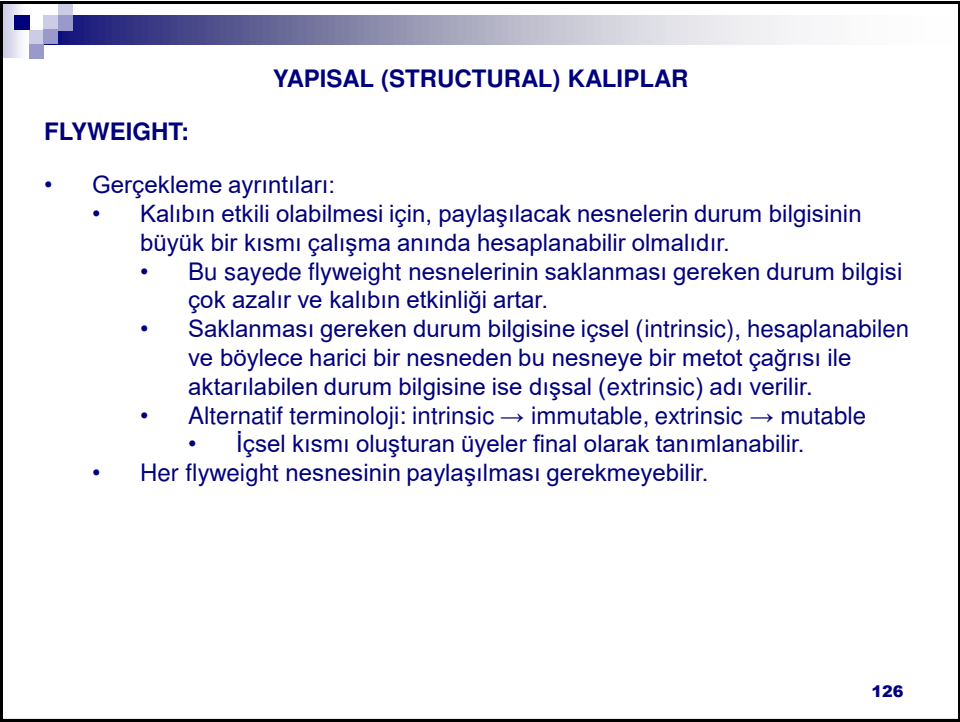
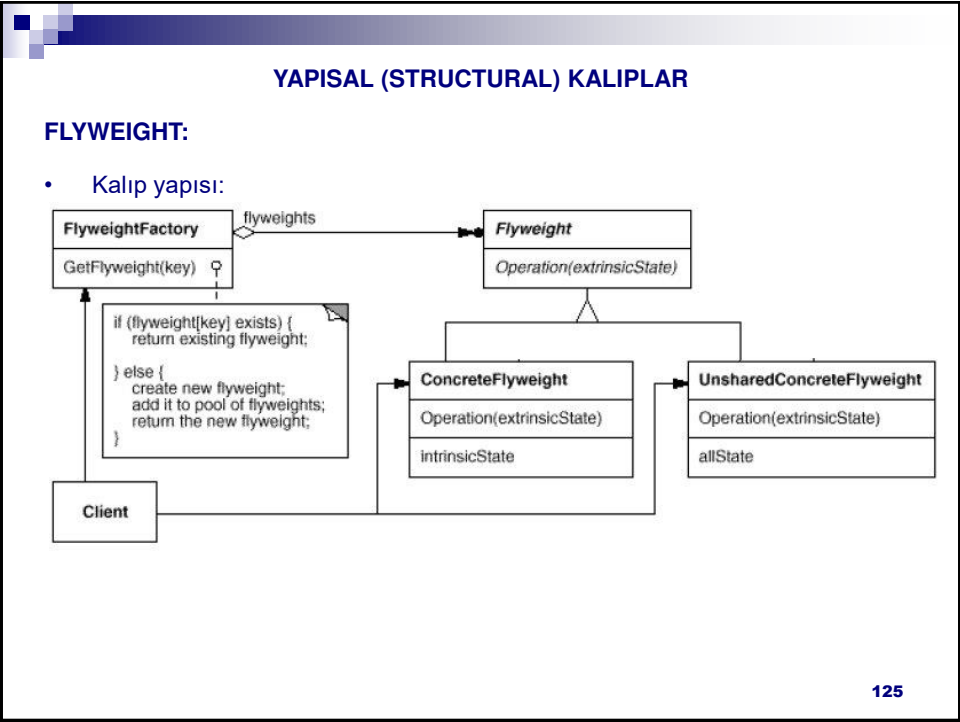


YAPISAL (STRUCTURAL) KALİPLAR

FLYWEIGHT:

- Örnek çözüm:



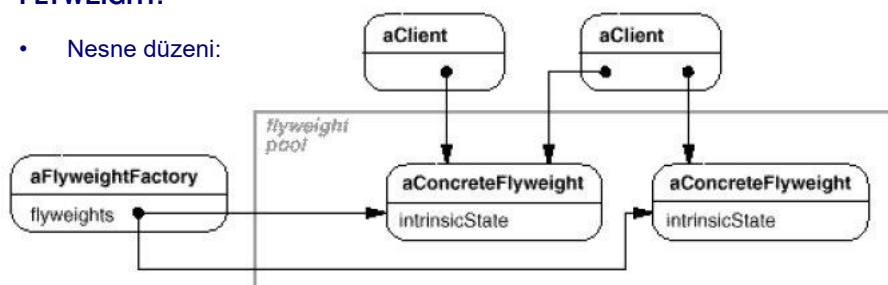


FLYWEIGHT:

- 127

FLYWEIGHT:

- Nesne düzeni:



- 128

YAPISAL (STRUCTURAL) KALIPLAR

FLYWEIGHT:

- Kalıbın kullanılabileceği anlar:
 - Şu koşulların tümünün doğru olması gerekmektedir:
 - Bir uygulamanın çok fazla sayıda nesne kullandığı,
 - Nesnelerin çok fazla olması nedeniyle saklama masraflarının çok yüksek olması,
 - Nesnelerin durum bilgisinin içsel ve dışsal olarak iki bölüme ayrılabilmesi (içsel bilgi ne kadar çoksa, ortak kullanım sayesinde elde edilen tasarruf o kadar yüksek olacaktır),
 - Dışsal durumları ayrılan nesnelerin bir çok grubunun yerini, paylaşılabilir az sayıda nesnenin alabileceği,
 - Uygulamanın nesne kimliğine bağımlı olmadığı anlarda.

129

ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

ÇÖZÜLECEK PROBLEM:

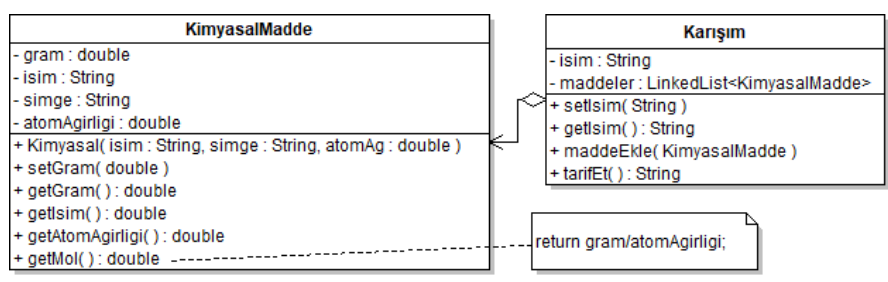
- Bir kimya fabrikasının üretim bilgi sistemi hazırlanıyor.
- Fabrikada bir çok kimyasal madde üretilmektedir.
- Fabrikada birkaç kimyasal maddenin birleşiminden oluşan çok sayıda karışımlar da hazırlanmaktadır.
- Aynı madde doğal olarak birden fazla karışımda yer alabilmektedir.

- Kaynak: DP Java Workbook p128

130

ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

İLK ÇÖZÜM:

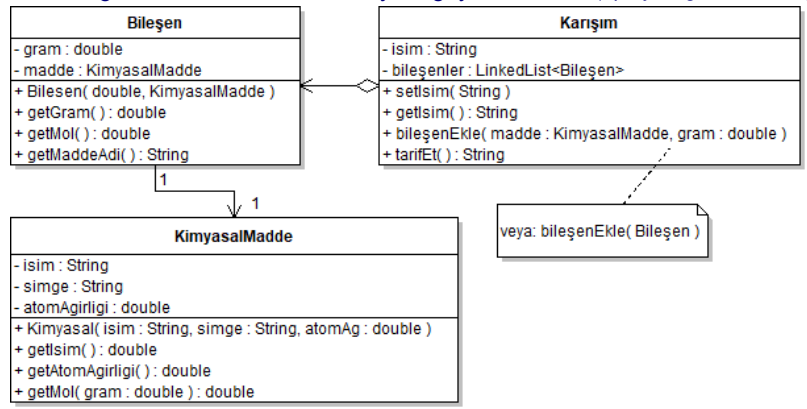


- Kaynak kod basit olduğu için notlarda verilmemiş, kaynak kodları içeren sıkıştırılmış dosyada verilmiştir (dp.flyweight.solution1 paketleri. a: en uygun olmayan bu asıl çözüm. b: Doğru sonuç vermeyen alt maddelerle ilgili çözümler).
- Ör. 100gr barutta 75gr potasyum nitrat, 15gr karbon, 10gr kükürt bulunur.
- Yukarıdaki çözüm bu tür bileşikler destekler ancak örneğin karbon başka gramajla başka karışımlarda da bulunacaktır.
- Sadece gram bilgisi farklı olan, kalan durum bilgisi aynı olan çok sayıda karbon nesnesi mi oluşturulmalıdır?
- İşin doğrusu, gram bilgisini madde sınıfından alıp, ayrı bir sınıfa taşımaktır.31

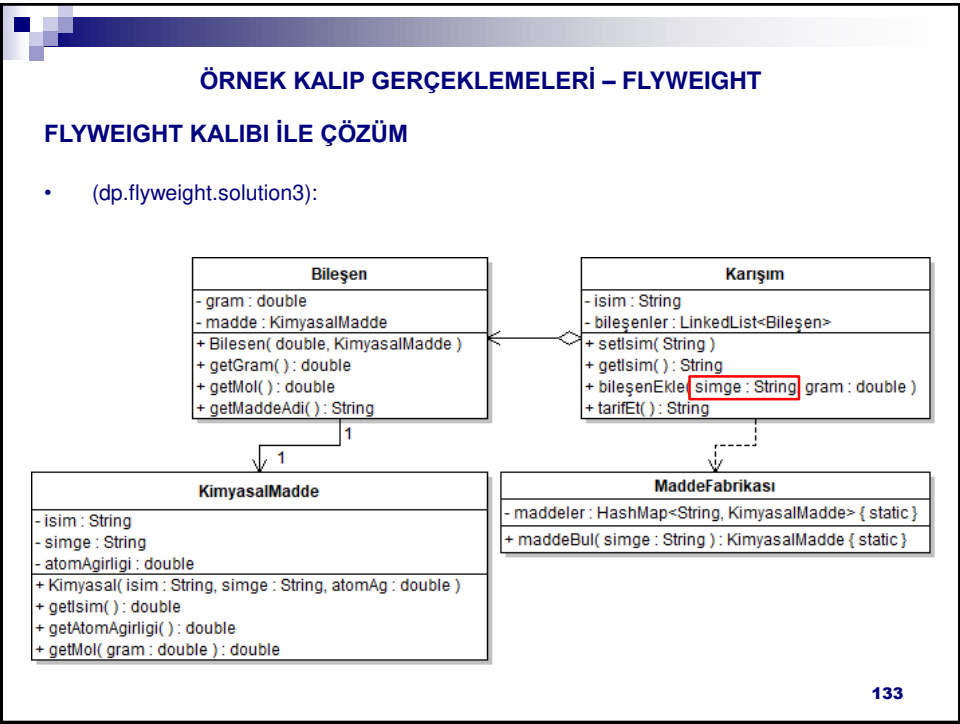
ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

DAHA DOĞRU MODELLEME:

- Gram bilgisinin madde sınıfından ayrıldığı yeni tasarım (dp.flyweight.solution2):



- Bu noktada gram, dışsal (extrinsic) bilgi olarak dikkatimizi çeker.
- Çok sayıda madde nesnesi olacağını da düşünüp Flyweight kalıbına yöneliriz.



ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

FLYWEIGHT KALIBI İLE ÇÖZÜM

- Kaynak kodlar:

```
package dp.flyweight.solution3;

public class KimyasalMadde {
    private String isim;
    private String simge;
    private double atomAğırlığı;

    public KimyasalMadde(String isim, String simge, double atomAğırlığı) {
        this.isim = isim; this.simge = simge;
        this.atomAğırlığı = atomAğırlığı;
    }

    public String getİsim() { return isim; }
    public String getSimge() { return simge; }
    public double getAtomAğırlığı() { return atomAğırlığı; }
    public double getMol( double gram ) {
        return gram/atomAğırlığı;
    }
}
```

134

ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

FLYWEIGHT KALIBI İLE ÇÖZÜM

- Kaynak kodlar:

```
package dp.flyweight.solution3;

public class Bilesen {
    private double gram;
    private KimyasalMadde madde;

    public Bilesen( double gram, KimyasalMadde madde ) {
        this.gram = gram; this.madde = madde;
    }
    public double getGram( ) { return gram; }
    public double getMol( ) { return madde.getMol(gram); }
    public String getMaddeAdi( ) { return madde.getIsim( ); }
}
```

135

ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

FLYWEIGHT KALIBI İLE ÇÖZÜM

- Kaynak kodlar:

```
package dp.flyweight.solution3;
import java.util.*;
public class Karisim {
    private String isim;
    private LinkedList<Bilesen> bilesenler = new LinkedList<Bilesen>();
    public String getIsim() { return isim; }
    public void setIsim(String isim) { this.isim = isim; }
    public void bilesenEkle( String simge, double gram ) {
        maddeler.add( new Bilesen(gram, MaddeFabrikasi.maddeBul(simge)) );
    }
    public String tarifEt( ) {
        String tarif = isim;
        for( Bilesen bilesen : bilesenler )
            tarif += "\n" + bilesen.getGram() + "gr " +
                bilesen.getMaddeAdi();
        return tarif;
    }
}
```

136

ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

FLYWEIGHT KALIBI İLE ÇÖZÜM

- Kaynak kodlar:

```
package dp.flyweight.solution3;
import java.util.*;
public class MaddeFabrikasi {
    private static HashMap<String, KimyasalMadde> maddeler = new
        HashMap<String, KimyasalMadde>();
    static {
        maddeler.put( "C", new KimyasalMadde("Karbon", "C", 12.0107) );
        maddeler.put( "S", new KimyasalMadde("Kükürt", "S", 32.066) );
        maddeler.put( "KNO3", new KimyasalMadde("Potasyum Nitrat", "KNO3",
            101.1032) );
        maddeler.put( "NaNO3", new KimyasalMadde("Sodyum Nitrat", "NaNO3",
            84.9947) );
    }
    public static KimyasalMadde maddeBul( String simge ) {
        return maddeler.get(simge);
    }
}
```

137

ÖRNEK KALIP GERÇEKLEMELERİ – FLYWEIGHT

FLYWEIGHT KALIBI İLE ÇÖZÜM

- Kaynak kodlar:

```
package dp.flyweight.solution3;
public class MainApp {
    public static void main(String[] args) {
        Karisim karaBarut = new Karisim();
        karaBarut.setIsim("Kara barut");
        karaBarut.maddeEkle("NaNO3", 75);
        karaBarut.maddeEkle("C", 15);
        karaBarut.maddeEkle("S", 10);
        Karisim temizBarut = new Karisim();
        temizBarut.setIsim("Temiz barut");
        temizBarut.maddeEkle("KNO3", 75);
        temizBarut.maddeEkle("C", 14);
        temizBarut.maddeEkle("S", 11);
        System.out.println( karaBarut.tarifEt() );
        System.out.println( temizBarut.tarifEt() );
    }
}
```

138

YAPISAL (STRUCTURAL) KALIPLAR

PROXY:

- Amaç:
 - Bir nesneye erişimi denetlemek için, bu nesnenin yerine geçecek bir başka nesneyi araya koymak.
- Örnek:
 - Ekrana bir resim çizmek zaman alıcı bir işlem olabilir.
 - Bu nedenle kelime işlem programları belgedeki resimleri ancak ilgili sayfa ekranda gösterileceği zamanda yükler ve çizer.

```
classDiagram
    class ATextDocument {
        image
    }
    class AImageProxy {
        fileName
    }
    class AImage {
        data = ...
    }
    ATextDocument --> AImageProxy : image
    AImageProxy ..> AImage : fileName
```

139

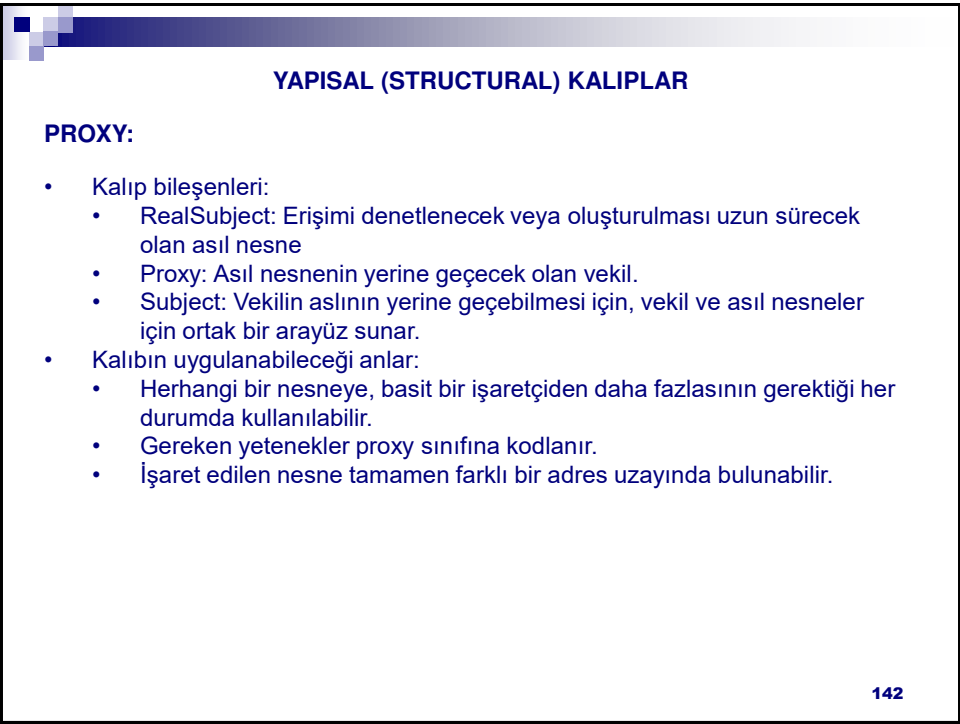
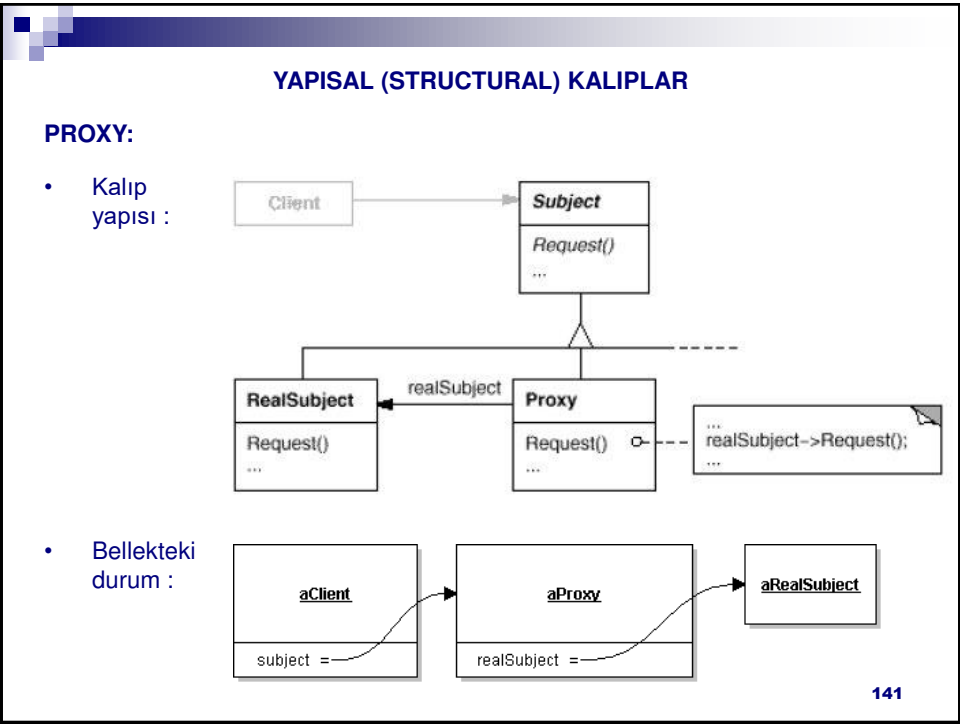
YAPISAL (STRUCTURAL) KALIPLAR

PROXY:

- Örnek çözüm:

```
classDiagram
    class DocumentEditor {
    }
    class Graphic {
        Draw()
        GetExtent()
        Store()
        Load()
    }
    class Image {
        Draw()
        GetExtent()
        Store()
        Load()
        imageImp
        extent
    }
    class ImageProxy {
        Draw()
        GetExtent()
        Store()
        Load()
        fileName
        extent
    }
    DocumentEditor --> Graphic
    Graphic <|-- Image
    Graphic <|-- ImageProxy
    ImageProxy ..> Image : image
```

140



ÖRNEK KALIP GERÇEKLEMELERİ – PROXY

ÇÖZÜLECEK PROBLEM:

- Biz izlediğin-kadar-öde sunucusuna gelen taleplerin emniyet altına alınması.
 - Sunucu kendisine verilen adrese istenilen programı gönderecek.
 - Ancak göndermeden önce istemcinin ödeme yapıp yapmadığına bakılmalı.
 - Sunucu yazılımında bu yetenek yok ve yazılımın kaynak koduna erişemiyoruz.
 - Araya koyacağımız bir vekil bu işi çözer.
 - (Protection proxy)

143

ÖRNEK KALIP GERÇEKLEMELERİ – PROXY

ÇÖZÜM:

StreamDestination

- server : StreamSource

- address : String

+ StreamDestination(address : String)

+ getAddress() : String

+ setServer(StreamSource)

+ makePayment(double) : boolean

+ requestProgram(programID : String) : boolean

«interface» StreamSource

+ stream(programID, destination : String) : boolean

StreamServer

+ stream(programID, destination : String) : boolean

StreamProxy

- original : StreamSource

+ StreamProxy(original : StreamSource)

+ stream(programID, destination : String) : boolean

+ checkPayment(programID, destination : String) : boolean

MainApp

144

72

ÖRNEK KALIP GERÇEKLEMELERİ – PROXY

ÇÖZÜM:

- Kaynak kodlar:

```
package dp.proxy.example;
public interface StreamSource {
    public boolean stream(String programID, String destination);
}

public class StreamServer implements StreamSource {
    public boolean stream(String programID, String destination) {
        // Programı hazırla ve gönder
        return true;
    }
}
```

145

ÖRNEK KALIP GERÇEKLEMELERİ – PROXY

ÇÖZÜM:

- Kaynak kodlar (devam):

```
public class StreamDestination {
    private StreamSource server;
    private String address;

    public StreamDestination(String address) {this.address = address; }
    public String getAddress() { return address; }
    public void setServer(StreamSource server) { this.server = server; }
    public boolean makePayment( double amount ) {
        //ödemeyi yap.
        return true;
    }
    public boolean requestProgram( String programID ) {
        return server.stream(programID, address);
    }
}
```

146

ÖRNEK KALIP GERÇEKLEMELERİ – PROXY

ÇÖZÜM:

- Kaynak kodlar (devam):

```
public class StreamProxy implements StreamSource {
    private StreamSource original;
    public StreamProxy(StreamSource original) {
        this.original = original;
    }
    public boolean stream(String programID, String destination) {
        if( checkPayment(programID, destination) )
            return original.stream(programID, destination);
        else
            return false;
    }
    public boolean checkPayment(String programID, String destination) {
        // Ödemeyi destination parametresine göre kontrol et
        // Ödeme yapılmışsa true, aksi halde false döndür.
        return false;
    }
}
```

147

ÖRNEK KALIP GERÇEKLEMELERİ – PROXY

ÇÖZÜM:

- Kaynak kodlar (devam):

```
public class MainApp {
    public static void main(String[] args) {
        StreamServer realServer = new StreamServer();
        StreamProxy proxyServer = new StreamProxy(realServer);
        StreamDestination client = new StreamDestination("12:aa:34:ff:54");
        client.setServer(proxyServer);
    }
}
```

148

1.3. DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

149

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

CHAIN OF RESPONSIBILITY:

- Amaç:
 - Birbirleri ile ilişkili bir nesneler zinciri veya hiyerarşisindeki bir nesnenin, aldığı bir mesajı işleyemeyeceği durumlarda, bu mesajın gerekli işlemi yapabilecek bir başka nesneye iletmek.
 - Mesajın işlenememesinin nedeni nesnenin arayüzünde o mesajın olmaması değil, nesnenin mesajı işleyecek yetki veya bilgiye sahip olmamasıdır.
- Örnek:
 - Kullanıcı bir dialog kutusundaki bileşenlerden birinin işlevi hakkında yardım almak istiyor.
 - Eğer o bileşene özel bir yardım içeriği varsa gösterilir, aksi halde yardım mesajı o bileşeni içeren bir üst bileşene iletilir.
 - İletim mesaj bir nesne tarafından cevaplanana kadar veya zincirin sonuna gelinceye dek sürer.
 - Bir bileşen, hangi üst bileşen içinde yer aldığını bilecektir.

150

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

CHAIN OF RESPONSIBILITY:

aPrintButton

handler

anOKButton

handler

aSaveDialog

handler

aPrintDialog

handler

anApplication

handler

specific

general

151

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

CHAIN OF RESPONSIBILITY:

aPrintButton

[I can Help]
HandleHelp()

aPrintDialog

[I can Help]
HandleHelp()

anApplication

152

76

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

CHAIN OF RESPONSIBILITY:

Çözümü oluşturacak sınıf yapısı:

```
classDiagram
    class HelpHandler {
        HandleHelp()
    }
    class Application
    class Widget
    class Dialog
    class Button {
        HandleHelp()
        ShowHelp()
    }
    HelpHandler <|-- Application
    HelpHandler <|-- Widget
    Widget <|-- Dialog
    Widget <|-- Button
    HelpHandler --> HelpHandler : handler->HandleHelp()
```

153

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

CHAIN OF RESPONSIBILITY:

Kalıp yapısı:

```
classDiagram
    class Client
    class Handler {
        HandleRequest()
        successor
    }
    class ConcreteHandler1 {
        HandleRequest()
    }
    class ConcreteHandler2 {
        HandleRequest()
    }
    Client --> Handler
    Handler <|-- ConcreteHandler1
    Handler <|-- ConcreteHandler2
    Handler --> Handler : successor
```

154

Client

Handler

ConcreteHandler1

ConcreteHandler2

aClient

aConcreteHandler

aConcreteHandler

aHandler

successor

successor

Client: Yanıtlanacak mesajın kaynağı.

Handler: Yanıtlanacak mesajı tanımlayan arayüz.

Zincir bağlantısını da (successor) gerçekleyebilir (Soyut sınıf olarak).

Concrete HandlerX: Mesajı yanıtlar veya bağlantılı olduğu bir başkasına iletir.

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

CHAIN OF RESPONSIBILITY:

- Kalıbın uygulanabileceği anlar:
 - Bir mesajı yanıtlayabilecek birden fazla nesnenin bulunduğu ve yanıt verecek nesnenin önceden bilinmeyeceği anlarda.
 - Bir mesajı yanıtlayabilecek nesnelerin çalışma anında tanımlanabileceği anlarda.
- Kalıbın zayıf yönleri:
 - Zincir doğru yapılandırılmazsa, bir mesajın yanıtlanacağını garanti bulunmaz.
- Gerçekleme ayrıntıları:
 - Halihazırda bir zincir yoksa, zinciri oluşturacak metod(lar) Handler soyut sınıfında veya ConcreteHandler gerçeklemelerinde ayrıca kodlanır.
 - İsteğin birden fazla türden olabileceği durumlarda ya birden fazla HandleRequestX metodları tanımlanır, ya da isteği simgeleyen bir kalıtım hiyerarşisi kurulabilir.

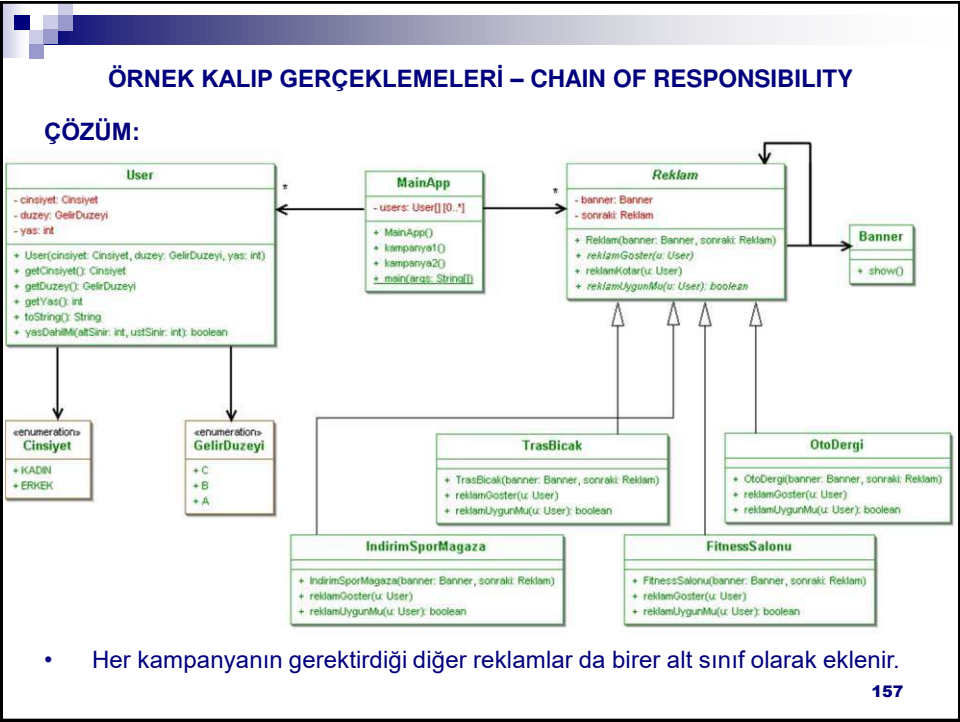
155

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜLECEK PROBLEM:

- Bir reklam yönetim yazılımı hazırlanıyor.
 - Farklı web sitelerinde kampanyaların çeşitli kurallarına göre çeşitli reklamlar gösterilmek isteniyor.
 - Kampanya 1:
 - Spor sitesinin 18-39 yaş arası erkek ziyaretçilerine traş bıçağı reklamı göster, 18-39 yaş arası kadın ziyaretçilerine fitness salonu reklamı göster, diğer ziyaretçilere spor mağazasındaki indirimin reklamını göster.
 - Kampanya 2:
 - Haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamını göster, aksi halde spor sitesinin kampanyasını aynen uygula.
 - Kampanya 3:
 - Ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından birini göster.
 - ...

• Kaynak: YES 156



ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar

```
package dp.chainOfResponsibility.example;
public abstract class Reklam {
    private Banner banner;    private Reklam sonraki;
    public Reklam(Banner banner, Reklam sonraki) {
        this.banner = banner; this.sonraki = sonraki;
    }
    public abstract boolean reklamUygunMu ( User u );
    public abstract void reklamGoster( User u );
    public final void reklamKotar( User u ) {
        if( reklamUygunMu(u) ) {
            System.out.println( this.getClass().getName() + "->" + u );
            reklamGoster(u);
        }
        else if( sonraki != null ) {
            System.out.println( this.getClass().getName() + "->" +
                sonraki.getClass().getName() );
            sonraki.reklamKotar(u);
        }
    }
}
```

158

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.chainOfResponsibility.example;
public class TrasBicak extends Reklam {
    public TrasBicak(Banner banner, Reklam sonraki) {
        super(banner, sonraki);
    }
    public void reklamGoster(User u) {
        //Gerekli komutlar
    }
    public boolean reklamUygunMu(User u) {
        if( u.getCinsiyet() == Cinsiyet.ERKEK )
            if( u.yasDahilMi(18, 39))
                return true;
        return false;
    }
}
```

159

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.chainOfResponsibility.example;
public class FitnessSalonu extends Reklam {
    public FitnessSalonu(Banner banner, Reklam sonraki) {
        super(banner, sonraki);
    }
    public void reklamGoster(User u) {
        //Gerekli komutlar
    }
    public boolean reklamUygunMu(User u) {
        if( u.getCinsiyet() == Cinsiyet.KADIN )
            if( u.yasDahilMi(18, 39))
                return true;
        return false;
    }
}
```

160

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.chainOfResponsibility.example;
public class OtoDergi extends Reklam {
    public OtoDergi(Banner banner, Reklam sonraki) {
        super(banner, sonraki);
    }
    public void reklamGoster(User u) {
        //Gerekli komutlar
    }
    public boolean reklamUygunMu(User u) {
        if( u.getCinsiyet() == Cinsiyet.ERKEK
            && u.getDuzey() == GelirDuzeyi.A )
            if( u.yasDahilMi(18, 29))
                return true;
        return false;
    }
}
```

161

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.chainOfResponsibility.example;
public class IndirimSporMagaza extends Reklam {
    public IndirimSporMagaza(Banner banner, Reklam sonraki) {
        super(banner, sonraki);
    }
    public void reklamGoster(User u) {
        //Gerekli komutlar
    }
    public boolean reklamUygunMu(User u) { return true; }
}
```

- Her kampanyanın gerektirdiği diğer reklamlar da birer alt sınıf olarak eklenir.

162

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.chainOfResponsibility.example;
public class MainApp {
    private User[] users;
    public MainApp( ) {
        users = new User[3];
        users[0] = new User( Cinsiyet.ERKEK, GelirDuzeyi.C, 23 );
        users[1] = new User( Cinsiyet.ERKEK, GelirDuzeyi.A, 53 );
        users[2] = new User( Cinsiyet.KADIN, GelirDuzeyi.B, 33 );
    }
    public void kampanya1( ) {
        Reklam[] reklamlar = new Reklam[3];
        reklamlar[2] = new IndirimSporMagaza(new Banner(), null);
        reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2]);
        reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1]);
        reklamlar[0].reklamKotar(users[0]);
        reklamlar[0].reklamKotar(users[1]);
        reklamlar[0].reklamKotar(users[2]);
    }
}
//sonraki yansıda devam edecek
```

163

ÖRNEK KALIP GERÇEKLEMELERİ – CHAIN OF RESPONSIBILITY

ÇÖZÜM:

- Kaynak kodlar (devam)

```
public void kampanya2( ) {
    Reklam[] reklamlar = new Reklam[4];
    reklamlar[3] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[2] = new FitnessSalonu(new Banner(), reklamlar[3]);
    reklamlar[1] = new TrasBicak(new Banner(), reklamlar[2]);
    reklamlar[0] = new OtoDergi(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}
public static void main(String[] args) {
    MainApp app = new MainApp();
    app.kampanya1();
    app.kampanya2();
}
} //end class
```

- Not: Reklam ile somut sınıflar arasında Template Method kalıbı kurulu. Kitap sırasına geçince bu kalıbı henüz görmemiş olduk, ileride göreceğiz.

164

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

COMMAND:

- Amaç:
 - Nesnelere giden istekleri de birer nesne haline çevirmek.
 - Bir başka deyişle: Metotları nesne yapmak!
 - Böylece mesajları kuyruklamak veya mesajların güncesini tutmak gibi işlemler mümkün olabilir.
- Örnekler:
 - GUI kütüphanelerinde, örneğin bir menü seçeneği veya bir düğmeye tıklandığında herhangi bir nesnenin herhangi bir metodunu çalıştırmak
- Çözüm: 'Herhangi bir nesnenin herhangi bir metodu' yerine, belli bir arayüzün belli bir metodu çalıştırılır.

```
classDiagram
    class Application {
        Add(Document)
    }
    class Menu {
        Add(Menuitem)
    }
    class MenuItem {
        Clicked()
    }
    class Command {
        Execute()
    }
    Application o--> Menu
    Menu o--> MenuItem
    MenuItem o--> Command : command
    MenuItem ..> Command : command->Execute()
```

The diagram illustrates the Command Pattern. It shows four classes: Application, Menu, MenuItem, and Command. Application has a method Add(Document). Menu has a method Add(Menuitem). MenuItem has a method Clicked(). Command has a method Execute(). Application has an aggregation relationship with Menu. Menu has an aggregation relationship with MenuItem. MenuItem has an aggregation relationship with Command, labeled 'command'. A note on the MenuItem class indicates that it calls 'command->Execute()'.

165

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

COMMAND:

- Çözümün mümkün kıldığı olasılıklar:
 - Bir menü seçeneği ile (MenuItem nesnesi) bir dosya açma komutu ilişkilendirilebilir.

```
classDiagram
    class Application {
        Add(Document)
    }
    class Command {
        Execute()
    }
    class OpenCommand {
        Execute()
        AskUser()
    }
    Application o--> OpenCommand : application
    Command <|-- OpenCommand
```

The diagram illustrates the Command Pattern. It shows three classes: Application, Command, and OpenCommand. Application has a method Add(Document). Command is an abstract class with a method Execute(). OpenCommand is a concrete class that inherits from Command and implements the Execute() method. OpenCommand has a method AskUser(). Application has an aggregation relationship with OpenCommand, labeled 'application'. A note on the OpenCommand class indicates that it calls 'name = AskUser()', 'doc = new Document(name)', 'application->Add(doc)', and 'doc->Open()'.

166

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

COMMAND:

- Çözümün mümkün kıldığı olasılıklar:
 - Önceki şekildeki Application sınıfı birden fazla Document örneği içerebilir.
 - Bunlardan biri üzerinde metin yapıştırma komutunun gerçekleşmesi:

```
classDiagram
    class Command {
        Execute()
    }
    class Document {
        Open()
        Close()
        Cut()
        Copy()
        Paste()
    }
    class PasteCommand {
        Execute()
    }
    Command <|-- PasteCommand
    PasteCommand --> Document : document
```

167

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

COMMAND:

- Kalıp yapısı:

```
classDiagram
    class Client
    class Invoker {
        <<aggregates>>
    }
    class Command {
        Execute()
    }
    class Receiver {
        Action()
    }
    class ConcreteCommand {
        Execute()
        state
    }
    Client --> Invoker
    Client --> Receiver
    Invoker --> Command
    Invoker --> ConcreteCommand
    Command <|-- ConcreteCommand
    ConcreteCommand --> Receiver : receiver
```

168

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

COMMAND:

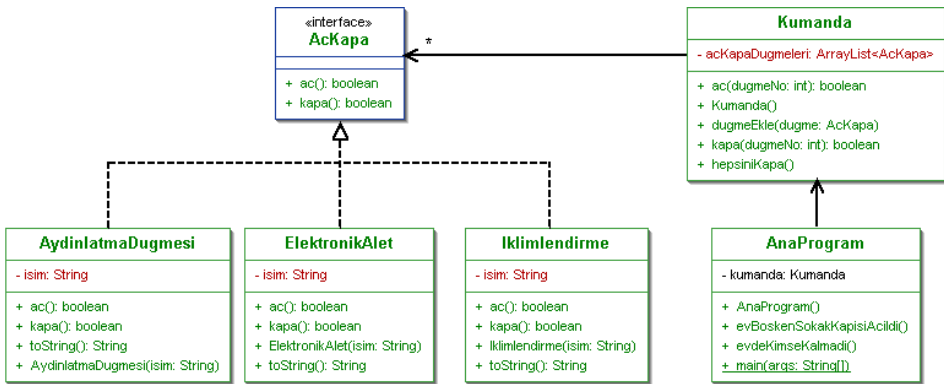
- Kalıp bileşenleri:
 - Command: Komut tanımlama arayüzü
 - ConcreteCommand: Komut gerçeklemeleri. Komutlar belli bir alıcının eylemlerini kullanacak şekilde gerçekleştirilebilir.
 - Client: Bir komut gerçeklemesi nesnesi oluşturur ve komutun alıcısını belirler.
 - Receiver: Alıcı nesne gerçeklemesi. Komut gerçeklemesi hakkında bilgi sahibi olması gerekebilir. Herhangi bir nesne alıcı olabilir, veya herhangi bir alıcısı olmayan bir komut nesnesi de bulunabilir.

169

ÖRNEK KALIP GERÇEKLEMELERİ – COMMAND

ÇÖZÜLECEK PROBLEM:

- Bir akıllı ev yazılımı üretiyoruz. Yazılımda basit bir açma/kapama kumandası ekranı var. Kumanda ekranındaki aç/kapa düğmelerine odaların ışıklarını, televizyonları, vb. açma/kapama işlevleri kazandıracağız.



- Esinlenme: Head First DP

170

ÖRNEK KALIP GERÇEKLEMELERİ – COMMAND

ÇÖZÜM:

- Kaynak kodlar

```
package dp.command.example;
public interface AcKapa {
    public boolean ac( );
    public boolean kapa( );
}

package dp.command.example;
public class AydinlatmaDugmesi implements AcKapa {
    private String isim;
    public AydinlatmaDugmesi(String isim) { this.isim = isim; }
    public String toString( ) { return "AcKapa:" + isim; }
    public boolean ac() { return true; }
    public boolean kapa() { return true; }
}
```

171

ÖRNEK KALIP GERÇEKLEMELERİ – COMMAND

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.command.example;
import java.util.*;
public class Kumanda {
    private ArrayList<AcKapa> acKapaDugmeleri;
    public Kumanda( ) { acKapaDugmeleri = new ArrayList<AcKapa>(); }
    public void dugmeEkle( AcKapa dugme ) { acKapaDugmeleri.add(dugme); }
    public boolean ac( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).ac();
    }
    public boolean kapa( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).kapa();
    }
    public void hepsiniKapa( ) {
        for( AcKapa dugme : acKapaDugmeleri )
            dugme.kapa();
    }
}
```

172

ÖRNEK KALIP GERÇEKLEMELERİ – COMMAND

ÇÖZÜM:

- Kaynak kodlar (devam)

```
public class AnaProgram {
    Kumanda kumanda;
    public AnaProgram( ) {
        kumanda = new Kumanda();
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Antre" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Salon" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Mutfak" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Oda" ) );
        kumanda.dugmeEkle( new ElektronikAlet( "Vestel TV" ) );
        kumanda.dugmeEkle( new Iklimlendirme( "Kombi" ) );
    }
    public void evBoskenSokakKapisiAcildi( ) {
        kumanda.ac(0); kumanda.ac(5);
    }
    public void evdeKimseKalmadi( ) { kumanda.hepsiniKapa( ); }
    public static void main(String[] args) {
        AnaProgram prg = new AnaProgram();
        prg.evBoskenSokakKapisiAcildi();
        System.out.println("Evden çıkılıyor");
        prg.evdeKimseKalmadi();
    }
}
```

173

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

ITERATOR:

- Amaç:
 - Parçalardan oluşan bir bileşenin parçalarını,
 - bileşenlerin iç yapısını açığa çıkarmadan,
 - istenilen bir biçimde dolaşmak.
- Örnek:
 - Bir veri yapısı içerisinde dolaşmak.
- Sorun:
 - Veri yapısının arayüzünü birçok farklı dolaşım türü ile şişirmek istemeyiz.
- Not: Java'da herhangi bir Collection'dan, kendisini gezmek üzere, bir Iterator nesnesi istenebilir, ancak bu nesne Iterator tasarım kalıbına tam olarak uymaz
 - Java'nın Iterator'u daha basit.
 - Basit demek kötü demek değildir, çoğu durumda Java'nın Iterator'u da işimize yarar.
 - KISS: Java'nın Iterator'u işinize yarıyorsa onu kullanın.

174

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

ITERATOR:

- Kalıp yapısı:

```
classDiagram
    class Aggregate {
        +CreateIterator()
    }
    class ConcreteAggregate {
        +CreateIterator()
    }
    class Iterator {
        +First()
        +Next()
        +IsDone()
        +CurrentItem()
    }
    class ConcreteIterator {
    }
    class Client {
    }
    Aggregate <|-- ConcreteAggregate
    Iterator <|-- ConcreteIterator
    Client --> Aggregate
    Client --> Iterator
    ConcreteAggregate ..> ConcreteIterator : return new ConcreteIterator(this)
```
- Kalıp bileşenleri:
 - Aggregate/ConcreteAggregate: Parçaları arasında dolaşılacak nesnenin arayüzü / gerçeklemesi.
 - (Gezgin)Iterator/Concreteliterator: Dolaşma işleminin arayüzü / gerçeklemesi.
 - java.util paketindeki veri yapıları bu kalıbı kullanır.

177

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

ITERATOR:

- Gerçekleme ayrıntıları:
 - Parçalar arasında dolaşırken değiştirme işlemine izin verilecek mi?
 - Dolaşım algoritmasını kim gerçekleyecek ve dolaşılacak nesneler birer birer mi yoksa hepsi birden mi istekçiye verilecek?
 - Gezgin'e (Iterator'a) komutlar verme sırasını Client (istekçi) belirler ve parçaları gezginden tek tek okur.
 - İstekçi bir dolaşım istemini gezgin'e verir ve gezgin parçaları bir dizide döndürür.

178

ÖRNEK KALIP GERÇEKLEMELERİ – ITERATOR

ÇÖZÜLECEK PROBLEM:

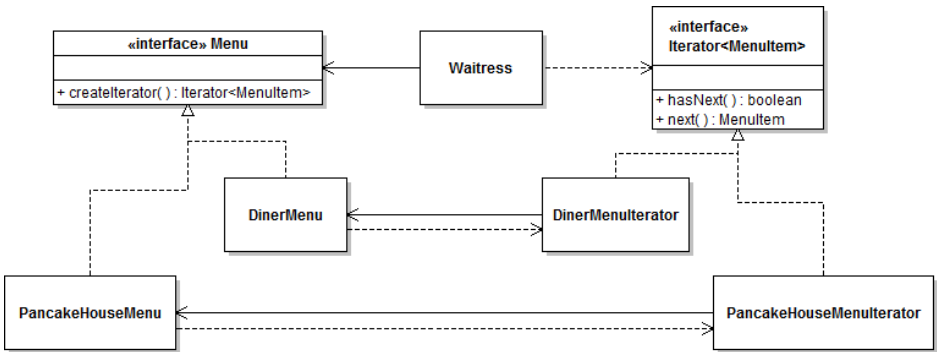
- Online yemek siparişi verebileceğiniz bir firma hızla kurulmak isteniyor.
- Bu amaçla biri kahvaltılıkta, biri yemekte uzman iki firma satın alınıyor.
- Menü elemanları gerçeklemesi aynı denk gelmiş ama her firmanın kendi ayrı yemek menüsü gerçeklemesi var.
- Farklı menü gerçeklemelerinden yola çıkarak, birleştirilmiş menü müşterilere ortak bir kod üzerinden nasıl sunulacak?
 - Ortak kod söz konusu olmazsa, her yeni satın alınan firma için menü sunma koduna yeni bir kod bloğu eklenmek zorunda kalınacak!
- İşler daha da çetrefil hale gelebilir:
 - İki firma da vejetaryenleri düşünüp hangi yemeklerinin et içermediğini işaretlemişler.
 - Bir vejetaryen menü sunmak için iki farklı firmanın menüsünü et ayrı dolaşarak et içermeyenleri göstermek gerekecek.
- Böylesi bir acemi çözümün kaynak kodu: Eclipse'ten.

• Kaynak: Head First DP

179

ÖRNEK KALIP GERÇEKLEMELERİ – ITERATOR

ITERATOR KALIBI İLE ÇÖZÜM:



180

ÖRNEK KALIP GERÇEKLEMELERİ – ITERATOR

ITERATOR KALIBI İLE ÇÖZÜM:

```
package iterator.exampleWithPattern;
import java.util.*;

public class Waitress {
    private Menu pancakeHouseMenu, dinerMenu;
    public Waitress(Menu pancakeHouseMenu, Menu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu; this.dinerMenu = dinerMenu;
    }
    public void printMenu() {
        Iterator<MenuItem> pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator<MenuItem> dinerIterator = dinerMenu.createIterator();
        System.out.println("MENU\n---\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }
    private void printMenu(Iterator<MenuItem> iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = iterator.next();
            printMenuItem(menuItem);
        }
    }
    private void printMenuItem( MenuItem menuItem ) {
        System.out.print(menuItem.getName() + ", ");
        System.out.print(menuItem.getPrice() + " -- ");
        System.out.println(menuItem.getDescription());
    }
}
```

181

ÖRNEK KALIP GERÇEKLEMELERİ – ITERATOR

ITERATOR KALIBI İLE ÇÖZÜM:

```
public void printVegetarianMenu() {
    System.out.println("\nVEGETARIAN MENU\n---\nBREAKFAST");
    printVegetarianMenu(pancakeHouseMenu.createIterator());
    System.out.println("\nLUNCH");
    printVegetarianMenu(dinerMenu.createIterator());
}

private void printVegetarianMenu(Iterator<MenuItem> iterator) {
    while (iterator.hasNext()) {
        MenuItem menuItem = iterator.next();
        if (menuItem.isVegetarian()) {
            printMenuItem(menuItem);
        }
    }
}
}
```

182

ITERATOR KALIBI İLE ÇÖZÜM:

183

ITERATOR KALIBI İLE ÇÖZÜM:

184

ÖRNEK KALIP GERÇEKLEMELERİ – ITERATOR

ITERATOR KALIBI İLE ÇÖZÜM:

```
package iterator.exampleWithPattern;
import java.util.*;
public class DinerMenuIterator implements Iterator<MenuItem>{
    MenuItem[] list; int position = 0;
    //implementing the interface by using arrays. Full version is in zip file.
}

package iterator.exampleWithPattern;
import java.util.*;
public class PancakeHouseMenuIterator implements Iterator<MenuItem> {
    ArrayList<MenuItem> items; int position = 0;
    //implementing the interface by using ArrayLists. Full version is in zip file.
}
```

185

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- Amaç:
 - Bir nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmek.
 - Böylece nesnelerin ikincil saklama ortamlarında saklanması, geri yüklenmesi, ağ üzerinden aktarımı mümkün olabilir.
 - Yine bu şekilde sona yapılan işlemi geri alma düzeneği (undo) kurulabilir.
 - Durum bilgisinin sadece bir kısmının saklanması da yeterli olabilir.

186

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- Kalıp yapısı:

Originator
SetMemento(Memento m)
CreateMemento()
state

Memento
GetState()
SetState()
state

Caretaker

return new Memento(state)

state = m->GetState()

- Kalıp bileşenleri:
 - Originator: Durum bilgisi saklanacak olan nesne
 - Memento: Durum bilgisinin gerekli kısımlarını saklar
 - Caretaker: Gerekli anlarda durum bilgisinin bir kopyasını alır ve elindeki kopyalardan birini geçerli durum haline getirir.

187

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- Kalıp bileşenlerinin etkileşimleri:

aCaretaker

anOriginator

aMemento

CreateMemento()

new Memento

SetState()

SetMemento(aMemento)

GetState()

188

94

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- Kalıbın uygulanabileceği anlar:
 - Nesnelerin durum bilgisinin tümü veya bir kısmının daha sonra o duruma dönebilmek üzere saklanması VE
 - Nesneye doğrudan bir işaretçinin veri gizliliği ilkesini bozmasının istenmediği anlarda.
- Sorun:
 - Veri gizliliği ilkesini sağlamak için Memento'nun metotlarına sadece Originator erişebilmelidir.
 - Bu gereksinim her dilde karşılanamaz.
 - C++: Memento metotları private yapılı ve Originator sınıfı Memento sınıfının arkadaşı (friend) olarak tanımlanır.
 - Java: Memento metotları package yapılı, Memento ve Originator sınıfı aynı pakette yer alır. Böylece Caretaker sınıfı başka paketlerde yer alabilir, ya da aynı paketdeki Caretaker sınıfına bir Memento veya State parametrelili erişim metotları konulmaz.
 - Java diğer seçenek: Memento, Originator sınıfının iç sınıfı yapılır.
 - Ancak bu durumda State clonable yapılmalı ve Originator.createMemento metodu klon döndürmelidir.

189

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- Dış sınıf ile çözüm: Önceki yansıda anlatıldığı gibi kodlanır.
- İç sınıf ile çözüm:

```
package dp.mementoV2.innerClass;
public class State implements Cloneable {
    private String durum;
    public String getDurum() {
        return durum;
    }
    void setDurum(String durum) {
        this.durum = durum;
    }
    public State clone() {
        State s = new State();
        s.durum = new String(durum);
        return s;
    }
}
```

190

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- İç sınıf ile çözüm (devam):

```
package dp.mementoV2.innerClass;
public class Originator {
    private State state;
    public Originator( ) { state = new State( ); }
    public Memento createMemento( ) {
        Memento m = new Memento( );
        m.setState(state.clone());
        return m;
    }
    public void setMemento( Memento m ) { state = m.getState(); }
    public void modify( String str ) { state.setDurum(str); }
    public String toString() { return state.getDurum(); }

    public class Memento {
        private State state;
        private State getState() { return state; }
        private void setState(State state) { this.state = state; }
    }
}
```

191

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEMENTO:

- İç sınıf ile çözüm (devam):

```
package dp.mementoV2Outer;
import dp.mementoV2.innerClass.*;
import dp.mementoV2.innerClass.Originator.Memento;

public class Caretaker {
    public void test( ) {
        Originator subject = new Originator( );
        subject.modify("ilk durum");
        Memento mem = subject.createMemento( );
        subject.modify("yeni durum");
        //bir süre sonra...
        subject.setMemento( mem );
        System.out.println(subject);
    }
    public static void main( String[] args ) {
        Caretaker ct = new Caretaker();
        ct.test();
    }
}
```

192

ÖRNEK KALIP GERÇEKLEMELERİ – MEMENTO

ÇÖZÜLECEK PROBLEM:

- Bir belge işleme programında geri alma ve yineleme işlemleri

DocumentMemento ile hiçbir şekilde ilişkisi yok

DocumentMemento

- documentState

Controller

1

Document

- currentState : DocumentMemento

- states : List<DocumentMemento>

- position : int

+ undo()

+ redo()

+ anyOperation()

*

1

```
public void undo( ) {
    position -= 1;
    currentState = states.get(position);
}

public void redo( ) {
    position += 1;
    currentState = states.get(position);
}

public void anyOperation( ) {
    states.add(currentState);
}
```

193

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

OBSERVER:

- Amaç:
 - Bir nesnenin durumu değiştiğinde, bu nesne ile ilişkili tüm diğer nesnelerin durumdan haberdar edilmelerini ve güncellenmelerini sağlamak.
 - Aralarındaki ilişkilerin çalışma anında kurulması gereken nesnelerde bu iş nasıl yapılabilir?
- Örnek:
 - Bir hesap tablosundaki bilgi değiştiğinde çeşitli çizelgelere de son durumun anında yansıtılması gerekebilir.

subject

a = 50%

b = 30%

c = 20%

observers

table

	a	b	c
x	60	30	10
y	50	30	20
z	80	10	10

bar chart

	a	b	c
height	60	30	10

pie chart

	a	b	c
percentage	50%	30%	20%

change notification

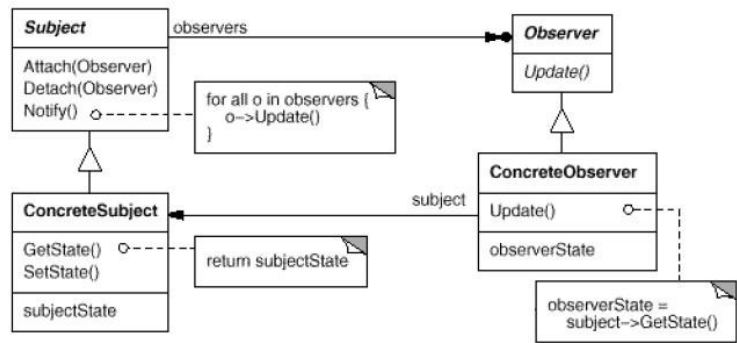
requests, modification

194

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

OBSERVER:

- Kalıp yapısı:



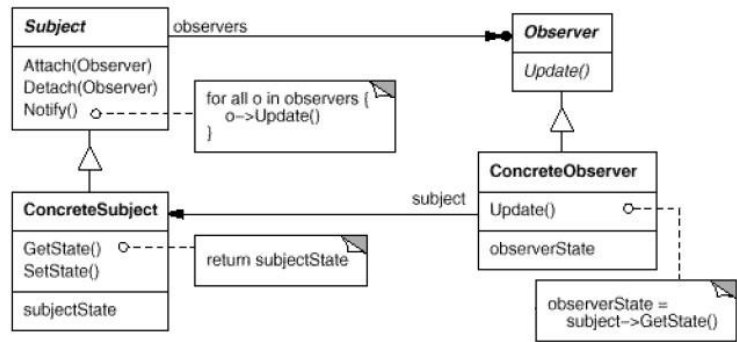
- Kalıp bileşenleri:
 - Subject: Durumundaki değişikliklerin izleneceği nesnenin arayüzü.
 - Birden fazla gözleyicisi olabilir. Bu nedenle gözleyici ekle/sil metotlarını tanımlayan soyut bir sınıf olmalıdır.
 - Observer: Gözleyici nesnelerin arayüzü.
- 195

195

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

OBSERVER:

- Kalıp yapısı:



- Kalıp bileşenleri:
 - ConcreteSubject: Subject gerçeklemeleri. Durumu değiştiğinde gözleyicilerini Observer arayüzünü kullanarak bilgilendirir.
 - ConcreteObserver: Observer gerçeklemeleri. Gözlediği nesneyi gösteren bir işaretçi tutabilir.
- 196

196

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

OBSERVER:

- Gerçekleme ayrıntıları :
 - Durum bilgisinin gözleyicilere gönderilmesi:
 - Pull model:
 - + Gözlenen sadece gözleyicileri haberdar eder, gözleyiciler ilgili durum bilgisini gözlenenden alır.
 - + Gözlenenin gözleyicilerle bağlantısını azaltan çalışma biçimidir.
 - + Gözleyici durum bilgisinin sadece istediği kısmını almayı seçebilir.
 - Gözleyici gözlenende nelerin değiştiğini kendisi çıkarmak zorunda kalır.
 - Push model: Gözlenen gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir.
 - Aynı türden gözlenen nesnelerin tümünün gözleyicilerinin ortak olması istenirse, gözlenen gözleyicilerini statik bir veri yapısı içerisinde tutulabilir.

197

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

OBSERVER:

- Gerçekleme ayrıntıları (devam):
 - Bir gözleyicinin birden fazla nesneyi gözlemesi istenebilir.
 - Gözlenen nesnenin durumundaki farklı türden değişiklikleri farklı türden gözleyicilerin izlemesi istenebilir.
 - Haber verme mesajının tetiklenmesi:
 - Gözleyiciler tetiklerse, gözlenenlerin istekçilerini kodlayan kişi tetikleme komutunu vermeyi unutabilir.
 - Gözlenen nesne tetiklerse, gözlenenlerin istekçilerinin peş peşe yapacağı durum değişiklikleri tek bir haber verme mesajı ile gönderilebileceği yerde birçok izleyiciye gidecek bir sürü mesaj gönderilmesine neden olabilir.
 - Kalıbın kullanılabileceği anlar:
 - Herhangi bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda.
 - Bir nesnedeki değişikliklerin derleme anında bilinmeyen sayıda nesneyi de etkilemesi gereken anlarda.
 - Bağlaşımı arttırmadan bir nesnenin diğer nesneleri herhangi bir durumdan haberdar etmesi istenen anlarda.

198

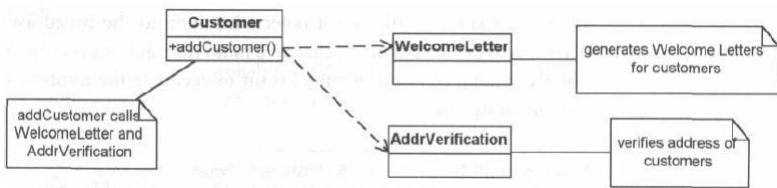
ÖRNEK KALIP GERÇEKLEMELERİ – OBSERVER

ÇÖZÜLECEK PROBLEM:

- Yazdığımız bir müşteri ilişkileri programına yeni müşteri eklendiğinde, müşteriye bir "hoş geldiniz" e-postası gönderilecek ve müşterinin ikamet adresinin doğru olup olmadığına bakılacak.

ÇÖZÜM 1:

- Bu davranışlar sabit bir şekilde kodlanır.



- Peki ya gereksinimler değişirse?

199

ÖRNEK KALIP GERÇEKLEMELERİ – OBSERVER

ÇÖZÜM 2:

- Yeni bir gereksinim ortaya çıkıyor:
 - Müşteri bir şubemizin 10km. yakınında ise ona gönderilecek mektupta indirim kuponları da olsun.
 - Benzer başka gereksinimlerin de ortaya çıkabileceğini görüp, şimdiden önlemimizi alalım.
 - Sisteme her yeni müşteri eklendiğinde, bir veya birkaç farklı işlem yapmak üzere bir alt yapı oluşturalım.
 - Müşteri eklendiği zaman sistem yapması gereken işler olabileceğinden haberdar edilsin.
 - Bu düşünce biçimi bizi observer kalıbına götürür.
 - Brick-and-mortar: Ağırlıklı olarak e-ticaret yapanlar, fiziksel binalarda yaptıkları ticareti böyle adlandırır.

200

ÖRNEK KALIP GERÇEKLEMELERİ – OBSERVER

ÇÖZÜM 2:

Observer kalıbının kullanılması:

Customer

- myObs : Vector<IObserver> {static}

+ attach(IObserver) {static}

+ detach(IObserver) {static}

+ getState()

+ notifyObs()

notifyObs metodu gözleyicilerin update metodunu çağırarak onları haberdar eder.

«interface»
IObserver

+ update (myCust : Customer)

AddrVerification

BrickAndMortar

WelcomeLetter

Sadece müşteri nesneleri izlenecekse bir soyut 'Subject' üst sınıfı hazırlamaya gerek kalmadan observer kalıbı kullanılabilir.

201

ÖRNEK KALIP GERÇEKLEMELERİ – OBSERVER

ÇÖZÜM 2:

```
package observer.example;
public interface IObserver {
    public void update (Customer myCust);
}
public class AddrVerification implements IObserver {
    public void update(Customer myCust) {
        String state = myCust.getState( );
        // TODO Gerekli işlemleri tamamla
    }
}
public class WelcomeLetter implements IObserver {
    public void update(Customer myCust) {
        String state = myCust.getState( );
        // TODO Gerekli işlemleri tamamla
    }
}
public class BrickAndMortar implements IObserver {
    //diğerleri gibi kodla
}
```

202

101

ÖRNEK KALIP GERÇEKLEMELERİ – OBSERVER

ÇÖZÜM 2:

```
package observer.example;
import java.util.*;
public class Customer {
    private static Vector<ICObserver> myObs =
        new Vector<ICObserver>( ); //Gözleyiciler ortak olacak
    public static void attach(ICObserver o){
        myObs.addElement(o);    }
    public static void detach(ICObserver o){
        myObs.remove(o);    }
    public String getState () {
        // TODO: Burada durum bilgisini döndür
        return null;    }
    public void notifyObs () {
        for( ICObserver obs : myObs ) {
            obs.update(this);
        }
    }
}
```

203

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STRATEGY:

- Amaç:
 - Aynı iş için olan farklı algoritmalar ile bu işin yapılmasını isteyecek istemcileri birbirlerinden soyutlamak.
 - Böylece kullanılan algoritmayı çalışma anında değiştirilebilir.
- Örnek:
 - Metinlerin gösteriminde kullanılabilecek alt satıra geçme (line breaking) algoritmaları çok çeşitlidir.
 - Bu algoritmaları gerçeklemek istiyoruz, ancak:
 - Bunları farklı metin uygulamalarında kullanabilmek istiyoruz.
 - Bunları aynı uygulamada istediğimiz anda birbirlerinin yerine kullanabilmek istiyoruz.

204

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STRATEGY:

Örnek çözüm:

Uygulama

- bölmeleyici: SatırBölümleme

+ metinDüzenle()

bölmeleyici.bölmele()

«abstract»
SatırBölümleme

+ bölmele() {abstract}

KesmeliBöl

+ bölmele()

KelimedenBöl

+ bölmele()

SınırdanBöl

+ bölmele()

Varsayılan bir bölümleme algoritması söz konusu ise üst sınıfın ve ilgili metodunun soyut olması zorunlu değildir ancak kalıbın genel yapısı bu şekildedir.

205

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STRATEGY:

Kalıp yapısı:

Context

ContextInterface()

Strategy

AlgorithmInterface()

ConcreteStrategyA

AlgorithmInterface()

ConcreteStrategyB

AlgorithmInterface()

ConcreteStrategyC

AlgorithmInterface()

Kalıp bileşenleri:

• Strategy: Desteklenecek tüm algoritmaların ortak arayüzünü belirler.

• ConcreteStrategyX: Algoritma gerçeklemeleri.

• Context: Algoritmaya ihtiyaç duyan nesne

• Bir/birkaç ConcreteStrategy nesnesi ile ilişkilendirilir.

• ContextInterface(): Gerekli hallerde algoritma gerçeklemesine durum bilgisini açan metot(lar).

206

103

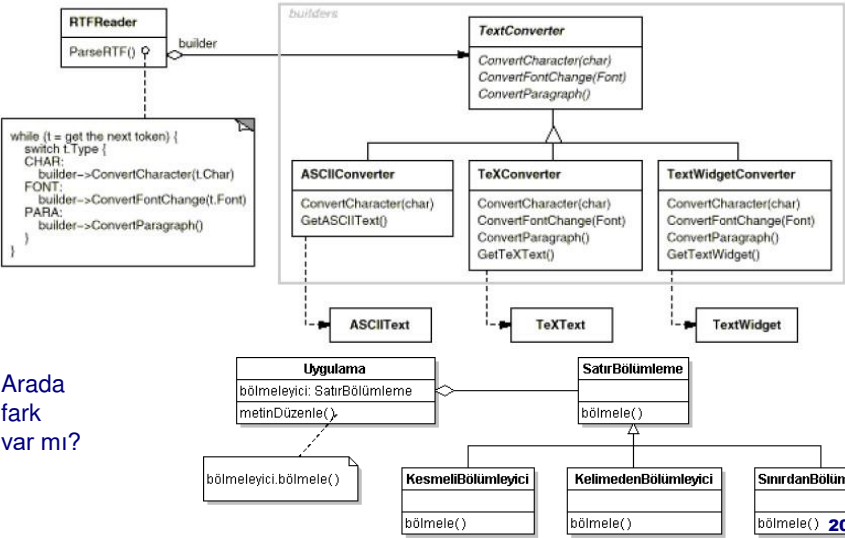
DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

STRATEGY:

- Gerçekleme ayrıntıları:
 - Strategy ve Context nesneleri seçilen algoritmayı gerçeklemek için etkileşimlerde bulunur.
 - Context nesnesini kullanacak istekçi nesneler Context nesnesine bir ConcreteStrategy nesnesi sağlar.
- Kalıbın uygulanabileceği anlar:
 - Bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda.
 - İstekçilerin kullanılacak algoritmanın gerçekleşmesini bilmemesi gerektiği hallerde.
- Kalıbın zayıf yönleri:
 - İstekçiler değişik stratejilerin varlığından haberdar olmak zorundadırlar.
 - Context'in açtığı durum bilgisinin tümüne her tür stratejinin ihtiyacı olmayabilir (haberleşmede ek yük).

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

STRATEGY ve BUILDER KALIPLARI:



Arada fark var mı?

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STRATEGY ve BUILDER KALIPLARI:

- Tasarım kalıpları benzer sınıf yapılarına sahip olabilir.
- Kalıpları farklılaştıran amaçları, kullanım yerleri ve gerçekleştirme ayrıntılarıdır.
- Örneğimizde ilişkilerin elmas tarafındaki nesne:
 - Builder kalıbında parçaları nasıl oluşturup birleştireceğini bilir ve denetler.
 - Strateji kalıbında işlerin nasıl yürütüldüğünü bilmez.

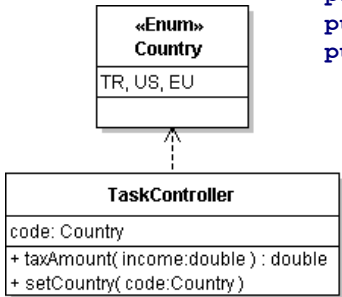
209

ÖRNEK KALIP GERÇEKLEMELERİ – STRATEGY

ÇÖZÜLECEK PROBLEM:

- Uluslar arası pazara hitap eden bir kişisel finans programı yazılıyor.
- Değişik ülkelerdeki değişik vergi kuralları ile işlem yapılması gerekmektedir.
- Ana programımız: TaskController

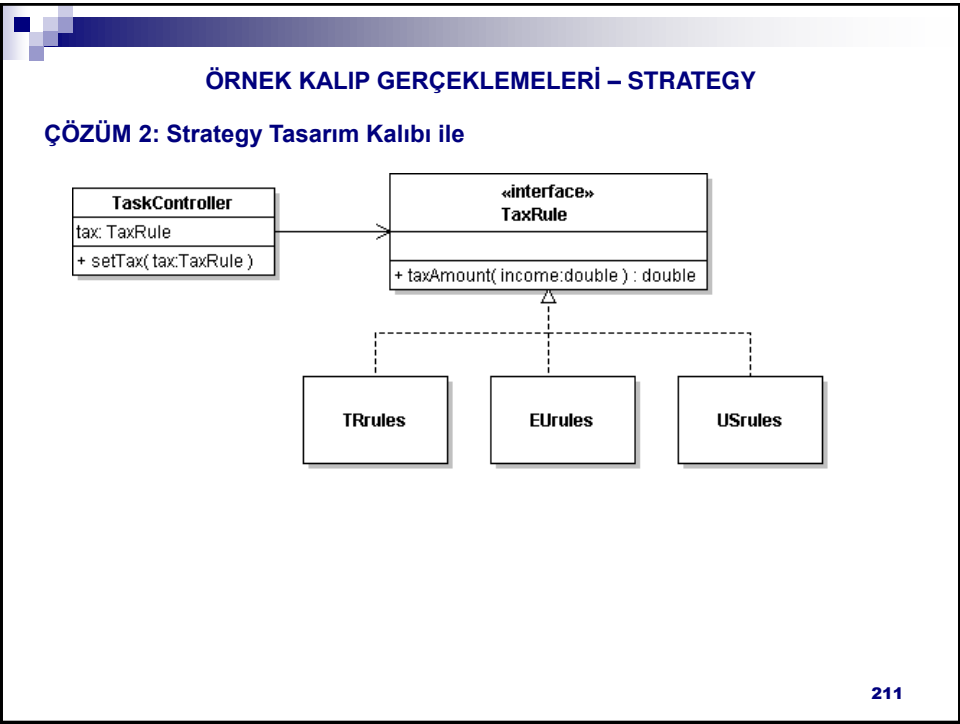
ÇÖZÜM 1: switch-case



```
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code){
        this.code = code;
    }
    public double taxAmount(double income){
        switch (code) {
            case TR: return income * 0.18;
            case EU: return income * 0.15;
            case US: return income * 0.12;
            default: return income * 0.10;
        }
    }
}
```

210

- Kaynak: DP Explained



ÖRNEK KALIP GERÇEKLEMELERİ – STRATEGY

ÇÖZÜM 2: Strategy Tasarım Kalıbı ile

```
package strategy.solution2;
public interface TaxRule {
    public double taxAmount( double income );
}
public class TaskController {
    private TaxRule tax;
    public void setTax(TaxRule tax) { this.tax = tax; }
    public double taxAmount( double income ) {
        return tax.taxAmount( income );
    }
}
public class TRrules implements TaxRule {
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

212

ÖRNEK KALIP GERÇEKLEMELERİ – STRATEGY

ÇÖZÜM 2: Strategy Tasarım Kalıbı ile

```
public class EÜrules implements TaxRule {
    public double taxAmount(double income) {
        return income * 0.15;
    }
}
public class USrules implements TaxRule {
    public double taxAmount(double income) {
        return income * 0.12;
    }
}
```

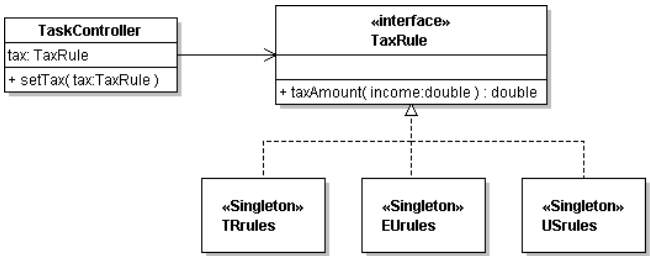
- Kalıp asıl esnekliğini vergi hesaplamanın daha karmaşık kurallara dayandığı zaman gösterir ki, gerçek hayatta da durum öyledir.
- Diğer mali işlemler için Strategy kalıbı tekrarlanır.

213

ÖRNEK KALIP GERÇEKLEMELERİ – SINGLETON

SINGLETON ve STRATEGY BİRLİKTELİĞİ:

- Kişisel finans programında farklı ülkeler için farklı kurallar olabilir, ancak belli bir ülke için sadece tek bir kural bulunur.
- Bu gereksinimi karşılamak üzere Singleton kalıbı kullanılabilir:
 - TRrules, EÜrules, USrules sınıfları Singleton olarak gerçekleştirilebilir.



214

ÖRNEK KALIP GERÇEKLEMELERİ – SINGLETON

ÇÖZÜM:

```
package singleton.example;
public class TRrules implements TaxRule {
    private static TRrules instance;
    private TRrules( ) {
        //gerekli ilklendirmeler
    }
    public static TRrules getInstance() {
        if( instance == null )
            instance = new TRrules( );
        return instance;
    }
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

- EUrules ve USrules sınıfları da benzer şekilde gerçekleştirilir.

215

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STATE:

- Amaç:
 - Bir nesnenin iç durumu değiştiğinde davranışının da değişmesini sağlamak.
 - Bu değişiklik büyük ölçekli olmalıdır.
 - Küçük ölçekli: Bir üye alanın değerinin değişmesi yüzünden bir metodun hesapladığı sonucun değişmesi
 - Büyük ölçekli: Nesnenin önceki çalışma biçimini tamamen değiştirmesi.
- Örnek:
 - Bir ağ kütüphanesinin bir parçası olarak, TCP gerçekleştirilmesi yapılıyor.
 - Bir TCP bağlantısı, herhangi bir anda şu üç durumdan birinde olabilir:
 - Established, Listen, Close
 - Bağlantı, o andaki durumuna göre farklı şekilde işler
- Esnek olmayan çözüm: Switch-case ile.
 - Benzerini önceden gördük!

216

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STATE:

- Önerilen çözüm:
 - Neyin değiştiğini bularak sarmalamak: TCP durumları
 - Parça-bütün ilişkisini kullanmaya teşvik: TCP soketi sınıfı ve soket durumunu simgeleyen üye

```
classDiagram
    class TCPConnection {
        +state
        +Open()
        +Close()
        +Acknowledge()
    }
    class TCPState {
        +Open()
        +Close()
        +Acknowledge()
    }
    class TCPEstablished {
        +Open()
        +Close()
        +Acknowledge()
    }
    class TCPListen {
        +Open()
        +Close()
        +Acknowledge()
    }
    class TCPClosed {
        +Open()
        +Close()
        +Acknowledge()
    }
    TCPConnection o--> TCPState : state
    TCPState <|-- TCPEstablished
    TCPState <|-- TCPListen
    TCPState <|-- TCPClosed
```

217

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STATE:

- Kalıp yapısı:

```
classDiagram
    class Context {
        +Request()
    }
    class State {
        +Handle()
    }
    class ConcreteStateA {
        +Handle()
    }
    class ConcreteStateB {
        +Handle()
    }
    Context o--> State : state
    State <|-- ConcreteStateA
    State <|-- ConcreteStateB
```

218

- Kalıp bileşenleri:
 - Context: Davranışı durum bilgisi ile birlikte değişen varlığı simgeler.
 - State/ConcreteStateX: Değişen davranışın arayüzü/gerçeklemeleri.
- Gerçekleme ayrıntıları:
 - Durum değişiminden genellikle Context sorumlu olur,
 - Aksi halde?

109

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

STATE:

- Kalıbın kullanılabileceği anlar:
 - Bir nesnenin çalışma şeklinin çalışma anında o nesnenin durumuna göre değişebileceği yerlerde
 - Bir nesnenin durumuna bağlı, karmaşık ve uzun olan çok kısımlı karar verme komutlarının verilmesi gerektiği anlarda.

STATE ve STRATEGY KALIPLARININ FARKI:

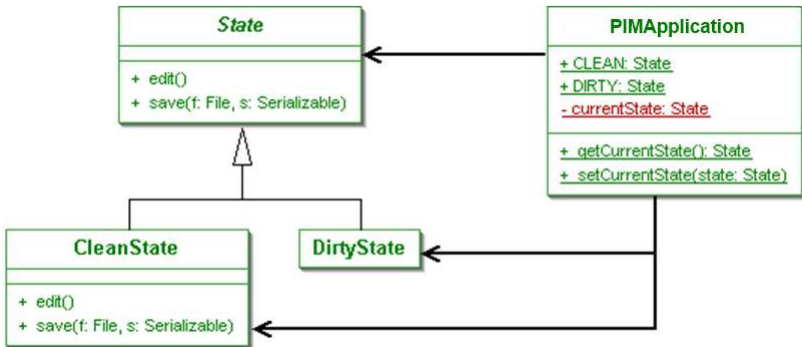
- State:
 - Context'teki durum değiştikçe, Context kendi ConcreteState'ini seçer.
 - State geçişleri açıkça yapılır (Context'te verilen komutlarla veya ConcreteState'ler içerisinden)
 - Durum geçişleri daha siktir.
- Strategy:
 - Strateji geçişleri seyrekir.
 - Strateji nesneleri Context hakkında daha az şey bilir, kendilerine gelen parametrelere göre işlem yapar.

219

ÖRNEK KALIP GERÇEKLEMELERİ – STATE

ÇÖZÜLECEK PROBLEM:

- Bir PIM yazılımında kayıtların iki durumu olabilir: Temiz ve Kirlili
• EN: Dirty and Clean
- Çözümün sınıf şeması:



- Kaynak: Applied Java Patterns

220

ÖRNEK KALIP GERÇEKLEMELERİ – STATE

ÇÖZÜM:

- Kaynak kodlar

```
package dp.state.example;
import java.io.*;
public abstract class State {
    public void save(File f, Serializable s) throws IOException {
        FileOutputStream fos = new FileOutputStream(f);
        ObjectOutputStream out = new ObjectOutputStream(fos);
        out.writeObject(s);
    }
    public void edit( ) { /*Gerekli işler*/ }
}
```

221

ÖRNEK KALIP GERÇEKLEMELERİ – STATE

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.state.example;
import java.io.*;
public class CleanState extends State {
    public void edit( ){
        PIMApplication.setCurrentState(PIMApplication.DIRTY);
        super.edit();
    }
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
    }
}
```

222

ÖRNEK KALIP GERÇEKLEMELERİ – STATE

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.state.example;
import java.io.*;
public class DirtyState extends State {
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
        PIMApplication.setCurrentState(PIMApplication.CLEAN);
    }
}
```

223

ÖRNEK KALIP GERÇEKLEMELERİ – STATE

ÇÖZÜM:

- Kaynak kodlar (devam)

```
package dp.state.example;
public class PIMApplication {
    public static final State CLEAN = new CleanState();
    public static final State DIRTY = new DirtyState();
    private static State currentState = CLEAN;

    public static State getCurrentState(){ return currentState; }
    public static void setCurrentState(State state){
        currentState = state;
    }
    public void editARecord( ) {
        currentState.edit();
    }
    public void saveRecords( File f, Serializable s ) {
        currentState.save(f, s);
    }
}
```

224

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

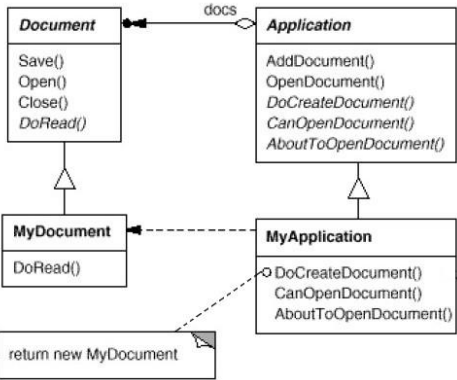
TEMPLATE METHOD:

- Amaç: Bir algoritmanın bazı adımlarını belirleyip bazılarının ayrıntılarının belirlenmesini ertelemek.
- Örnek:
 - Farklı belge türleri ile çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program hazırlanıyor.
 - Kullanıcı çerçevenin sunduğu Belge ve Uygulama sınıflarından kalıtım ile yeni sınıflar türeterek, kendi yazılımını hazırlıyor.
- Sorun:
 - Bir belge açar veya oluştururken her türlü ince ayrıntıyı kullanıcıya atmak istemiyoruz; bazı temel adımların uygun sıra ile yürütülmesini çerçeve programa bırakmak istiyoruz.

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

TEMPLATE METHOD:

- Örnek çözüm:



```
void Application::OpenDocument (const
char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

TEMPLATE METHOD:

- Kalıp yapısı:

AbstractClass

TemplateMethod()
PrimitiveOperation1()
PrimitiveOperation2()

ConcreteClass

PrimitiveOperation1()
PrimitiveOperation2()

...

PrimitiveOperation1()
...
PrimitiveOperation2()
...

- Kalıp bileşenleri:
 - ConcreteClass: Algoritmanın alt düzey ayrıntılarını gerçekler.
 - AbstractClass: TemplateMethod() içerisinde algoritmanın üst düzey veya değişmeyecek ayrıntılarını gerçekler, alt düzey ayrıntıları ConcreteClass alt sınıflarına aktarır.
- Gerçekleme ayrıntıları:
 - Alt düzey metotlar alt sınıflarda mutlaka tanımlanmalıdır (abstract (Java) veya 'pure' virtual (C++) olmalıdır).
 - Üst düzey metot (TemplateMethod) alt sınıflar tarafından yeniden tanımlanamamalıdır (final olmalıdır).

227

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

TEMPLATE METHOD:

- Kalıbın uygulanabileceği anlar:
 - Bir algoritmanın değişmez kısımlarını bir kere kodlayıp değişebilecek kısımlarını alt sınıflar üzerinden gerçeklemek istenildiğinde
 - Alt sınıfların davranışlarının denetlenmesi istenildiğinde
 - Tutarsızlığı veya kod tekrarını önlemek üzere kardeş sınıfların ortak davranışlarını bir noktada (üst sınıfta) toplamak istenildiğinde

228

ÖRNEK KALIP GERÇEKLEMELERİ – TEMPLATE METHOD

ÇÖZÜLECEK PROBLEM:

- Bir programda veritabanından kayıtlar çekilecek.
- Bu işlem SQL ile bir SELECT komutu hazırlanarak verilebilir.
- Programın farklı VTYS'ler ile çalışabilmesi gerekmektedir.
- Farklı VTYS'lerde VT ile bağlantı kurma ve SQL komutlarını biçimlendirme farklı olabilir.
 - Özellikle saklı yordam (SP) dilleri VTYS'ler arasında çok farklılık gösterir.
- Tüm farklılıklara rağmen, kayıt çekme işleminin değişmeyen temel adımları vardır:
 1. Bir bağlantı komutu biçimlendirilir (CONNECT).
 2. VT'na bağlantı komutu gönderilir.
 3. Bir kayıt seçme komutu biçimlendirilir (SELECT).
 4. VT'na seçme komutu gönderilir.
 5. Seçilen veri kümesi geri döner.
- Ortak temel adımların olması, strateji yerine Template Method kalıbının kullanımını daha doğru bir seçim kılar.

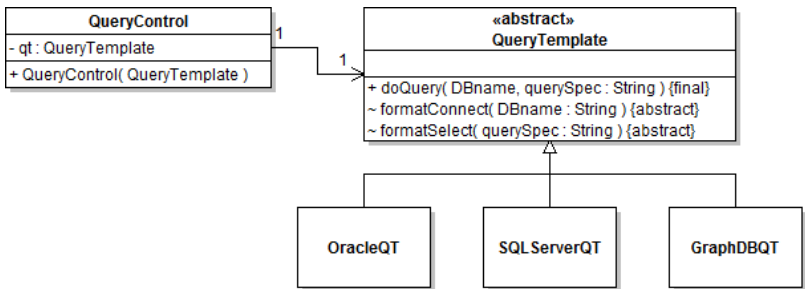
• Kaynak: DP Explained

229

ÖRNEK KALIP GERÇEKLEMELERİ – TEMPLATE METHOD

ÖNERİLEN ÇÖZÜM:

- Sınıf şeması:
 - Komut biçimlendirme metotları iç çalışma ile ilgili görülerek protected tanımlanmıştır.
 - Bir sınıf kütüphanesi veya çerçeve program hazırlanmadığına göre, Java için paket görünürlüğü kullanılabilir.



230

ÖRNEK KALIP GERÇEKLEMELERİ – TEMPLATE METHOD

ÖNERİLEN ÇÖZÜM:

```
package templateMethod.example;
public abstract class QueryTemplate {
    public final void doQuery( String DBname,
        String querySpec ) {
        String dbCommand;
        dbCommand = formatConnect( DBname );
        //dbCommand komutunu yürüterek bağlantıyı kur.
        dbCommand = formatSelect( querySpec );
        //dbCommand komutunu yürüterek kayıtları oku.
        //Oluşan veri kümesini geri döndür.
    }
    abstract String formatConnect( String DBname );
    abstract String formatSelect( String querySpec );
}
```

231

ÖRNEK KALIP GERÇEKLEMELERİ – TEMPLATE METHOD

ÖNERİLEN ÇÖZÜM:

```
package templateMethod.example;
public class OracleQT extends QueryTemplate {
    String formatConnect(String DBname) {
        //Oracle'a göre özel biçimlendirme komutlarını ver.
        return DBname;
    }
    String formatSelect(String querySpec) {
        //Oracle'a göre özel biçimlendirme komutlarını ver.
        return querySpec;
    }
}
public class SQLServerQT extends QueryTemplate {
    //Komutları bu kez SQL Server'a göre biçimlendir.
    String formatConnect(String DBname) { return DBname; }
    String formatSelect(String querySpec) { return querySpec; }
}
```

232

ÖRNEK KALIP GERÇEKLEMELERİ – TEMPLATE METHOD

ÖNERİLEN ÇÖZÜM:

```
package templateMethod.example;
public class QueryControl {
    private QueryTemplate qt;
    public QueryControl(QueryTemplate qt) {
        this.qt = qt;
    }
    public void aMethodThatNeedsSelect( ) {
        qt.doQuery("myDB", "*");
    }
}
```

233

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

VISITOR:

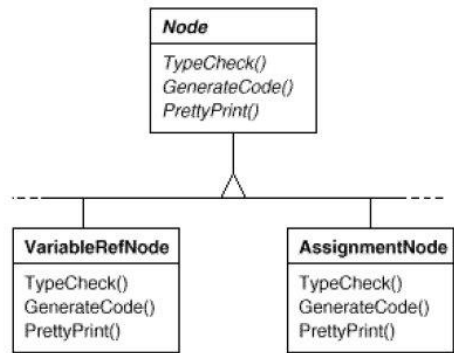
- Amaç:
 - Herhangi bir nesne yapısını oluşturan elemanlar üzerinde yapılacak bazı işlemler tanımlamak.
- Sorun:
 - Nesne yapısı her türden bağıntılar içeren sınıfların örnekleri arasında kurulmuş olabilir.
 - Üzerinde işlem yapılan elemanların sınıf yapılarını değiştirmek istemiyoruz.
- Eleştiri:
 - Madem daha yapılacak işlemler var, eylemler ve sorumluluklar tam olarak belirlenmemiş demektir. Dolayısıyla mevcut sınıf yapılarını değiştirerek farklı bir tasarım yapmak daha doğru olabilir.
- Örnek:
 - Yazılan bir derleyici, kaynak kodu farklı türlerden token'lara ayırıyor.
 - Bu parçalar (Node) farklı tiplerden oluşmaktadır.
 - Derleyici bütün parçalar üzerinde çeşitli işlemler yapacaktır.
 - Farklı tip parçalarda işlemler farklı yürütülecektir.

234

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

VISITOR:

- Örnek parça yapısı:

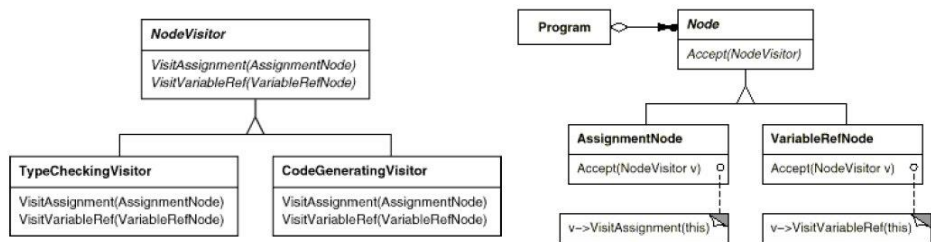


- Bu işlemlerin çeşitli parça yapılarına dağıtılması, bir bakım kabusu oluşturur.
- Her tür düğümde her tür işlemin yapılmasının da anlamı yoktur.

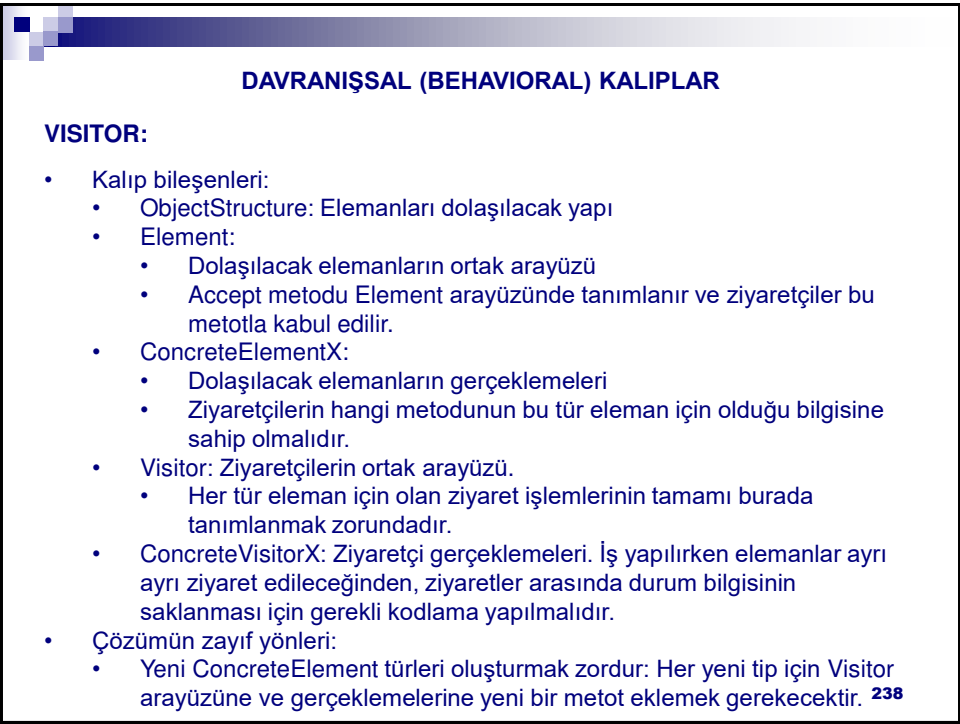
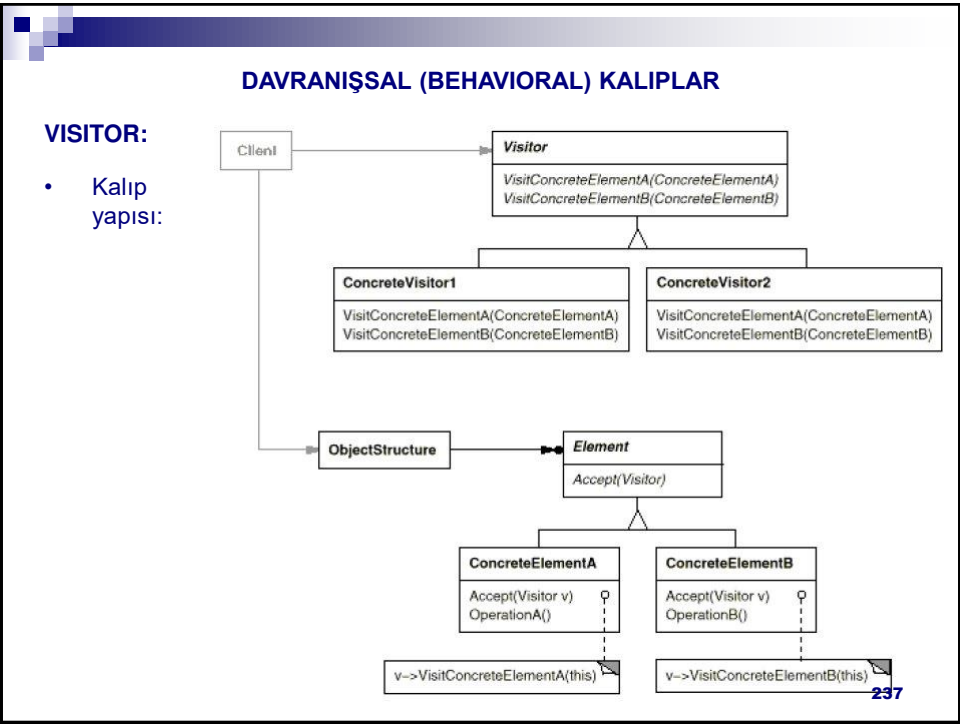
DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

VISITOR:

- Örnek Çözüm: Derleyici parçaları tek tek ziyaret ederek her parça üzerinde sadece gerekli işlemleri yapar.



- Düşüm gerçeklemleri kendi üzerlerindeki gerekli işlemleri çağırarak ziyaretçi gerçeklemlerini kabul eder (Accept metodu ile).
- Bu şekilde parçalarda sadece gerekli işlemler yürütülür.
- Çözümün zayıf yönü: Düşümler ziyaretçinin hangi metodunun bu düşüm türü için olduğunu bilmelidir.



DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

VISITOR:

- Kalıp etkileşimleri:

```
sequenceDiagram
    participant OS as anObjectStructure
    participant CEA as aConcreteElementA
    participant CEB as aConcreteElementB
    participant CV as aConcreteVisitor

    OS->>CEA: Accept(aVisitor)
    activate CEA
    CEA->>CV: VisitConcreteElementA(aConcreteElementA)
    activate CV
    CV->>CEA: OperationA()
    deactivate CV
    CEA->>OS: 
    deactivate CEA
    OS->>CEB: Accept(aVisitor)
    activate CEB
    CEB->>CV: VisitConcreteElementB(aConcreteElementB)
    activate CV
    CV->>CEB: OperationB()
    deactivate CV
    CEB->>OS: 
    deactivate CEB
```

- Kalıbın kullanılabileceği anlar:
 - Değişen arayüzlere sahip nesnelerden oluşan bir yapıda, nesneler üzerinde farklı işlemler yapılması gerektiği anlarda VE
 - Bu nesne yapısındaki türlerin değişmediği/ender değiştiği koşullarda. **239**

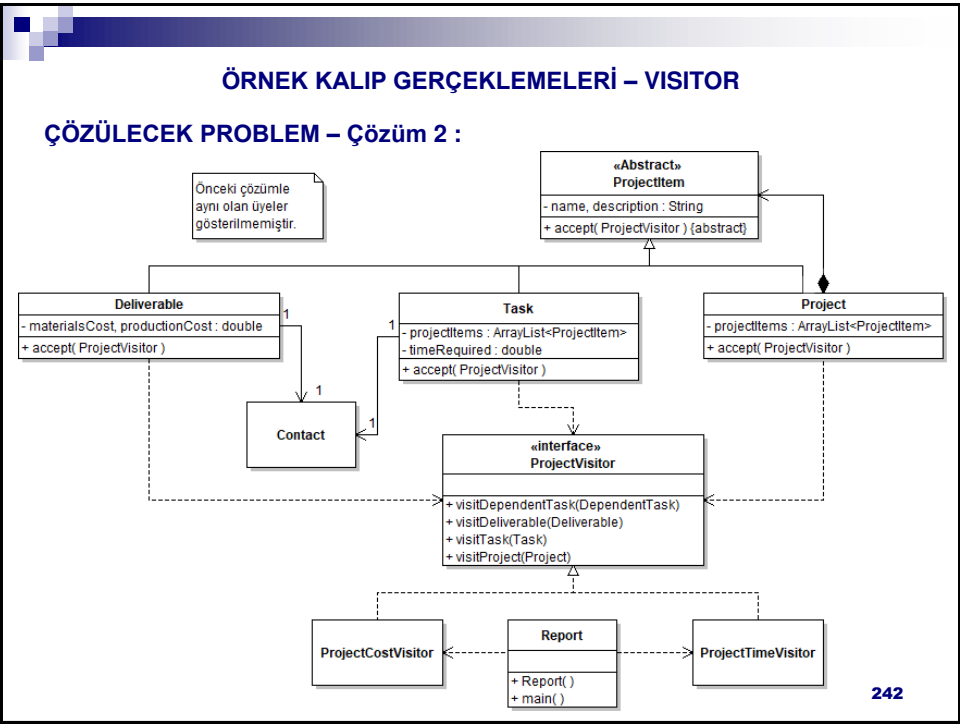
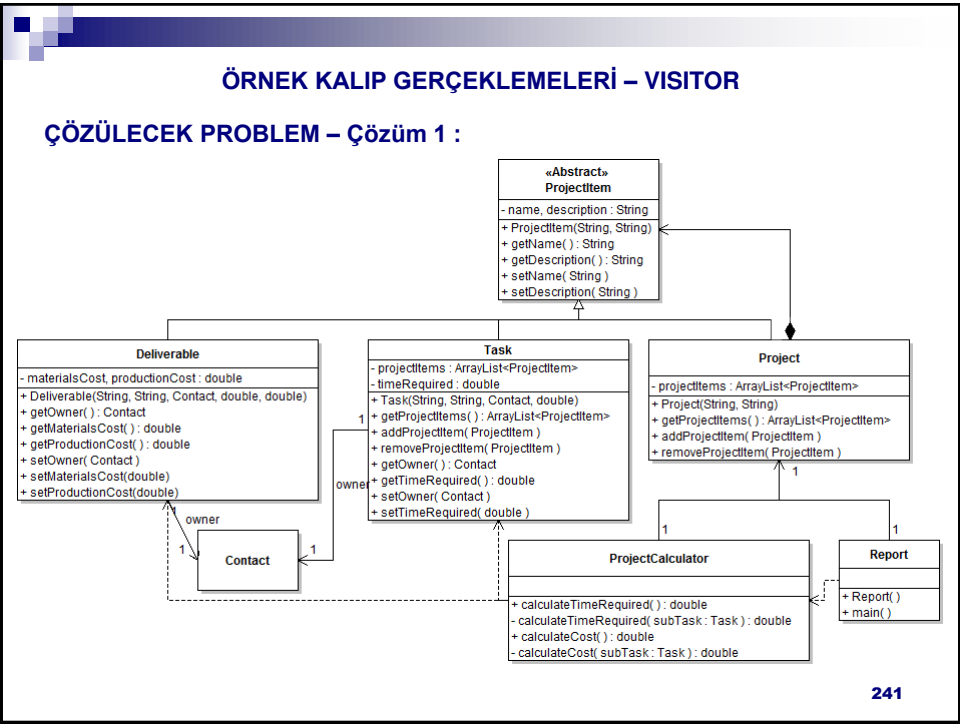
ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 1 :

- Bir proje yönetim yazılımı hazırlıyoruz.
- Aşağıdaki temel varlıkları modelimize alıyoruz:
 - Project: Proje hiyerarşisinin kökü
 - Task: Projede tanımlı bir görev
 - Deliverable: Projenin ara çıktıları veya sonucu olarak üretilebilecek herhangi bir kod, belge, ürün, vb.

- Kaynak: Applied Java Patterns

240



ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 1 :

- Kaynak kodlar (uygulama alanı):

```
package dp.visitor.solution1;
public abstract class ProjectItem{
    private String name;
    private String description;

    public ProjectItem(String newName, String newDescription){
        name = newName;
        description = newDescription;
    }
    public String getName(){ return name; }
    public String getDescription(){ return description; }
    public void setName(String newName){ name = newName; }
    public void setDescription(String newDescription){ description = newDescription; }
}
```

243

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 1 :

- Kaynak kodlar (uygulama alanı):

```
package dp.visitor.solution1;
import java.util.ArrayList;
public class Project extends ProjectItem{
    private ArrayList<ProjectItem> projectItems = new ArrayList<>();

    public Project(String newName, String newDescription){
        super(newName, newDescription);
    }

    public ArrayList<ProjectItem> getProjectItems(){ return projectItems; }

    public void addProjectItem(ProjectItem element){
        if (!projectItems.contains(element)){
            projectItems.add(element);
        }
    }
    public void removeProjectItem(ProjectItem element){
        projectItems.remove(element);
    }
}
```

244

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

```
package dp.visitor.solution1;
import java.util.ArrayList;
public class Task extends ProjectItem{
    private ArrayList<ProjectItem> projectItems = new ArrayList<>();
    private Contact owner;
    private double timeRequired;

    public Task(String newName, String newDesc, Contact newOwner, double newTimeRequired){
        super(newName, newDesc);
        owner = newOwner; timeRequired = newTimeRequired;
    }
    public ArrayList<ProjectItem> getProjectItems(){ return projectItems; }
    public Contact getOwner(){ return owner; }
    public double getTimeRequired(){ return timeRequired; }
    public void setOwner(Contact newOwner){ owner = newOwner; }
    public void setTimeRequired(double newTimeRequired){ timeRequired = newTimeRequired; }

    public void addProjectItem(ProjectItem element){
        if (!projectItems.contains(element)){
            projectItems.add(element);
        }
    }
    public void removeProjectItem(ProjectItem element){
        projectItems.remove(element);
    }
}
```

245

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

```
package dp.visitor.solution1;
public class Deliverable extends ProjectItem{
    private Contact owner;
    private double materialsCost;
    private double productionCost;

    public Deliverable(String newName, String newDescription,
        Contact newOwner, double newMaterialsCost, double newProductionCost){
        super(newName, newDescription);
        owner = newOwner;
        materialsCost = newMaterialsCost;
        productionCost = newProductionCost;
    }

    public Contact getOwner(){ return owner; }
    public double getMaterialsCost(){ return materialsCost; }
    public double getProductionCost(){ return productionCost; }

    public void setMaterialsCost(double newCost){ materialsCost = newCost; }
    public void setProductionCost(double newCost){ productionCost = newCost; }
    public void setOwner(Contact newOwner){ owner = newOwner; }
}
```

246

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

Raporlama – Çözüm 1:

- Kaynak kodlar:

```
package dp.visitor.solution1;
public class Report {
    private Project project;
    public Report() {
        project = new Project("Yüksek Lisans", "Lisansüstü eğitim (Yüksek lisans)");
        Task gorev; Deliverable urun;
        gorev = new Task("2014-1 dersleri", "2014-2015 Güz yarıyılı (4 ders)", null, 15);
        urun = new Deliverable("BLM 5128", "Yazılım Kalitesi", null, 45, 30);
        gorev.addProjectItem(urun);
        //3 ders daha ekle
        project.addProjectItem(gorev);
    }
    public static void main(String[] args) {
        Report rep = new Report();
        ProjectCalculator calc = new ProjectCalculator(rep.project);
        System.out.println("Proje adı ve açıklaması: " + rep.project.getName() + ": " +
            rep.project.getDescription());
        System.out.println("Proje süresi: " + calc.calculateTimeRequired() + " hafta.");
        System.out.println("Proje maliyeti: " + calc.calculateCost() + " saat çalışma.");
    }
}
```

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

Raporlama – Çözüm 1:

- Kaynak kodlar (devam):

```
package dp.visitor.solution1;
public class ProjectCalculator {
    private Project project;
    public ProjectCalculator(Project project) { this.project = project; }
    public double calculateTimeRequired( ) {
        double time = 0.0;
        for( ProjectItem item : project.getProjectItems() )
            if( item instanceof Task )
                time += calculateTimeRequired((Task)item);
        return time;
    }
    private double calculateTimeRequired( Task subTask ) {
        double time = subTask.getTimeRequired();
        for( ProjectItem item : subTask.getProjectItems() )
            if( item instanceof Task )
                time += calculateTimeRequired((Task)item);
        return time;
    }
}
```

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

Raporlama – Çözüm 1:

- Kaynak kodlar (devam):

```
public double calculateCost( ) {
    double cost = 0.0;
    for( ProjectItem item : project.getProjectItems() ) {
        if( item instanceof Deliverable )
            cost += ((Deliverable)item).getMaterialsCost()+
                ((Deliverable)item).getProductionCost();
        if( item instanceof Task )
            cost += calculateCost((Task)item);
    }
    return cost;
}

private double calculateCost( Task subTask ) {
    double cost = 0.0;
    for( ProjectItem item : subTask.getProjectItems() ) {
        if( item instanceof Deliverable )
            cost += ((Deliverable)item).getMaterialsCost()+
                ((Deliverable)item).getProductionCost();
        if( item instanceof Task )
            cost += calculateCost((Task)item);
    }
    return cost;
}
}
```

249

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 2 :

- Kaynak kodlar (sadece değişen kısımlar):

```
package dp.visitor.solution2;
public abstract class ProjectItem{
    abstract public void accept(ProjectVisitor v);
}

package dp.visitor.solution2;
public class Project extends ProjectItem{

    public void accept(ProjectVisitor v){ v.visitProject(this); }
}

package dp.visitor.solution2;
public class Task extends ProjectItem{

    public void accept(ProjectVisitor v){ v.visitTask(this); }
}

package dp.visitor.solution2;
public class Deliverable extends ProjectItem{

    public void accept(ProjectVisitor v){ v.visitDeliverable(this); }
}
```

250

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 2 :

- Kaynak kodlar :

```
package dp.visitor.solution2;
public interface ProjectVisitor{
    public void visitDependentTask(DependentTask p);
    public void visitDeliverable(Deliverable p);
    public void visitTask(Task p);
    public void visitProject(Project p);
}
```

251

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 2 :

- Kaynak kodlar :

```
package dp.visitor.solution2;
public class ProjectCostVisitor implements ProjectVisitor {
    private double cost = 0.0;
    public ProjectCostVisitor(Project project) {
        project.accept(this);
    }
    public void visitDependentTask(DependentTask p) { visitTask(p); }
    public void visitDeliverable(Deliverable p) {
        cost += p.getMaterialsCost()+p.getProductionCost();
    }
    public void visitTask(Task p) {
        for( ProjectItem item : p.getProjectItems() )
            item.accept(this);
    }
    public void visitProject(Project p) {
        for( ProjectItem item : p.getProjectItems() )
            item.accept(this);
    }
    public double getCost() {
        return cost;
    }
}
```

252

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

ÇÖZÜLECEK PROBLEM – Çözüm 2 :

- Kaynak kodlar :

```
package dp.visitor.solution2;
public class ProjectTimeVisitor implements ProjectVisitor {
    private double time = 0.0;
    public ProjectTimeVisitor(Project project) {
        project.accept(this);
    }
    public void visitDependentTask(DependentTask p) {visitTask(p);}
    public void visitDeliverable(Deliverable p) { }
    public void visitTask(Task p) {
        time += p.getTimeRequired();
        for( ProjectItem item : p.getProjectItems() )
            item.accept(this);
    }
    public void visitProject(Project p) {
        for( ProjectItem item : p.getProjectItems() )
            item.accept(this);
    }
    public double getTime() {
        return time;
    }
}
```

253

ÖRNEK KALIP GERÇEKLEMELERİ – VISITOR

Raporlama – Çözüm 2:

- Kaynak kodlar:

```
package dp.visitor.solution2;
public class Report {
    private Project project;
    public Report() {
        //önceki ile aynı kurucu
    }
    public static void main(String[] args) {
        Report rep = new Report();
        ProjectCostVisitor vc = new ProjectCostVisitor(rep.project);
        System.out.println("Proje adı ve açıklaması: " + rep.project.getName() + ": " +
            rep.project.getDescription());
        System.out.println("Proje maliyeti: " + vc.getCost() + " saat çalışma.");
        ProjectTimeVisitor vt = new ProjectTimeVisitor(rep.project);
        System.out.println("Proje süresi: " + vt.getTime() + " hafta.");
    }
}
```

254

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEDIATOR:

Amaç:

Bir nesneler kümesinin aralarındaki etkileşimlerin sarmalanması.

Böylece nesneler birbirleri ile doğrudan değil de dolaylı olarak ilişki kurarlar.

Sonuç olarak söz konusu nesnelerin arasındaki ilişkiler bu nesneleri değiştirmeden de değiştirilebilir.

Örnek:

Çeşitli GUI bileşenleri içeren bir font seçme penceresi hazırlanıyor.

GUI bileşenleri kullanılarak font ile ilgili yapılan her değişiklik, anında örnek metin alanına yansıtılacaktır.

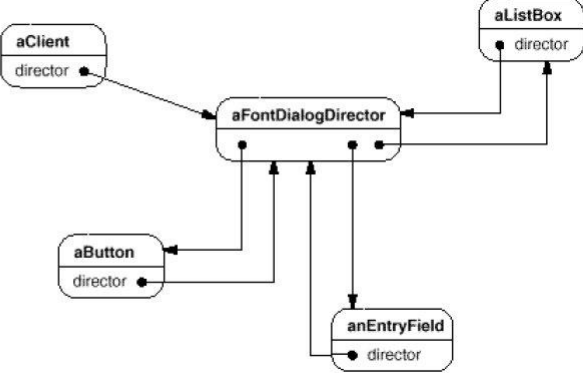
255

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEDIATOR:

Örnek:



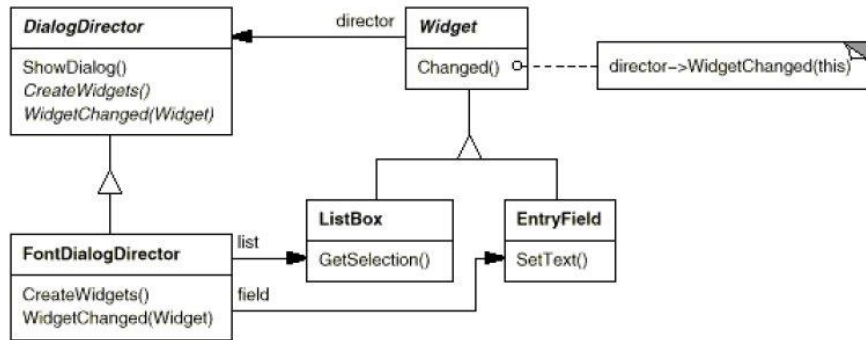


256

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

MEDIATOR:

- Örnek çözüm:
 - GUI nesnelere ortak bir arayüz hazırlayarak, aralarındaki etkileşimleri genel olarak tanımlayacak bir başka soyut sınıf tasarlanır.
 - Bu soyut sınıftan gerçeklemeler oluşturmak için Template Method kalıbı kullanılır.

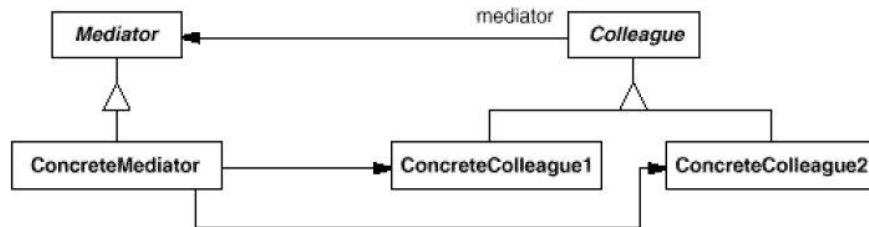


257

DAVRANIŞSAL (BEHAVIORAL) KALİPLAR

MEDIATOR:

- Kalıp yapısı:
 - Colleague gerçekleştirmeleri kendi aralarında doğrudan değil, bir Mediator gerçekleştirmesi üzerinden yaparlar.



- Kalıp bileşenleri:
 - Mediator: Colleague nesneleri ile etkileşim kurabilecek bir arayüz sağlar.
 - ConcreteMediator: Colleague nesneleri arasında eşgüdüm sağlar ve bunların idamesinden sorumludur (sahiplik ilişkisi).
 - Colleague/ConcreteColleague: Etkileşimde bulunacak nesnelerin arayüzü ve gerçeklemeleri.

258

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEDIATOR:

- Nesneler arasındaki etkileşimler:

```
classDiagram
    class aColleague {
        mediator
    }
    class aConcreteMediator {
    }
    aColleague --> aConcreteMediator
    aConcreteMediator --> aColleague
```

259

DAVRANIŞSAL (BEHAVIORAL) KALIPLAR

MEDIATOR:

- Kalıbın zayıf yönleri:
 - Mediator gerçeklemelerinin yapısı aşırı karmaşıklıdır.
 - Bu nedenle Mediator nesnelerinin yeniden kullanımı da zorlaşır.
- Kalıbın kullanılabileceği anlar:
 - Bir grup nesnenin arasında iyi tanımlanmış ancak karmaşık etkileşimlerin bulunduğu anlarda,
 - Birden fazla nesne arasına dağılmış olan bir davranışı tek bir sınıfta toplamak istendiği, ancak bu davranışın fazla değişim göstermeyeceği durumlarda.

ÖRNEK KALIP GERÇEKLEMELERİ – MEDIATOR

ÇÖZÜLECEK PROBLEM:

- Bu kalıp için örnek yapılmayacaktır.

260

130



NESNEYE DAYALI TASARIM VE MODELLEME

KISIM 2: REFACTORING

261



REFACTORING (YENİDEN YAPILANDIRMA)

REFACTORING NEDİR?

- Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmeye refactoring adı verilmiştir.
- Yazılımın dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesi anlamındadır.

REFACTORING İLE AMAÇLANAN NEDİR?

- Mevcut kod üzerinde, kodun iç kalite özelliklerini daha iyiye götürecek şekilde değişiklikler yapılması amaçlanır.
 - Kodun anlaşılabilirliğinin artırılması
 - Koda esneklik kazandırılması
 - Yeniden kullanılabilirliğin artırılması
 - Yüksek uyum ve düşük bağımlılığın sağlanması
 - vb.

262

REFACTORING (YENİDEN YAPILANDIRMA)

REFACTORING NASIL YAPILIR?

- Çevik yaklaşımla yapılır:
 - Mevcut gereksinimleri karşılayacak ilk tasarım yapılır.
 - İlk tasarımın mükemmel olması ve her bilinmezi öngörmeye çalışması gerekmez.
 - Yeterince iyi olan tasarım gerçeklenmeye başlanır.
 - Önceki tasarım adımlarından birinde yanlış karar vermiş olduğunuzu anlayınca refactoring yapılır.
- Sürekli sınama ile yapılır:
 - Yapılan değişikliklerin yeni hatalar oluşturmadığından emin olmak için, her değişikliğin ardından kodun yeni hali sınanmalıdır.

NE ZAMAN REFACTORING YAPILIR?

- Kodunuz derlenip toparlanmaya gerek duyduğu zaman yapılır.
 - Kodunuza yeni özellikler eklemekte zorlanmaya başladığınızda
 - Bir hatayı bulmak için zorlanmaya başladığınız, bulduğunuzda da düzeltmek için bir çok ayrı yeri kurcalamak zorunda kaldığınızda
 - vb.
- Kodunuz kötü kokmaya başladığında yapılır!

263

CODE SMELLS (KÖTÜ KOKAN / KUSURLU KOD)

CODE SMELL NEDİR?

- Yazılan kodu iyileştirme gerektiğinin işaretleridir.
- Kod içerisinde karşılaşılan tersliklerdir.
- Kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir.
- Zamanla bir yazılım yamalı bohçaya dönüşebilir.
 - Değişik zamanlarda farklı kişilerce kodun değişik kısımlarında değişiklikler yapılır.
 - Bu değişiklikler birlikte düşünülerek planlanabilseydi, daha iyi bir çözüme ulaşılması mümkün olabilirdi.
- Yazılım elimizden kayıp gitmeden, tehlike işaretlerini sezip önlem almak yerinde olacaktır.

CODE SMELL VE REFACTORING

- Kodda karşılaşılan terslikleri düzeltmek üzere yeniden yapılandırma eylemleri yürütülür.
- Bir yapılandırma eylemi yeni kokular çıkartabilir, bu durumda yeni yapılandırma eylemleri yürütülür.

264

BİRİM SINAMALARI (Unit Testing)

- En küçük yazılım bileşeninin sınanmasıdır.
- NYP'de bireysel sınıfların sınanmasıdır.
- Ne zaman tasarlanır?
 - Kodlamadan önce (TDD), kodlama sırasında veya kodlamanın ardından.
 - Bir sınıfın tek başına yürütemediği sorumlulukların sınanması için, bu sınıfın ihtiyaç duyduğu diğer sınıfların yerine geçecek kod gerekebilir.
 - Vekil, sahte, yalancı kod/sınıf, vb. (Stub, dummy, surrogate, proxy)
 - Vekil, sadece ihtiyaç duyulan sınıflar gerçekleştirilene dek kullanılır.

JUnit HAKKINDA

- Java ile yazılmış kodun birim sınamaları için bir çerçeve programdır (framework)
- Diğer diller için de sürümleri bulunmaktadır.
 - Ör. C# için csUnit
- IDE desteği:
 - Eclipse: IDE ile hazır geliyor (build path → add libraries)
 - NetBeans: IDE ile hazır geliyor

265

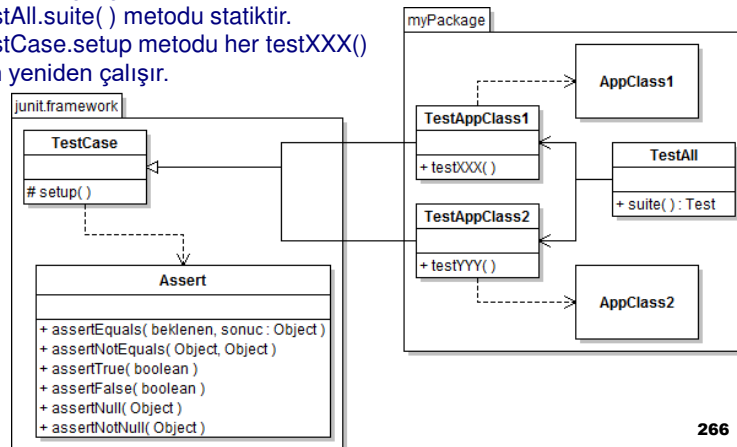
JUNIT İLE BİRİM SINAMALARI

jUnit Sürüm 3.X ile Test Case Hazırlamak

- Her sınıfın birim testi için ayrı sınıfta ayrı test case'ler hazırlamak, ardından test case'leri bir test suite altında toplamak tercih edilmiştir.
 - TestAll sınıfını yazmak zorunlu değildir, TestAppClass sınıfları bireysel olarak da çalıştırılabilir.
 - TestAll.suite() metodu statiktir.
 - TestCase.setup metodu her testXXX() için yeniden çalışır.
- 
- ```

classDiagram
 package myPackage {
 AppClass1
 }

```



266

## JUNIT İLE BİRİM SINAMALARI

### jUnit Sürüm 4.X Gelişmeleri

- Geriye doğru uyumluluk korunmakla birlikte, jUnit 4 sürümü ile annotation desteği gelmiştir (Büyük/küçük harf duyarlılığı vardır).
- Artık sına sınıflarının TestCase sınıfından kalıtımla türetilmesi gerekmemektedir ancak şu eklemeler yapılmalıdır:  

```
import static org.junit.Assert.*;
import org.junit.*;
```
- Artık sına metotlarının adlarını test kelimesi ile başlatmak gerekli değildir, ilgili metotların başına @Test annotation koymak yeterlidir.
- setup adlı metodun yerine ise @Before annotation gelmiştir.  

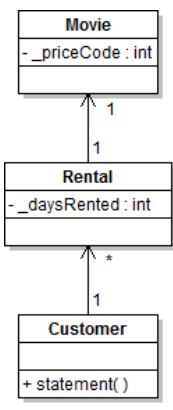
```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }
```
- Exception tanımlanan yazılımlarda atılması gereken exception'ların gerçekten ortaya çıkıp çıkmadığının sınılanması da mümkün olmuştur.  

```
@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
 doSomethingThatCreatesTheException();
}
```

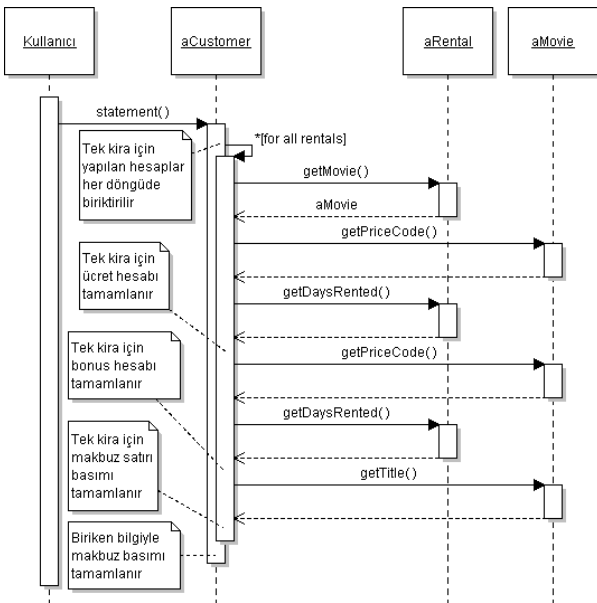
267

## JUNIT İLE BİRİM SINAMALARI

- Basit bir video kiralama örneği:



- Örneğin kaynak kodu: refactoring/fowler00



### JUNIT İLE BİRİM SINAMALARI

- Test cases:
  - kaynak kodu: testing/fowler00

```
package fowler_00;
import junit.framework.TestCase;
public class TestCustomer extends TestCase {
 private Customer yunus;
 private Movie matrix, monster, surrogate, terminator;
 protected void setUp() {
 yunus = new Customer("Yunus Emre Selçuk");
 matrix = new Movie("The Matrix",Movie.REGULAR);
 monster = new Movie("Monsters, Inc.",Movie.CHILDRENS);
 surrogate = new Movie("Surrogates", Movie.NEW_RELEASE);
 terminator = new Movie("Terminator Salvation",Movie.NEW_RELEASE);
 }
 public void testGetName() {
 String sonuc = yunus.getName();
 assertEquals("Yunus Emre Selçuk",sonuc);
 }
 public void testStatementWhenEmpty() {
 String sonuc = yunus.statement();
 String beklenen = "Rental Record for Yunus Emre Selçuk\n";
 beklenen += "Amount owed is 0.0\n";
 beklenen += "You earned 0 frequent renter points";
 assertEquals(beklenen, sonuc);
 }
}
```

269

### JUNIT İLE BİRİM SINAMALARI

- Test cases (devam):

```
public void testStatementWithMoviesLongRent() {
 yunus.addRental(new Rental(matrix, 3));
 yunus.addRental(new Rental(monster, 4));
 yunus.addRental(new Rental(surrogate, 2));
 String sonuc = yunus.statement();
 String beklenen = "Rental Record for Yunus Emre Selçuk\n";
 beklenen += "\tThe Matrix\t3.5\n";
 beklenen += "\tMonsters, Inc.\t3.0\n";
 beklenen += "\tSurrogates\t6.0\n";
 beklenen += "Amount owed is 12.5\n";
 beklenen += "You earned 4 frequent renter points";
 assertEquals(beklenen, sonuc);
}
public void testStatementWithMoviesShortRent() {
 yunus.addRental(new Rental(matrix, 2));
 yunus.addRental(new Rental(monster, 3));
 yunus.addRental(new Rental(surrogate, 1));
 String sonuc = yunus.statement();
 String beklenen = "Rental Record for Yunus Emre Selçuk\n";
 beklenen += "\tThe Matrix\t2.0\n";
 beklenen += "\tMonsters, Inc.\t1.5\n";
 beklenen += "\tSurrogates\t3.0\n";
 beklenen += "Amount owed is 6.5\n";
 beklenen += "You earned 3 frequent renter points";
 assertEquals(beklenen, sonuc);
}
```

270





## CODE SMELLS – DUPLICATED CODE

### YİNELENEN KOD

- Aynı kodun veya çok benzer kod yapısının birden fazla yerde tekrarlanması durumudur.
- Bu kodun işlevselliğinde değişiklik yapılması gerektiğinde, kodun tekrarlandığı her yerde aynı işlemlerin tekrarlanması gerekecektir.
- Ortaya çıkan sonuç bir bakım kabusudur.
- Çözümüne götürecek düzenlemeler (refactorings):
  - Extract Method: Yinelenen kodu bir metod altında toplamak amacı ile kullanılır.
  - Yinelenen kodu toplamaya yönelik diğer düzenlemelere ileride değinilecektir.

### ÜYE ALAN ADLANDIRMA

- Kod örneklerinde alt çizgi ile başlayan değişkenler, nesnenin üye alanlarıdır.
- IDE'lerde ise üye alanlar farklı renk ile vurgulanır.

273

## REFACTORING – EXTRACT METHOD

### METOT ÇIKARTMA

- Bir metod içindeki kodun bir kısmını yeni bir metod olarak belirlemeye dayanır.
- Bu yapılandırma bir çok kod kusurunu (smell) düzeltir.
- Oluşturulan yeni metoda, amacını açıkça belirten, isabetli bir ad verilmelidir.
  - Böylece alt düzey ayrıntılarla uğraşmadan, kodun anlaşılabilirliği artar.
- Örnek:

```
/** ÖNCE: */
void printOwing(double
 amount) {
 printBanner();
 //print details
 System.out.println
 ("name:" + _name);
 System.out.println
 ("amount" + amount);
}
```

```
/** SONRA: */
void printOwing(double
 amount) {
 printBanner();
 printDetails(amount);
}
void printDetails (double
 amount) {
 System.out.println
 ("name:" + _name);
 System.out.println
 ("amount" + amount);
}
```

274

## CODE SMELLS – LONG METHOD

### AŞIRI UZUN KOD

- Bir metodun uzunluğu arttıkça:
  - Anlaşılması zorlaşır,
  - Değişikliklerden etkilenme olasılığı artar,
  - Birden fazla ilgi alanını kapsama olasılığı artar (uyumun düşmesi).
- Çözüme götürecek düzenlemeler (refactorings):
  - Extract Method: Uyumlu (belli bir ortak amaca yönelik) komutlar seçilerek bir metod altında toplanır.
  - Aşırı uzun kodu kısaltmaya yönelik diğer düzenlemelere ileride değinilecektir.

275

## REFACTORING – EXTRACT METHOD

### METOT ÇIKARTMA

```
/** ÖNCE: */
void printOwing() {
 Enumeration e = _orders.elements();
 double outstanding = 0.0;

 // print banner
 System.out.println("*****");
 System.out.println("** Customer Owes **");
 System.out.println("*****");

 // calculate outstanding
 while (e.hasMoreElements()) {
 Order each = (Order) e.nextElement();
 outstanding += each.getAmount();
 }

 //print details
 System.out.println ("name:" + _name);
 System.out.println ("amount" +
 outstanding);
}
```

```
/** SONRA: */
void printOwing() {
 printBanner();
 printDetails(getOutstanding());
}

void printBanner() {
 System.out.println("*****");
 System.out.println("** Customer Owes **");
 System.out.println("*****");
}

double getOutstanding() {
 double outstanding = 0.0;
 for (Order each : _orders)
 outstanding += each.getAmount();
 /* for-each ile döngü kısaldı! */
 return outstanding;
}

void printDetails (double outstanding) {
 System.out.println ("name:" + _name);
 System.out.println ("amount" +
 outstanding);
}
```

276

## REFACTORING

### TASARIM İLKELERİ

- Büyük sınıflar ve az sayıda nesne yerine daha çok sayıda küçük sınıflar ve nesneler kullanılması önerilir.
  - Böylece uygulama mantığının programın çeşitli yerlerinde paylaşımı kolaylaşır.
  - Sınıfların anlaşılması kolaylaşır.
  - Yeniden kullanım olasılığı artar.
  - Uyum yükselir.
- Uzun ve çok iş yapan metotlar yerine kısa ve adı ile kendi kendini açıklayan metotların kullanılması önerilir.
  - Öyle ki, kodda yorum satırı bulunmasına hiç gerek kalmasin.
  - Kısa sınıflar için verilen yararlar kısa metotlar için de geçerlidir.
- Büyük bir kodu daha küçük parçalara bölünce, bu kez de yönetilmesi gereken parça sayısı artacaktır.
  - Ancak bu zarar, elde edilen yararlar karşısında daha küçük kalmaktadır.
  - Güncel derleyici ve yorumlayıcıların çoğu, bağlam değiştirme (context switching) ile gelen yükü büyük oranda azaltmıştır.
  - Böylece küçük nesne ve metotların kullanımı yüzünden başarım olumsuz etkilenmez.

277

## CODE SMELLS – DUPLICATED CODE

### YİNELENEN KOD (devam)

- Çözümüne götürecek diğer düzenlemeler:
  - Pull Up Method: Yinelenen kod kalıtım ağacındaki kardeş veya kuzen sınıflara yayılmışsa, oluşturulan metot ortak üst sınıfa çekilir.
  - Form Template Method: Kod aynı değil ancak benzerse, GoF Template Method tasarım kalıbı kullanılarak kardeş/kuzen sınıflarda düzenleme yapılır.
  - Eğer yinelenen veya benzer kod ilişkisiz sınıflarda yer alıyorsa:
    - Sorumluluklar doğru dağıtılmamış olabilir.
    - Metot bir sınıfa atanır, diğerleri bu sınıfı kullanır.
    - Ya da bu metot, oluşturulacak yeni bir sınıfa atanır ve diğerleri bu sınıfı kullanır.
  - Özetle, Extract Method ile oluşturulan yeni metoda en uygun yer aranır.

278

### REFACTORING – PULL UP METHOD

#### METODU YUKARI ÇEKME

- Kardeş sınıflarda yinelenen metotlar varsa, bunlar üst sınıfa çekilerek bir tek yerde kodlanır.
- Örnek:

/\*\* ÖNCE: \*\*/

Employee

Salesman

Engineer

getName

getName

⇒

Employee

getName

Salesman

Engineer

279

### REFACTORING – PULL UP METHOD

#### METODU YUKARI ÇEKME

- Özel durumlar:

/\*\* ÖNCE: \*\*/

Customer

addBill (dat: Date, amount: double)

lastBillDate

Regular Customer

createBill (Date)

chargeFor (start: Date, end: Date)

Preferred Customer

createBill (Date)

chargeFor (start: Date, end: Date)

/\*\* SONRA: \*\*/

Customer

lastBillDate

addBill (dat: Date, amount: double)

createBill (Date)

chargeFor (start: Date, end: Date)

Regular Customer

chargeFor (start: Date, end: Date)

Preferred Customer

chargeFor (start: Date, end: Date)

- Özel durumlar:
  - Template Method tasarım kalıbı kullanılabilir.

280

140

## CODE SMELLS – LONG METHOD

### AŞIRI UZUN KOD

- Çözüme götürecek diğer düzenlemeler:
  - Decompose Conditional: Karar verme (if-switch-case) ve döngülerin yoğun olması çoğu kez kodu uzatır.
  - Replace Temp with Query: Geçici yerel değişkenlerin kullanımı genelde aynı geçici değişkenin kullanımı bahanesi ile kodun uzamasını teşvik eder.
  - Replace Method with Method Object: Üstteki düzenlemeler geçici değişkenleri ve uzun parametre listesini yok edememişse kullanılabilircek daha karmaşık bir düzenleme.

281

## REFACTORING – DECOMPOSE CONDITIONAL

### KARAR VERME KOMUTUNU PARÇALAMA

- Aşırı karmaşık if komutlarını sadeleştirmek, anlaşılabilirliği artırır:

```
/** ÖNCE: */
if (date.before (SUMMER_START) || date.after(SUMMER_END))
 charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
```

```
/** SONRA: */
if (notSummer(date))
 charge = winterCharge(quantity);
else charge = summerCharge (quantity);
private boolean notSummer(Date date) {
 return date.before (SUMMER_START) ||
 date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
 return quantity * _summerRate;
}
private double winterCharge(int quantity) {
 return quantity * _winterRate + _winterServiceCharge;
}
```

282

### CODE SMELLS – LONG PARAMETER LIST

#### UZUN PARAMETRE LİSTESİ

- Bir metodun parametre sayısının fazla olması, hatta bu parametrelerden bazılarının başka metotlarda da birlikte geçmesi durumudur.
  - Demek ki aslında bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını (1) veya tamamını (2) oluşturmaktadır.
- Çözüme götürecek düzenleme
  - Introduce Parameter Object: Metot parametrelerinin sayısı çok ise azaltmaya yönelik bir düzenlemedir.
  - Preserve Whole Object: Metot parametrelerinin sayısı çok ise azaltmaya yönelik diğer bir düzenlemedir.

283

### REFACTORING – INTRODUCE PARAMETER OBJECT

#### PARAMETRE NESNESİ OLUŞTURMA

- Bir metod çok fazla parametre alıyorsa, uygun parametreleri üye alan olarak yeni bir sınıfta toplayın ve o türden bir nesneyi parametre olarak kullanın.

/\*\* ÖNCE: \*\*\*/

Customer

amountInvoicedIn(start: Date, end: Date)  
amountReceivedIn(start: Date, end: Date)  
amountOverdueIn(start: Date, end: Date)

⇒

/\*\* SONRA: \*\*\*/

Customer

amountInvoicedIn(DateRange)  
amountReceivedIn(DateRange)  
amountOverdueIn(DateRange)

- Bunu sadece parametre listesini kısaltmak amacıyla yapmış olmazsınız:
  - Oluşturulan DateRange sınıfı sadece iki veriyi birleştiren bir kayıt yapısı şeklinde kalmaz,
  - isWithinRange(Date) gibi uygun metotlar da içererek, “decompose conditional” veya “extract method” düzenlemesi de yapılmış olur.

284

## REFACTORING – PRESERVE WHOLE OBJECT

### TÜM NESNEYİ GÖNDER

- Bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine, nesnenin kendisini gönderin.

```
/** ÖNCE: */
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);
```

```
/** SONRA: */
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

- Bu düzenlemenin 1. zayıf yönü: Bağımlılık artabilir
  - Parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir bağımlılık oluşur.
- Bu düzenlemenin 2. zayıf yönü: Geriye doğru uyumluluk kaybedilerek refactoring tanımı esnetilmiştir.
  - Çünkü söz konusu metodların imzası değişmiştir.
- Bu durumlardan biri bile çok sakıncalı olacaksa bu düzenlemeyi kullanmayın.

285

## REFACTORING – REPLACE TEMP WITH QUERY

### GEÇİCİ DEĞİŞKEN YERİNE ERİŞİM METODU KULLANIMI

- Bir ifadeyi bir geçici değişkende saklayarak kullanmak yerine, ifadeden metod(lar) çıkartın.

```
/** ÖNCE: */
double getPrice() {
 int basePrice = _quantity * _itemPrice;
 double discountFactor;
 if (basePrice > 1000) discountFactor = 0.95;
 else discountFactor = 0.98;
 return basePrice * discountFactor;
}
```

```
/** SONRA: */
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
 if (basePrice() > 1000) return 0.95;
 else return 0.98;
}
```

286

## REFACTORING – REPLACE TEMP WITH QUERY

### GEÇİCİ DEĞİŞKEN YERİNE ERİŞİM METODU KULLANIMI

- Avantaj: Kopyala-yapıştır kodlamayı daha ortaya çıkmadan azaltmak.
  - Geçici değişkeni kullanan metodunkine benzer bir işlem eklenmek istendiğinde, mevcut metottan yeni metoda kopyala-yapıştır yapılacaktır.
  - Bu kod parçacığı ile ilgili bir hata bulunursa tüm sınıfta buradan kopyala-yapıştır yapılan yerler aranıp her yerde düzeltme yapmak gerekecek.
  - Halbuki geçici değişkenlerle yapılan işlemler sorgu metotlarında toplanırsa, düzeltmeler de sadece sorgu metotlarında yapılacaktır.
- Tartışma: Bu düzenleme başarımı düşürür.
  - Sadece işlem yapmak yerine bir de fazladan metot çağırma yükü geliyor.
- Şerh:
  - Modern derleyiciler bu ek yükü ortadan kaldırmış veya önemsizleştirmiştir.

287

## REFACTORING – REPLACE TEMP WITH QUERY

### GEÇİCİ DEĞİŞKEN YERİNE ERİŞİM METODU KULLANIMI

- Tartışma:
  - Geçici değişken metot içerisinde birden fazla yerde kullanılıyorsa aynı işlem tekrar tekrar yapılacak.
  - Bu da başarımı daha da fazla düşürecek

```
void printRecipt() { /**/ ÖNCE: /**/
 int totalAmount = 0, thisAmount;
 for(Item item : items) {
 thisAmount = item.getCharge();
 System.out.println(item.getName()+" "+thisAmount);
 totalAmount += thisAmount;
 } }
```

```
void printRecipt() { /**/ SONRA: /**/
 int totalAmount = 0;
 for(Item item : items) {
 System.out.println(item.getName()+" "+item.getCharge());
 totalAmount += item.getCharge();
 } }
```

288



## REFACTORING – REPLACE TEMP WITH QUERY

### GEÇİCİ DEĞİŞKEN YERİNE ERİŞİM METODU KULLANIMI

- Tartışma:
  - Az önceki durumda başarımlarım daha da düşecek.
- Şerh:
  - Başarımlarım, sistemin tüm özellikleri kodlandıktan sonra bir “profiler” ile değerlendirilmelidir.
  - Belki de sistemin darboğazı bu düzenlemenin sonucunda oluşmamıştır.
    - Hele ki geçici değişkenin sakladığı ifade çok karmaşık değilse.
  - Aksi halde geçici değişkeni tekrar ortaya çıkartırsınız, olur biter.

289

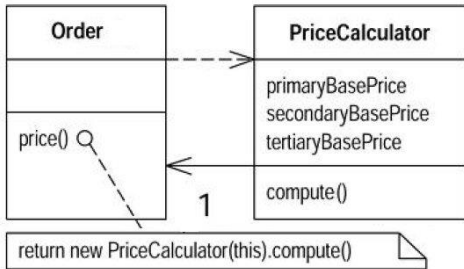
## REFACTORING – REPLACE METHOD WITH METHOD OBJECT

### METOD YERİNE YENİ TÜRDEN NESNE KULLANIMI

- Metot öyle uzun ve geçici yerel değişkenleri öyle arap saçı gibi kullanıyor ki, “extract method” düzenlemesini kullanamıyorsunuz.
- O zaman yeni bir sınıf oluşturup bütün metodu o sınıfa taşıyın, yerel değişkenleri de o sınıfın üyeleri yapın.
- Ardından bildiğiniz düzenleme eylemlerini daha rahat yürütebilirsiniz.
- Yeni sınıfın adını verirken, taşınan metodun adına benzer bir ad verin.

```
/** ÖNCE: */
class Order {
 double price() {
 double primaryBasePrice;
 double
 secondaryBasePrice;
 double tertiaryBasePrice;
 // long computation;
 ...
 }
}
```

/\*\* SONRA: \*/



290

### CODE SMELLS – LARGE CLASS

#### AŞIRI BÜYÜK SINIF

- Sınıfın büyük olması, sınıfa ilgisiz sorumluluklar atandığının göstergesidir.
  - Böylece uyum düşecektir.
- Üye alan sayısının fazlalığı, aşırı büyük sınıfın bir göstergesidir.
- Çözüme götürecek düzenlemeler:
  - Extract Class: Birlikte anlam ifade eden üye alanlardan bir veya daha fazla yeni sınıf oluşturulur.
  - Extract Subclass: Bazen oluşturulacak yeni sınıfın, mevcut sınıfın bir alt sınıfı olması daha uygun olabilir. Örneğin mevcut sınıf bazı üyelerini her zaman kullanmıyorsa, bunlar yeni bir alt sınıfa aktarılabilir.
  - Duplicate Observed Data: Eğer iş mantığı ile kullanıcı arayüzü arasında yüksek bağımlılık varsa, GUI sınıfları çok büyüyecektir. Bu durumda veri yeni bir uygulama alanı sınıfına kopyalanır ve aradaki eşgüdüm Observer tasarım kalıbına göre sağlanır.

291

### REFACTORING – EXTRACT CLASS

#### SINIF ÇIKARTMA

- Birbiri ile ilgisiz iki iş yapan bir sınıfı, bu işlere göre ikiye ayırmaktır.
- İkinci iş ile ilgili üyelerin seçilerek yeni bir sınıfa taşınmasına dayanır.
- Bu yapılandırma da bir çok kod kusurunu düzeltir.
- Örnek:

/\*\* ÖNCE: \*\*/

| Person             |
|--------------------|
| name               |
| officeAreaCode     |
| officeNumber       |
| getTelephoneNumber |

⇒

/\*\* SONRA: \*\*/

| Person             |
|--------------------|
| name               |
| getTelephoneNumber |

| Telephone Number   |
|--------------------|
| areaCode           |
| number             |
| getTelephoneNumber |

officeTelephone → 1

292

146

REFACTORING – EXTRACT SUBCLASS

ALT SINIF ÇIKARTMA

- Bir sınıfın bazı üyeleri her zaman kullanılmıyorsa, bu üyelerden yeni bir alt sınıf oluşturmak daha doğru olacaktır.
- Örnek:

/\*\* ÖNCE: \*\*\*/

/\*\* SONRA: \*\*\*/

Job Item

getTotalPrice  
getUnitPrice  
getEmployee

⇒

Job Item

getTotalPrice  
getUnitPrice

⬆

Labor Item

getUnitPrice  
getEmployee

293

REFACTORING – DUPLICATE OBSERVED DATA

GÖZLENEN VERİNİN ÇOĞULLANMASI

- Uygulama alanı ile ilgili bir verinin sadece bir GUI sınıfında saklanması doğru değildir.
- Bu veri uygulama alanı/bilgi düzeyi katmanında oluşturulacak yeni bir sınıfa taşınmalı ve,
- GUI nesnesi ile yeni bilgi nesnesi arasında Observer tasarım kalıbı kurulmalıdır.

/\*\* ÖNCE: \*\*\*/

/\*\* SONRA: \*\*\*/

Interval Window

startField: TextField  
endField: TextField  
lengthField: TextField  
  
StartField\_FocusLost  
EndField\_FocusLost  
LengthField\_FocusLost  
calculateLength  
calculateEnd

⇒

Interval Window

startField: TextField  
endField: TextField  
lengthField: TextField  
  
StartField\_FocusLost  
EndField\_FocusLost  
LengthField\_FocusLost

⬆

Interval

start: String  
end: String  
length: String  
  
calculateLength  
calculateEnd

Observer

Observable

294

### CODE SMELLS – DIVERGENT CHANGE & SHOTGUN SURGERY

#### DEĞİŞİKLİKLERİN FARKLI NOKTALARDA YAPILMASININ GEREKMESİ

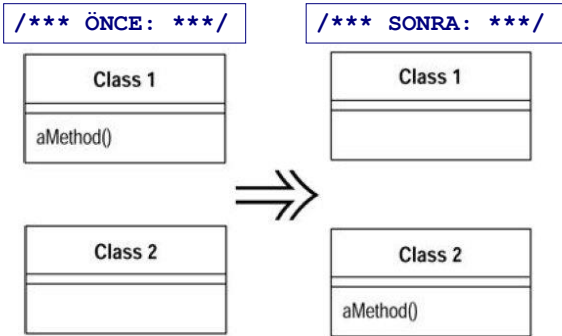
- Gereksinimlerde bir düzenleme olduğunda veya bir hatayı düzeltmek için kodda değişiklikler yapılması gerekecektir.
- Bu değişikliklerin mümkün olduğunca az yerde yapılması tercih edilir.
- Halbuki bir gereksinim değişikliğini karşılamak için gerekli kod değişiklikleri aynı sınıfın farklı metotlarında yapılıyorsa (divergent change) veya birden fazla sınıfta yapılıyorsa (shotgun surgery), bu işte bir terslik var demektir.
- Divergent change için kötü koku örneği:
  - Yeni bir veritabanı ile çalışmak gerekirse şu sınıfın şu 3 metodu değişir
- Shotgun surgery için kötü koku örneği:
  - Yeni bir tür kredi söz konusu olunca şu 2 sınıftaki şu 5 metot değişmeli
- Çözüme götürecek düzenlemeler:
  - Extract Class: Söz konusu metotlar ve ilgili üyelerden yeni bir sınıf çıkartılır.
  - Move Method / Move Field: Söz konusu üyeler daha uygun bir mevcut sınıfa taşınır.

295

### REFACTORING – MOVE METHOD

#### METOT TAŞINMASI

- Bir metot tanımlandığı sınıf içinden çok, başka bir sınıflardan çağrılıyorsa; en çok hangi sınıftan çağrılıyorsa o sınıfa taşınmalıdır.
- Bu metot ya eski sınıfından tamamen silinir, ya da eski sınıftaki hali sadece yeni sınıftaki halini çağırarak şekilde değiştirilir (delegation).
- Bu düzenleme de pek çok kokuyu giderdiğinden çok sık kullanılır ve bazı düzenlemelerin alt adımıdır (Ör: Sınıf çıkartma/Extract class).
- Örnek:



296

## CODE SMELLS – PRIMITIVE OBSESSION

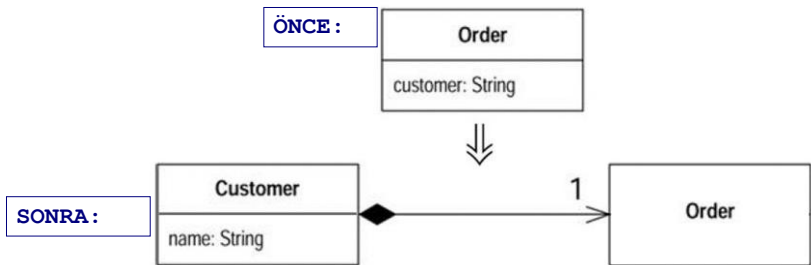
### İLKEL TİPLERİN AŞIRI KULLANIMI

- Yapısal programlamadan gelenler, ilkeleri aşırı yoğun olarak kullanır.
- Bu yoğun kullanımın farklı biçimlerini kelimeler ile tarif etmektense, her birine yönelik yeni düzenlemeleri incelemek tercih edilmiştir.
- Çözümüne götürecek düzenlemeler:
  - Replace Data Value with Object
  - Replace Type Code with Class
  - Replace Type Code with Subclasses
  - Replace Type Code with State/Strategy
  - Replace Array with Object
  - Extract Class ve Introduce Parameter Object düzenlemeleri de daha nesneye yönelimli bir program hazırlamak için kullanılabilir.

297

## REFACTORING – REPLACE DATA VALUE WITH OBJECT

- Bir üye alan, hele ki bu alan ile ilgili çeşitli davranışlar söz konusu ise, bir sınıf ile simgelenmelidir.
- Örnek:

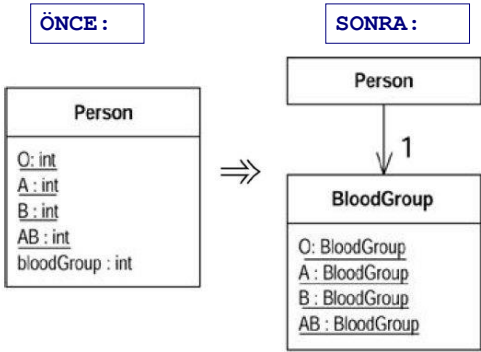


- Programın tümü incelendiğinde, belli bir müşterinin tüm siparişlerinin aranması gibi işlemler yapıldığı görülecektir.
- Bu işlemler çoğaldıkça, Müşteri sınıfı ayrılmadığı sürece, Sipariş sınıfı aşırı büyüyecektir.
- İyisi mi şimdiden bu sınıfları ayıralım.

298

### REFACTORING – REPLACE TYPE CODE WITH CLASS

- Yapısal programlamada sabit değişkenlerin kullanılması alışkanlığı ile ilgilidir.
- Örnek:



- Ya da kısaca enum kullanabilirsiniz.
  - Enum'u bilmeyen yoksa sonraki 2 yansıyı atlayabiliriz.

299

### REFACTORING – REPLACE TYPE CODE WITH CLASS

```
/** ÖNCE: */
class Person {
 public static final int O = 0;
 public static final int A = 1;
 public static final int B = 2;
 public static final int AB = 3;
 private int _bloodGroup;
 public Person (int bloodGroup) {
 _bloodGroup = bloodGroup;
 }
 public void setBloodGroup(int arg) { _bloodGroup = arg; }
 public int getBloodGroup() {
 return _bloodGroup;
 }
}
```

```
/** SONRA: */
class BloodGroup {
 public static final BloodGroup O
 = new BloodGroup(0);
 public static final BloodGroup A
 = new BloodGroup(1);
 public static final BloodGroup B
 = new BloodGroup(2);
 public static final BloodGroup
 AB = new BloodGroup(3);
 private static final
 BloodGroup[] _values = {O, A,
 B, AB};
 private final int _code;
 private BloodGroup (int code) {
 _code = code; }
 private int getCode() {
 return _code; }
 private static BloodGroup code
 (int arg) {
 return _values[arg]; }
}
```

- Dikkat: BloodGroup sınıfının tüm metotları private.

300

### REFACTORING – REPLACE TYPE CODE WITH CLASS

- Person sınıfı da yeni BloodGroup tipini kullanacak şekilde değiştirilir:

```
/** ÖNCE: */
class Person {
 public static final int O = 0;
 public static final int A = 1;
 public static final int B = 2;
 public static final int AB = 3;
 private int _bloodGroup;
 public Person (int bloodGroup)
 { _bloodGroup = bloodGroup; }
 public void setBloodGroup(int
 arg) { _bloodGroup = arg; }
 public int getBloodGroup() {
 return _bloodGroup;
 }
}
```

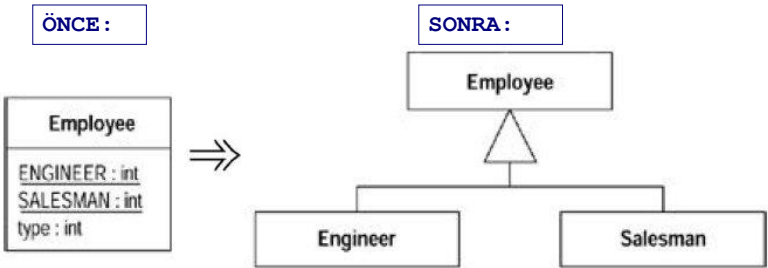
```
/** SONRA: */
class Person {
 private BloodGroup _bloodGroup;
 public Person (BloodGroup bg) {
 _bloodGroup = bg; }
 public BloodGroup
 getBloodGroup()
 { return _bloodGroup; }
 public void setBloodGroup (
 BloodGroup bg) {
 _bloodGroup = bg; }
}
```

- Not: Java'da enum sınıfları da tamamen bu şekilde çalışır ve daha sadedir. Java'da ilkel enum tanımı bile aslında bir enum sınıfı tanıımıdır.

301

### REFACTORING – REPLACE TYPE CODE WITH SUBCLASSES

- Bir sınıfın davranışını belirleyen bir ilkel tip kodu yerine kalıtımın kullanımı daha uygun olacaktır.
- Örnek:



302

### REFACTORING – REPLACE TYPE CODE WITH SUBCLASSES

```
/** ÖNCE: */
class Employee {
 private int _type;
 static final int ENGINEER = 0;
 static final int SALESMAN = 1;
 static final int MANAGER = 2;
 Employee (int type) {
 _type = type; }
 public void work() {
 switch(_type) {
 case ENGINEER:
 //do something
 break;
 case SALESMAN:
 //do something
 break;
 case MANAGER:
 //do something
 break;
 }
 }
}
```

```
/** SONRA: */
abstract class Employee {
 public abstract void work(){};
}
public class Engineer extends
 Employee {
 public void work() {
 //do something as an engineer
 }
}
public class Salesman...
```

- Not: Employee'nin önceki halinde case default varsa, sonraki halde Employee sınıfı abstract yapılmaz.
- Not: Bu örnek Replace Conditional with Polymorphism düzenlemesine de benzemektedir.

303

### REFACTORING – REPLACE TYPE CODE WITH STATE/STRATEGY

- Önceki örnekte kalıtım kullanımı bir nedenle mümkün değilse, State veya Strategy tasarım kalıbı da kullanılabilir.
- Örnek:

ÖNCE :

Employee

ENGINEER : int  
SALESMAN : int  
type : int

⇒

SONRA :

Employee

→ 1 → Employee Type

Employee Type

Engineer

Salesman

304



### REFACTORING – REPLACE ARRAY WITH OBJECT

- Çok boyutlu dizilerde kayıt tutmak yerine, tek bir nesne kullanın.
- Örnek:

```
/** ÖNCE: */
String[][] scores = new
 String[3][teamCount];
scores[0][0] = "Liverpool";
scores[1][0] = "15";
scores[2][0] = "67";
```

```
/** SONRA: */
Score[teamCount] scores;
scores[0] = new
 Score("Liverpool");
scores[0].setWins("15");
scores[0].setPoints("67");
```

305

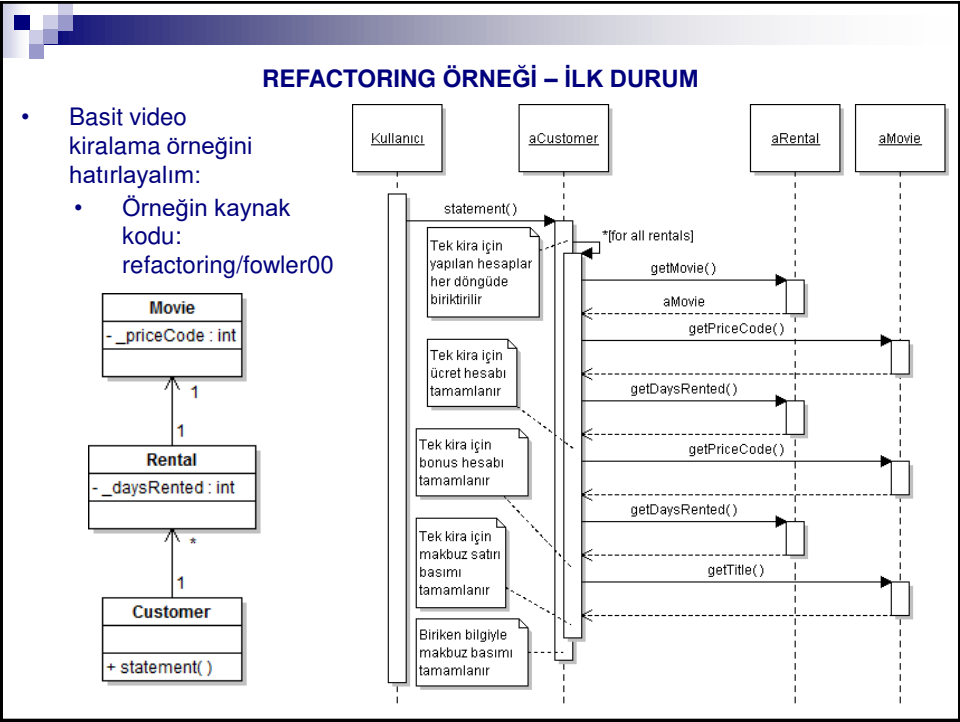
### CODE SMELLS – FEATURE ENVY

- Bir metod ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa, bu duruma 'Özellik kıskançlığı' adı verilir.
  - 'Çok' sıfatı ile o koddaki ortalama (veya %20 gibi bir eşikten fazla) farklı sınıf üyesi erişim sayısının üzerinde bir sayı ifade edilmektedir.
  - Sorumlulukların doğru atanmadığının göstergesi olabilir.
  - Elbette bağlaşım ilkesi nedeniyle başka sınıflardan nesnelere erişmek gerekecektir, bu nedenle hiçbir sınıf sadece kendi metotlarını kullanmaz. Öyle olsaydı bu kez de uyum ilkesini olumsuz yönde etkilemiş olurduk.
  - Strategy ve Visitor kalıpları bir özellik kıskançlığı durumu olarak yorumlanmamalıdır.
- Çözüm için olası refactoring eylemleri:
  - Move method
  - Extract method: Eğer kıskançlık metodun sadece bir kısmında ise o kısmı çıkartıp kıskanılan üyelerin bulunduğu sınıfa taşınabilir.

### BİR TASARIMIN ARDIŞIL DÜZENLEMELER İLE DEĞİŞTİRİLMESİ

- Şimdiye kadar incelenen düzenlemelerden bazılarının birlikte kullanılarak, yapısal programlama yaklaşımı izleri taşıyan bir tasarımın nasıl değiştirildiğinin bir örneği için Fowler'ın Refactoring kitabının 1. bölümü incelenebilir.

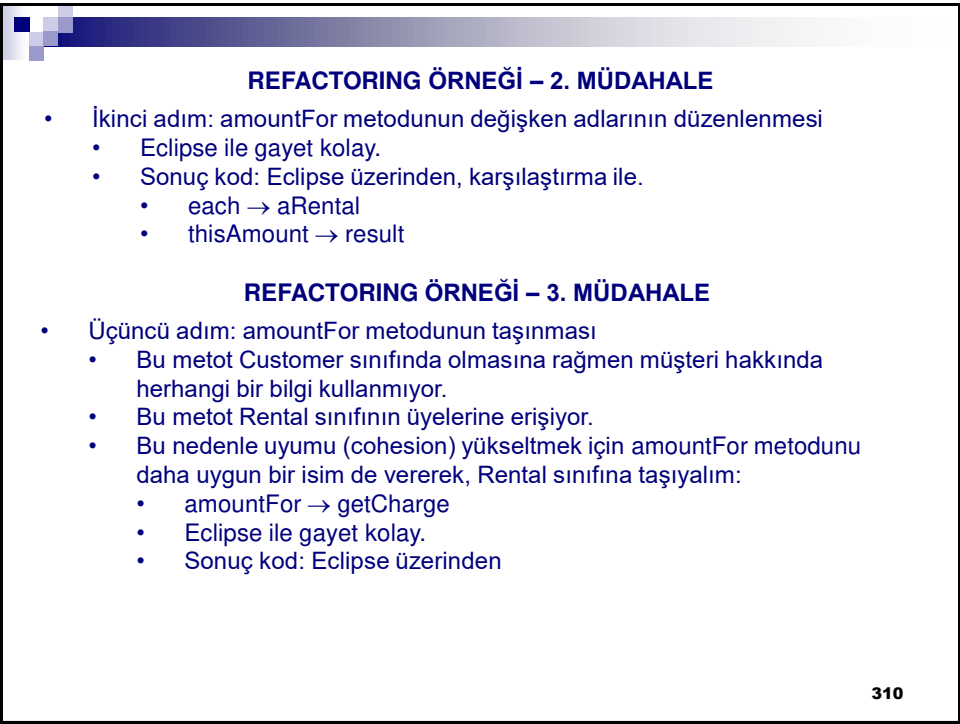
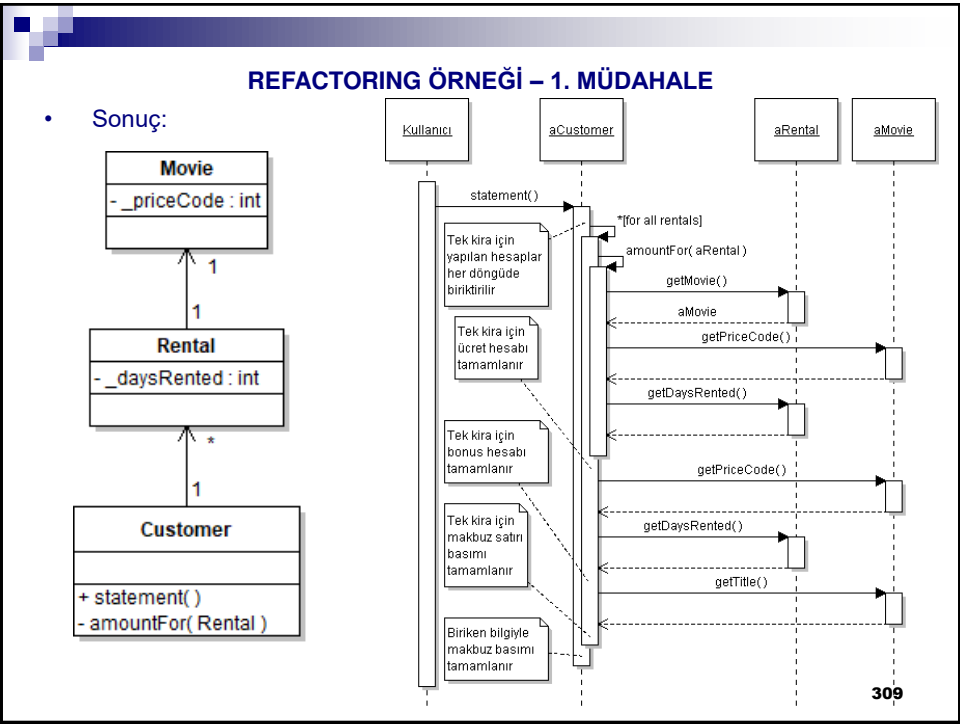
306

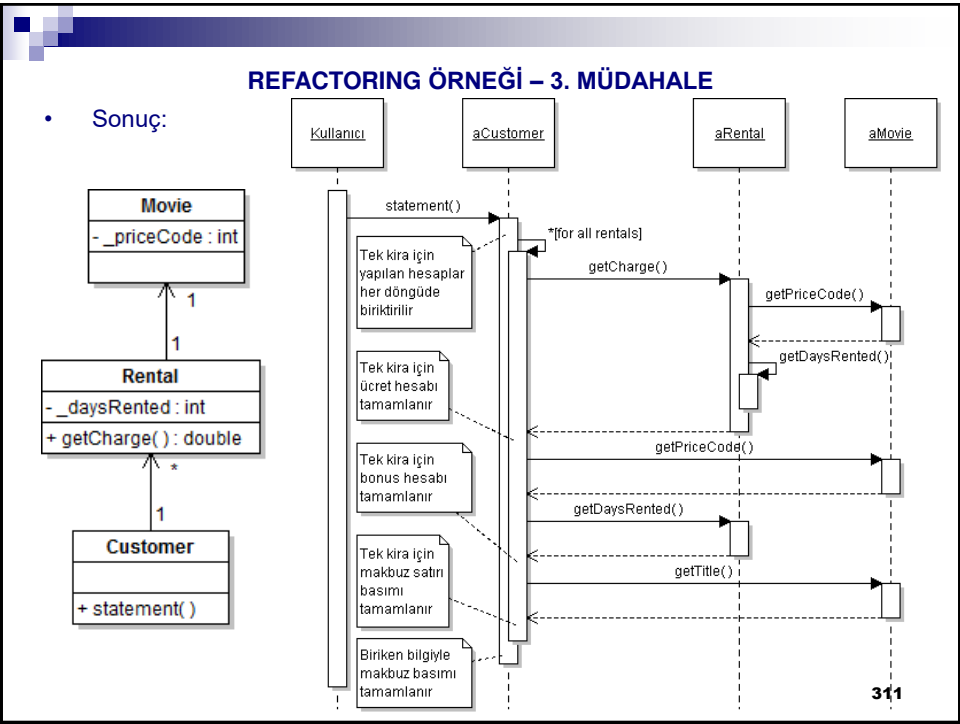


**REFACTORING ÖRNEĞİ – 1. MÜDAHALE**

- İlk adım: Aşırı uzun olan `statement` metodunu parçalamak
  - `statement` metodunun da en uzun parçası `switch` bloğu olarak gözüküyor.
  - Aslında `switch` kullanımı çokbiçimliliğe ihtiyaç duyulduğunun göstergesidir.
  - Ancak değişiklikleri küçük adımlarla yapmak, hata yapmamız halinde hatamızı düzeltmeyi kolaylaştıracaktır.
  - Sonuç olarak bu adımda `switch` bloğu, geçici değişkenlere de dikkat ederek, `amountFor` adlı bir metotta taşınmıştır:
    - Taşınacak alanda iki yerel değişkene erişim yapıldığı için, Eclipse düşünülen düzenlemeyi gerçekleştirememiştir.
    - Yerel değişkenler `thisAmount` ve `each` tanımlama sırasını değiştirince Eclipse de `extractMethod` işleminde başarılı oldu.
  - Sonuç kod: Eclipse üzerinden
    - `refactoring/fowler01` dizininde
  - Test case'lerimizi yeniden çalıştırmayı unutmayalım.
    - Sadece **Customer** sınıfında değişiklik yaptığımız için, **TestCustomer** birim testini çalıştırmak yeter.
      - `testing/fowler01` dizininde
      - Aslında test kodu değişmedi ancak **Customer** sınıfının yeni halini `fowler01` paketine aldığımız için **TestCustomer** sınıfı da bu yeni pakete kopyalandı.

**308**





**TESTLERDE KARA KUTU/BEYAZ KUTU YAKLAŞIMI**

- Kara kutu sınaması (Black-box testing): Sınanacak birimin iç işleyişi bilinmez, sadece birimin beklenen girdilere karşı beklenen çıktıları üretip üretilmediğine bakılır.
- Beyaz kutu sınaması (White-box testing): Sınanacak birimin iç işleyişi bilinir ve yapılacak sınamalar buna göre belirlenir.
- Tartışma:
  - Üçüncü adımın sonucunda, Rental sınıfı yeni bir metod kazandı.
  - Acaba Rental.getCharge metodu için yeni bir test case oluşturalım mı (beyaz kutu), yoksa mevcut test case'ler ile devam mı edelim (kara kutu)?
  - Beyaz kutuyu seçmek test kalitemizi artırır ancak sınırlı proje zamanımızın uzamasına neden olabilir.
  - Bu aşamada mevcut test case'ler ile devam etmeyi seçtim.

**312**

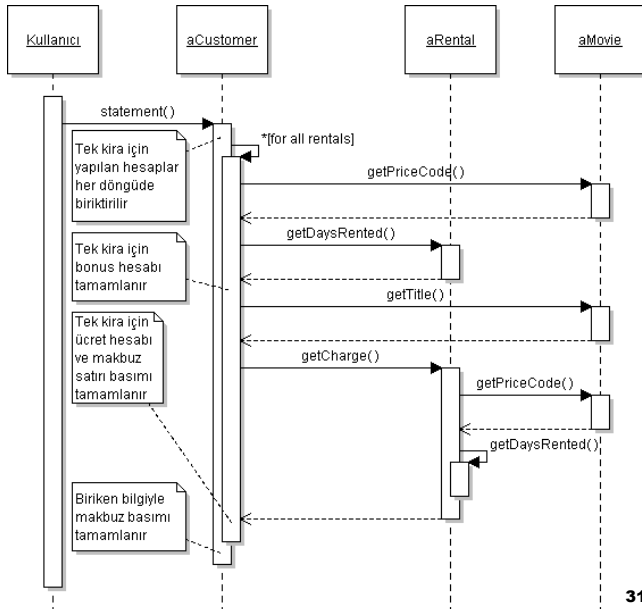
#### REFACTORING ÖRNEĞİ – 4. MÜDAHALE

- Dördüncü adım: statement metodundaki yerel thisAmount değişkeni gereksizdir.
  - Bu değişkene getCharge metodunun döndürdüğü değer atanmakta ve değişkenin geçerlilik süresince bu değişkenin değeri değişmemektedir.
  - Bu nedenle thisAmount yerine getCharge kullanılmıştır.
  - Bedel hesaplama daha ileride gerçekleştiği için etkileşim şeması değişecektir.
  - Sonuç kod: Eclipse üzerinden, karşılaştırma ile.

313

#### REFACTORING ÖRNEĞİ – 4. MÜDAHALE

- Sonuç:



314

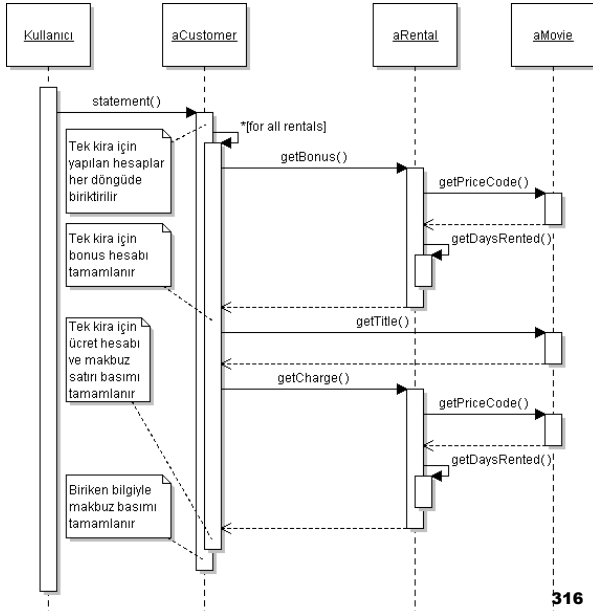
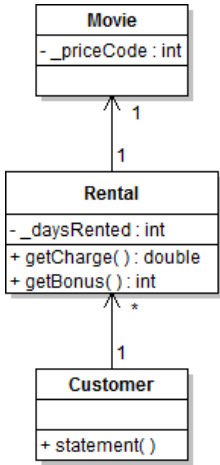
### REFACTORING ÖRNEĞİ – 5. MÜDAHALE

- Beşinci adım: Bonus hesaplaması için metot çıkartma işlemi.
  - Bonus (frequentRenterPoints) kiralama işlemi ile ilgili olduğu için, bu işlemin Rental sınıfına alınması yerinde olacaktır.
  - FrequentRenterPoints çok uzun → Bonus kullanıldı.
  - Sonuç kod: Eclipse üzerinden, karşılaştırma ile.
- UML şemaları hakkında:
  - Amacımız refactoring'i vurgulamak. Bu yüzden her getter/setter metodunu göstermedik.

315

### REFACTORING ÖRNEĞİ – 5. MÜDAHALE

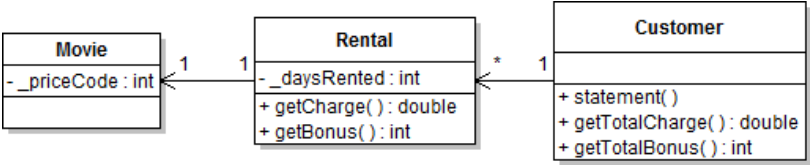
- Sonuç:



316

### REFACTORING ÖRNEĞİ – 6. MÜDAHALE

- Altıncı adım: statement metodundaki diğer yerel değişkenler olan totalAmount ve bonus'un kaldırılması
  - totalAmount yerine getTotalCharge() metodu eklendi
  - bonus yerine getTotalBonus() metodu eklendi
  - İlgili yerel değişkenler ilgili metotlara taşındı
  - Sonuç sınıf şeması:



317

### REFACTORING ÖRNEĞİ – 6. MÜDAHALE

- Sonuç etkileşim şeması: Dosya üstünden.
  - Şema fazla uzamaya başladı, ancak bunun nedeni kod fazlalığından ziyade, eski durumlarda ilkeller ve geçici değişkenler üzerinden yapılan işlemlerin artık metotlar üzerinden yapılmaya başlanmasıdır.
  - Anlamlı metot adları ile açıklama kutularının azalmasına dikkat.
    - Tasarımın anlaşılabilirliği artmıştır.
- Sonuç kod: Eclipse üzerinden, karşılaştırma ile.

318

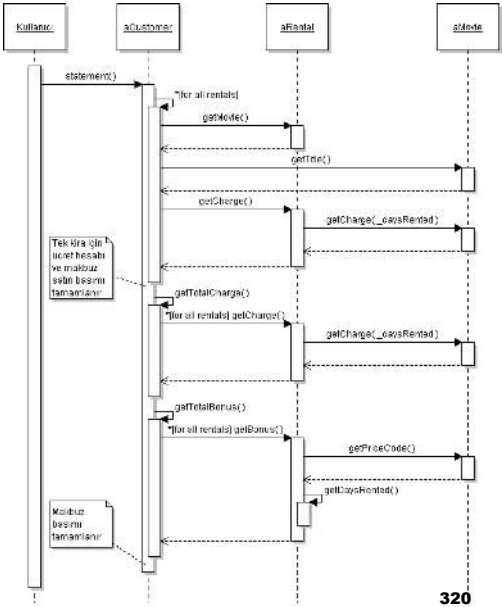
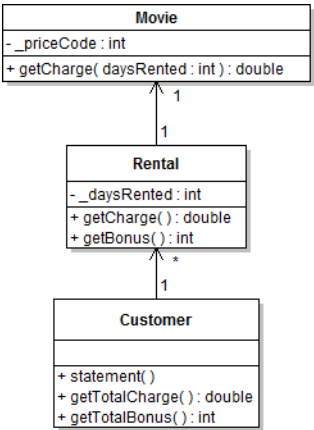
### REFACTORING ÖRNEĞİ – 7. MÜDAHALE

- Yedinci değişiklik: getCharge metodunun bölünmesi
  - getCharge metodu Rental üyelerine erişiyor diye 3. adımda bu sınıfa taşındı.
  - Ancak bu metot aynı zamanda Movie üyelerine de erişiyor.
  - Üstelik bu erişim switch-case bloğunda kullanılıyor.
  - Bir başka sınıfın üyelerinin durumlarına switch-case bloğunda erişmek akıllıca olmaz:
    - Bu başka sınıfa yeni durumlar eklenince bu durumun işlendiği tüm sınıflarda değişiklik gerekecektir.
  - Çözüm: getCharge metodunun bir kısmının Movie sınıfına taşınabilecek şekilde bölünmesi.
    - Öyle ki, her sınıfta sadece o sınıfın üyelerine doğrudan erişilsin.
    - Filmin kaç gün kiralandığı bilgisi Rental sınıfında, film türü ise Movie sınıfında saklanmaktadır.
    - Demek ki, bölme işlemi de bu şekilde yapılmalıdır.

319

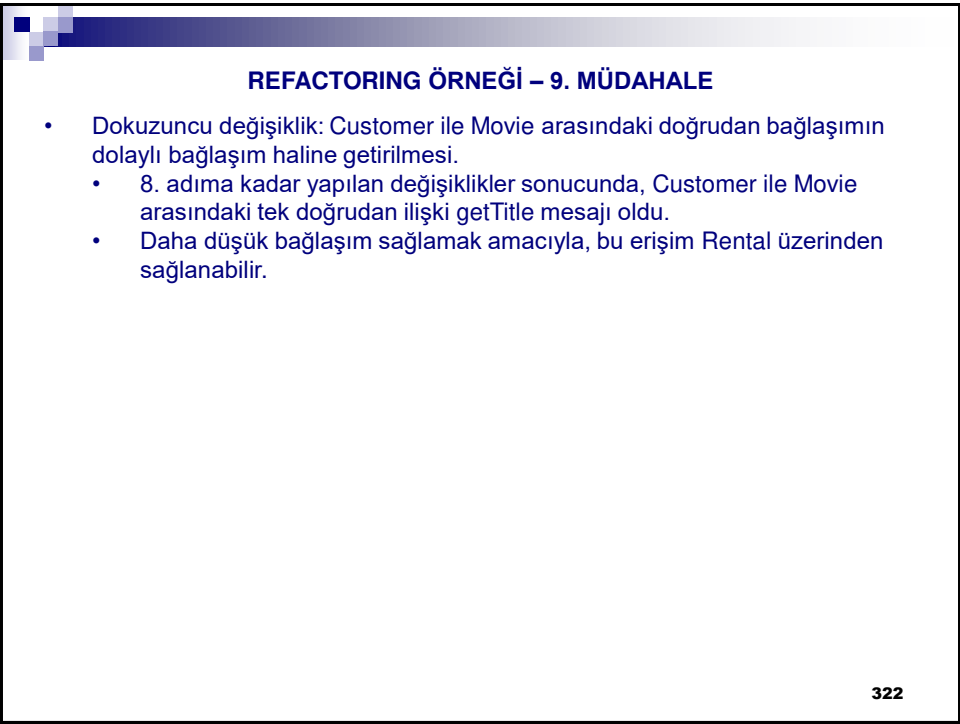
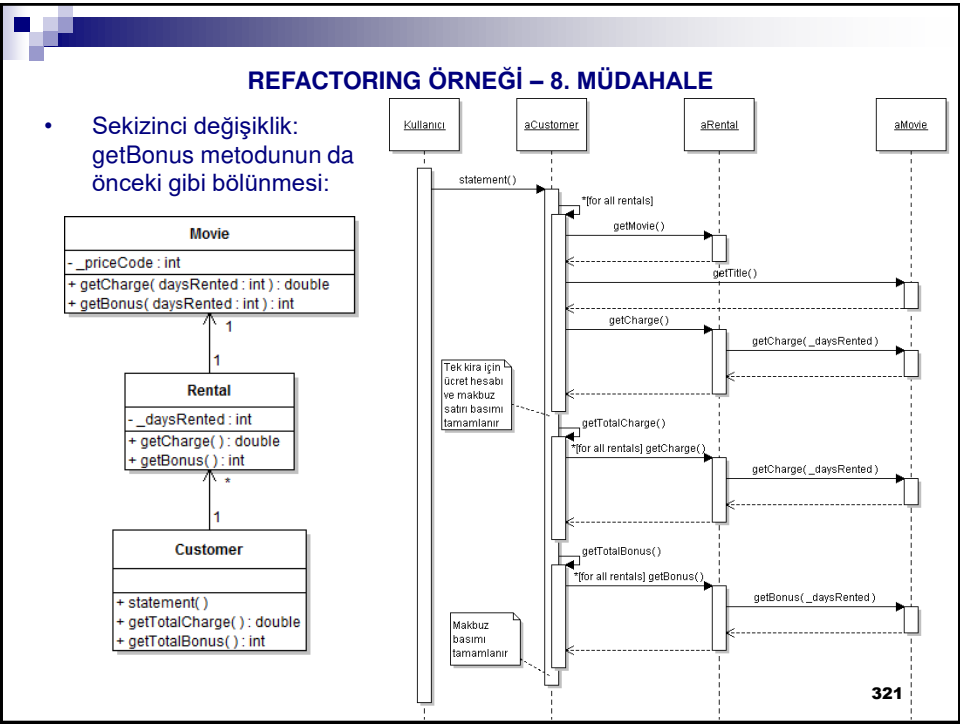
### REFACTORING ÖRNEĞİ – 7. MÜDAHALE

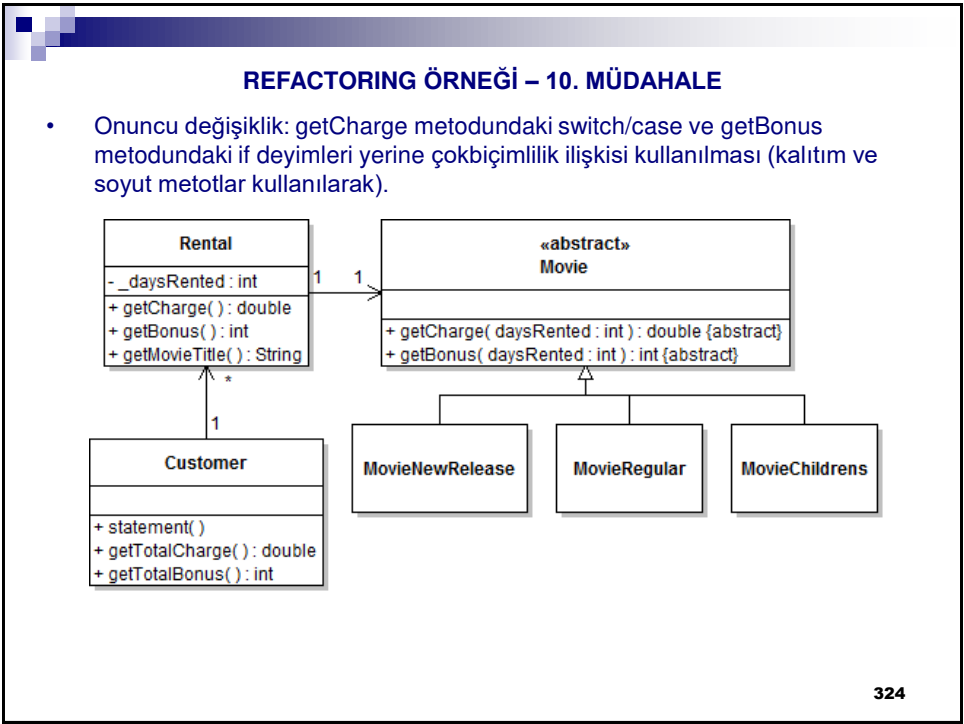
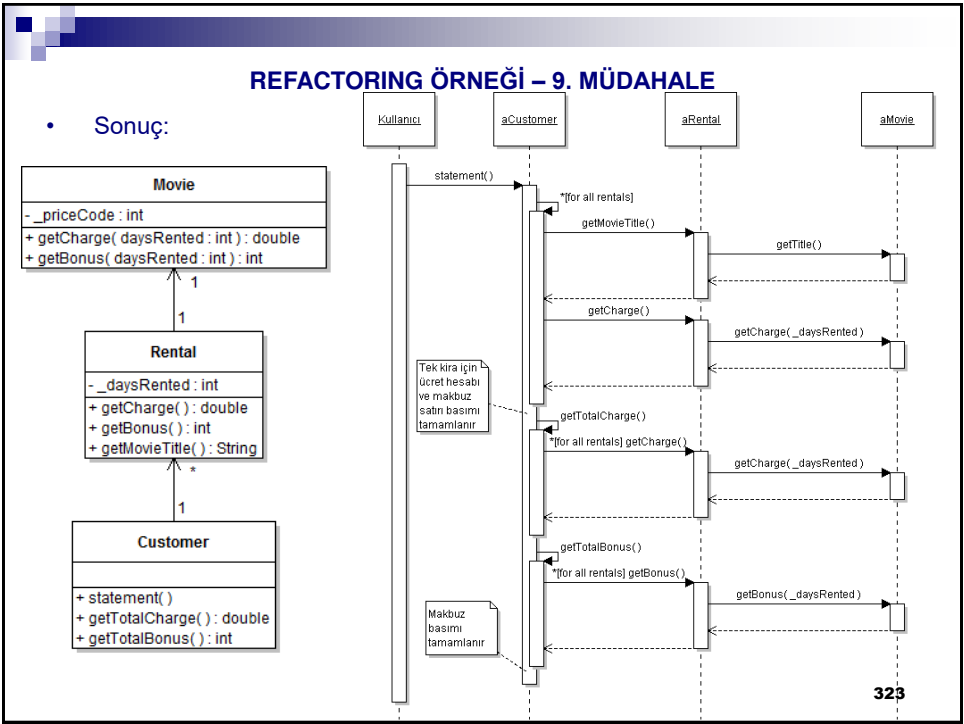
- Sonuç:



320







### ANTI PATTERNS (KARŞIT KALIPLAR)

#### ANTIPATTERN NEDİR?

- Karşıt kalıp çözüm bir tasarım kalıbı gibi görünür ancak öyle değildir.
- Karşıt kalıplar aslında tasarım kalıplarının karşıtları olarak yazılım geliştirme sürecinde tekrar eden bazı yanlışları ifade ederler.
- Karşıt kalıpları bilmek, yazılım geliştirme sürecinde karşılaşılabilecek ciddi problemleri önceden tahmin edebilmeyi ve tedbir almayı kolaylaştırır.

#### KARŞIT KALIP İLE KOD KUSURU FARKI:

- Karşıt kalıplar tasarım düzeyine, kod kusurları gerçekleştirme düzeyine daha yakındır.
- Karşıt kalıplar yazılım yaşam döngüsünün diğer evreleri ile de ilişkilidir.

#### KARŞIT KALIP TÜRLERİ

- Karşıt kalıplar üç ayrı grupta incelenmektedir: (henüz müfredata eklenmedi)
  - Yazılım geliştirme karşıt kalıpları
  - Yazılım mimarisi karşıt kalıpları
  - Yazılım proje yönetimi karşıt kalıpları

325